Concepts

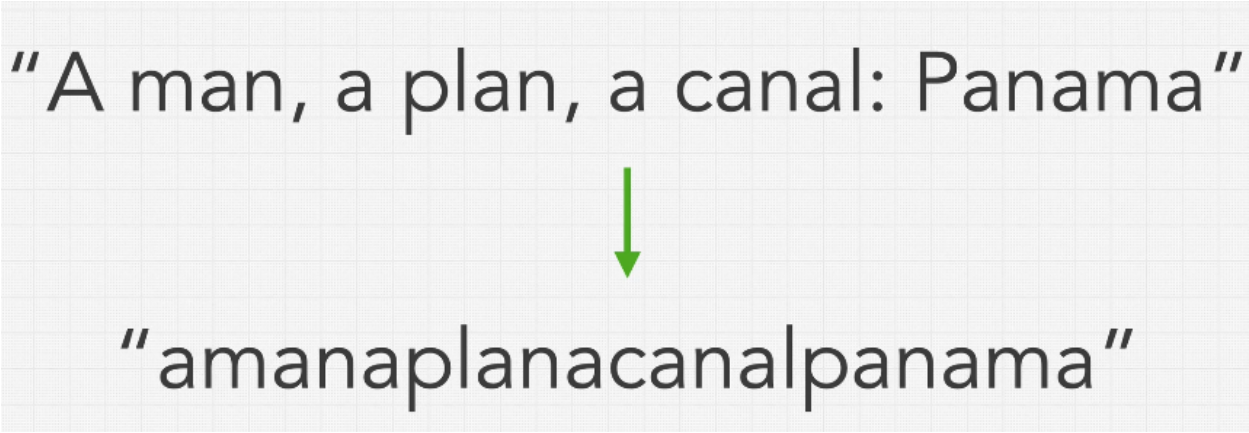
Palindrome

A palindrome is a string that reads the same forwards and backwards.

"aba" "a" "race_car" ""

"A_man,_a_plan,_a_canal:_Panama"

Only focus the content



Subproblems

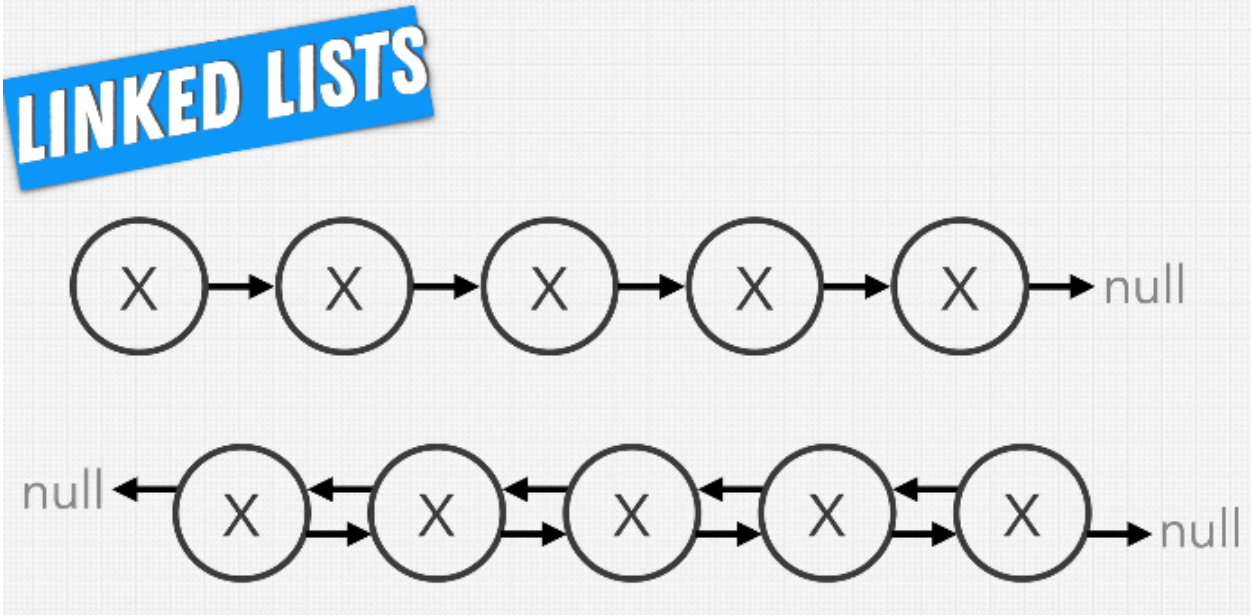
A subproblem is a problem we have to solve along the way to solving the main problem.

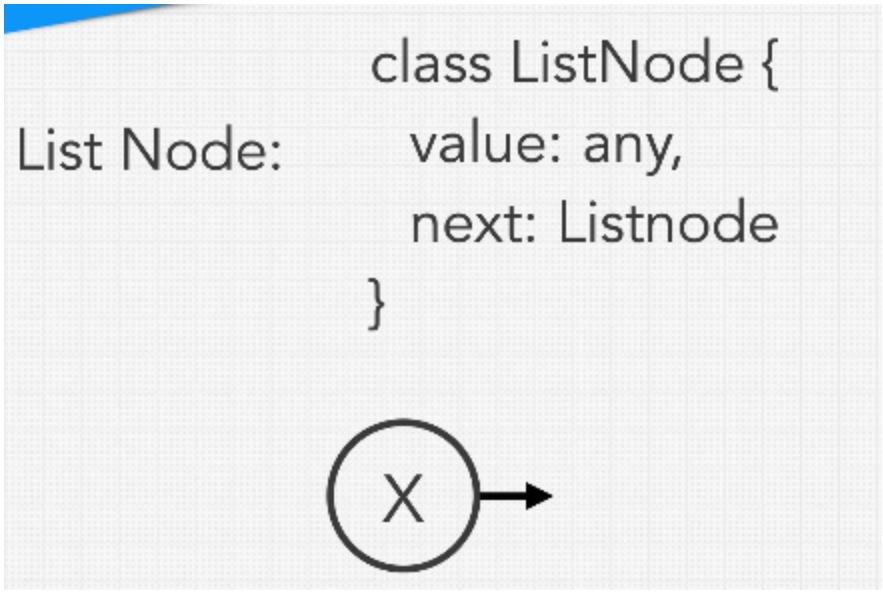
Given a string, find the length of the longest substring without repeating characters.

main problem: find length of longest unique substring

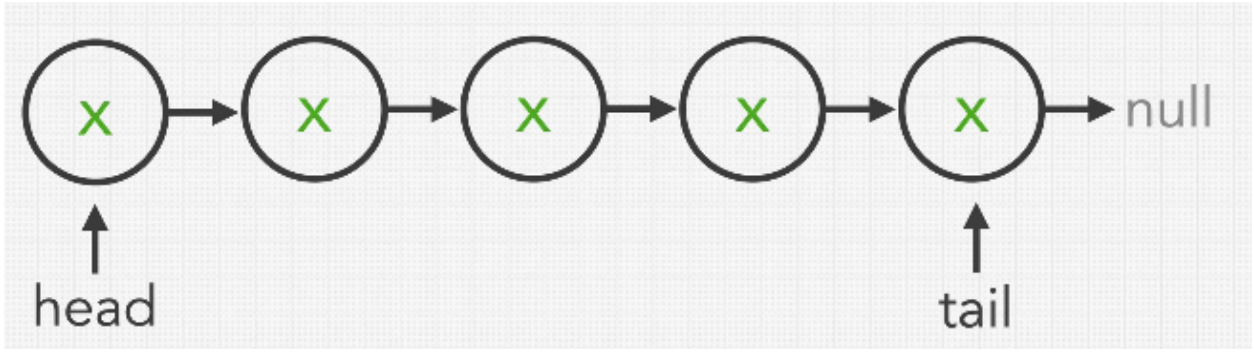
sub problem: pattern matching - unique substring

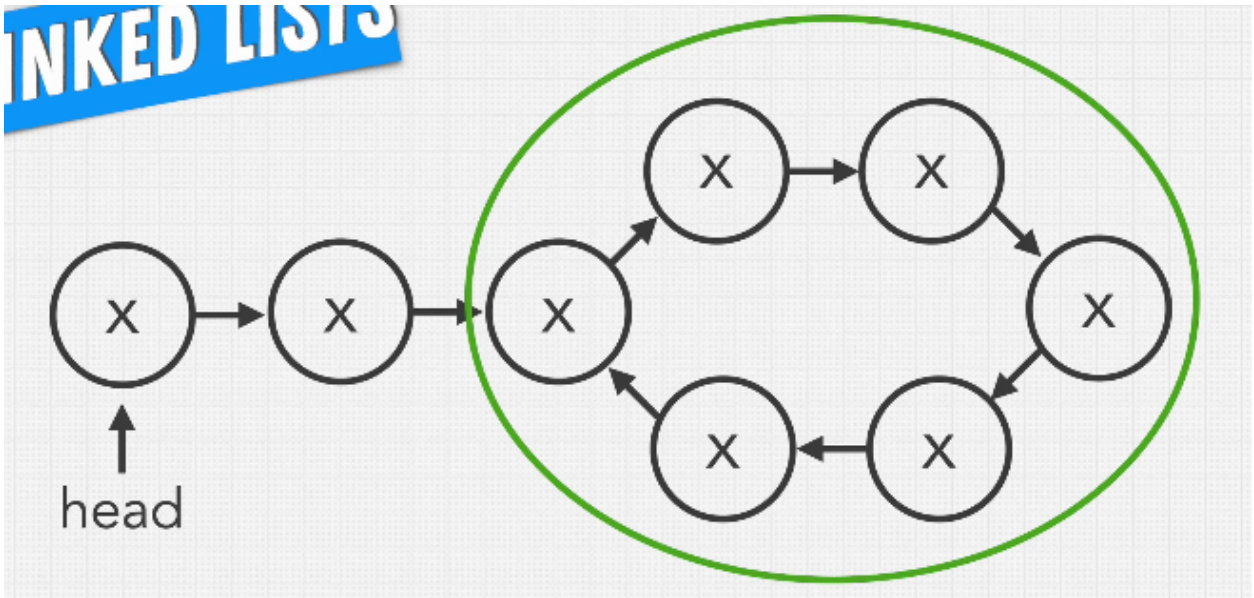
Linked lists

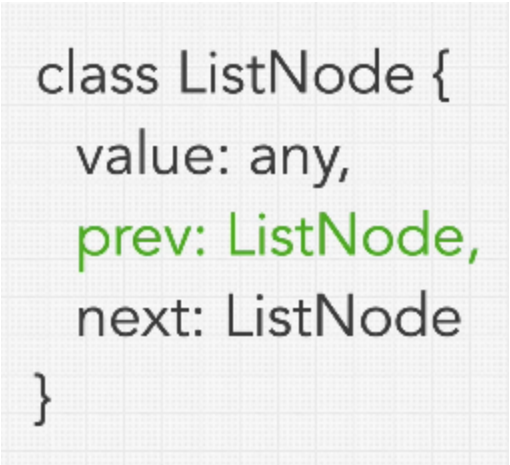




The value can hold anything







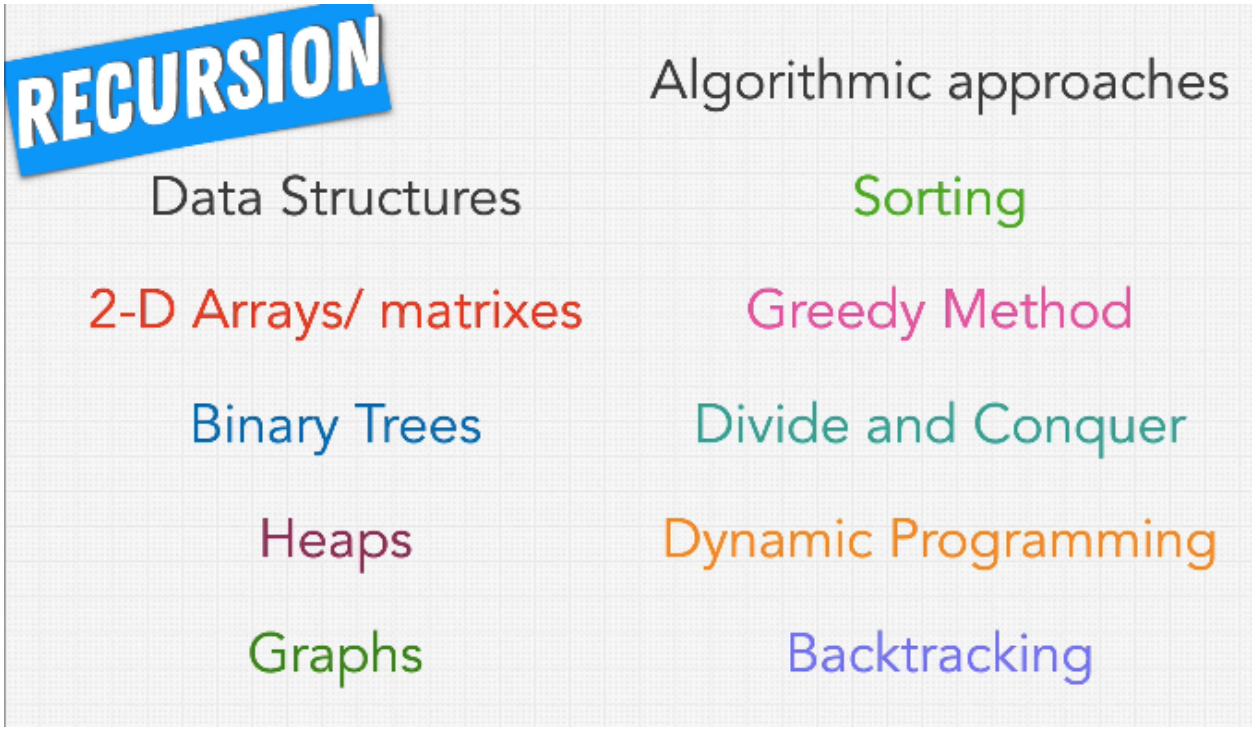
Stacks & queues

Stacks and queues are usually used within a big problem to solve sub problems.

This points to the order of how we get the data

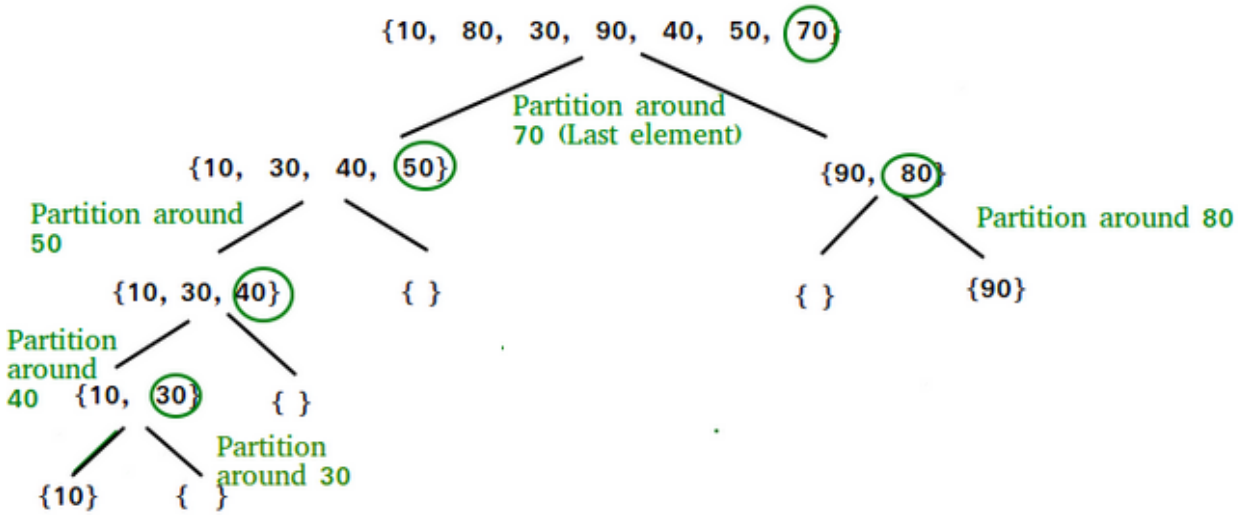
Recursion

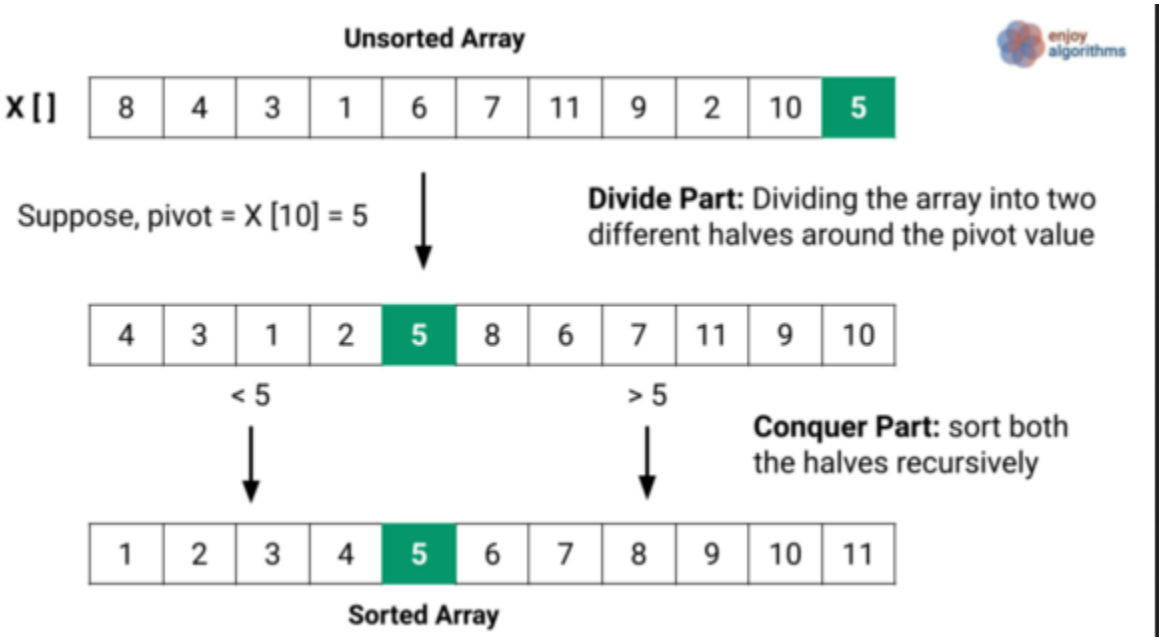
Approaches



Quick sort

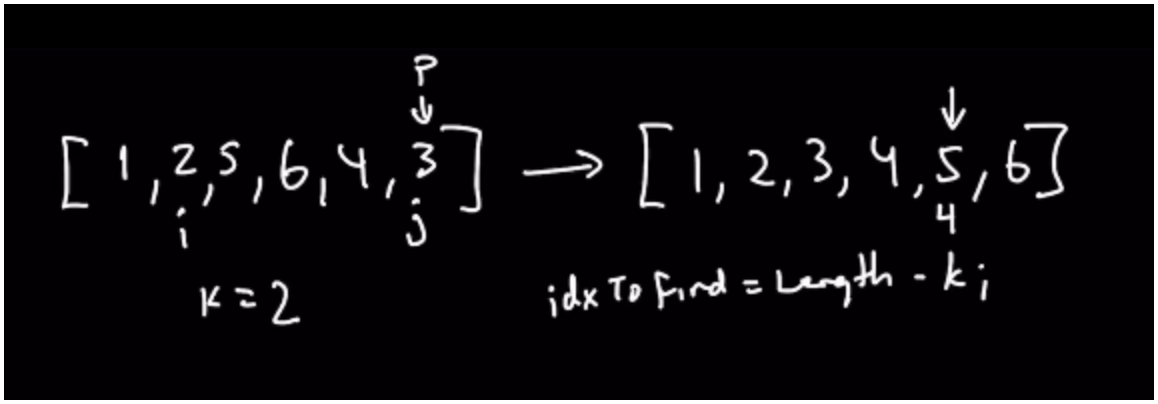
Sort the array in place



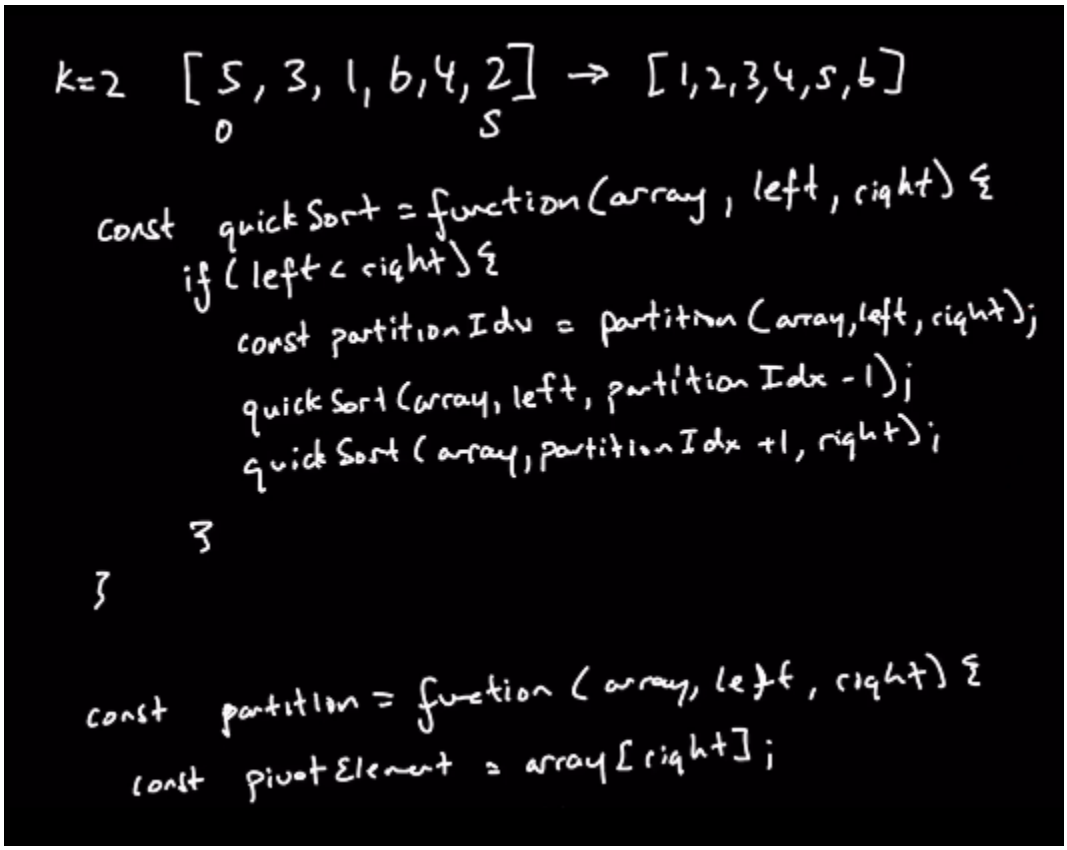


We sort in place, we don't use other array

We swap values to put higer values to right



To do this we use three pointers, the pointer i is in place, the j pointer is moving on and when detects a smaller value of i, it swaps



Binary tree

Breadth First Search

Depth First Search

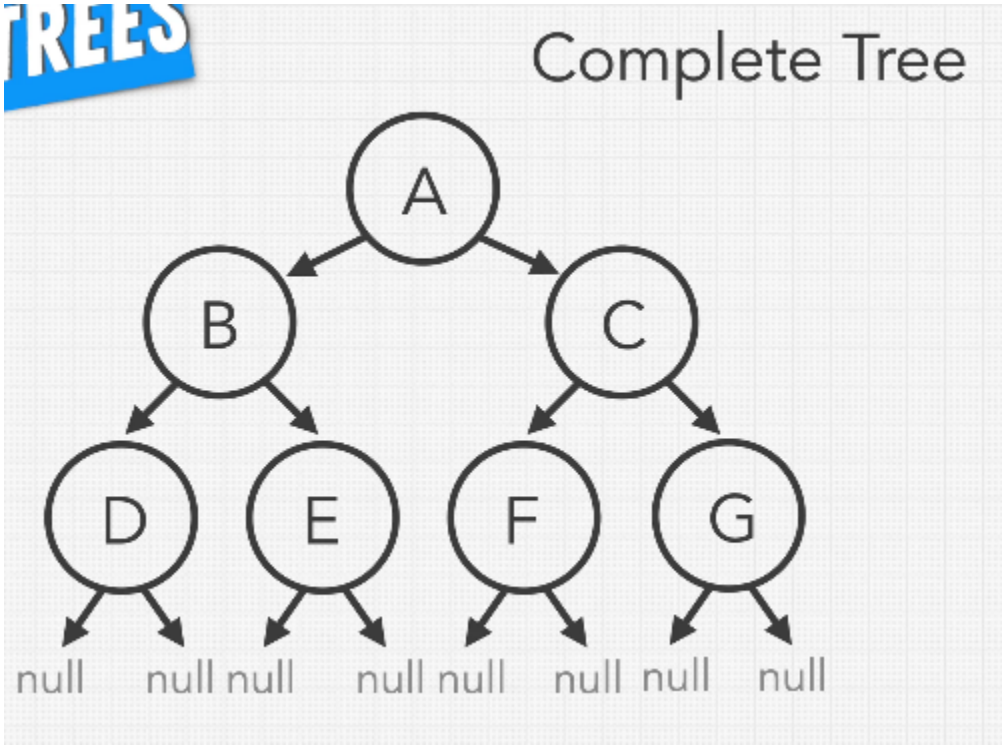
Pre-order traversal

In-order traversal

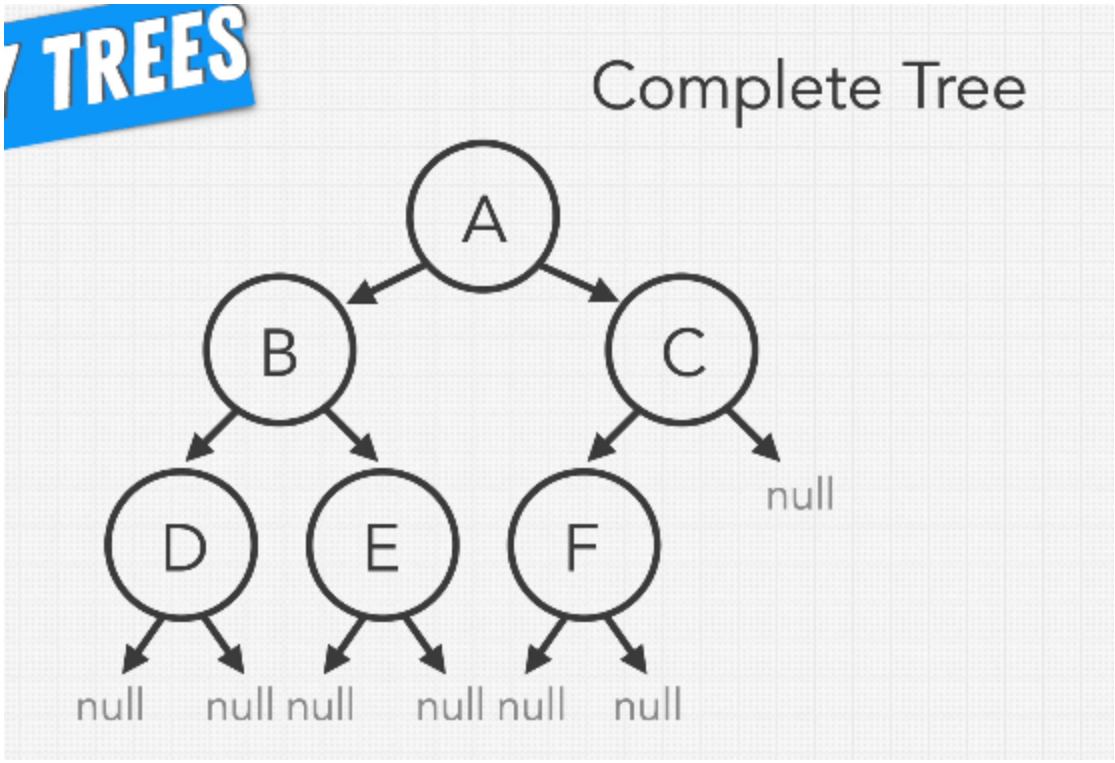
Post-order traversal

Full tree -> all nodes has either 2 or 0 childrens

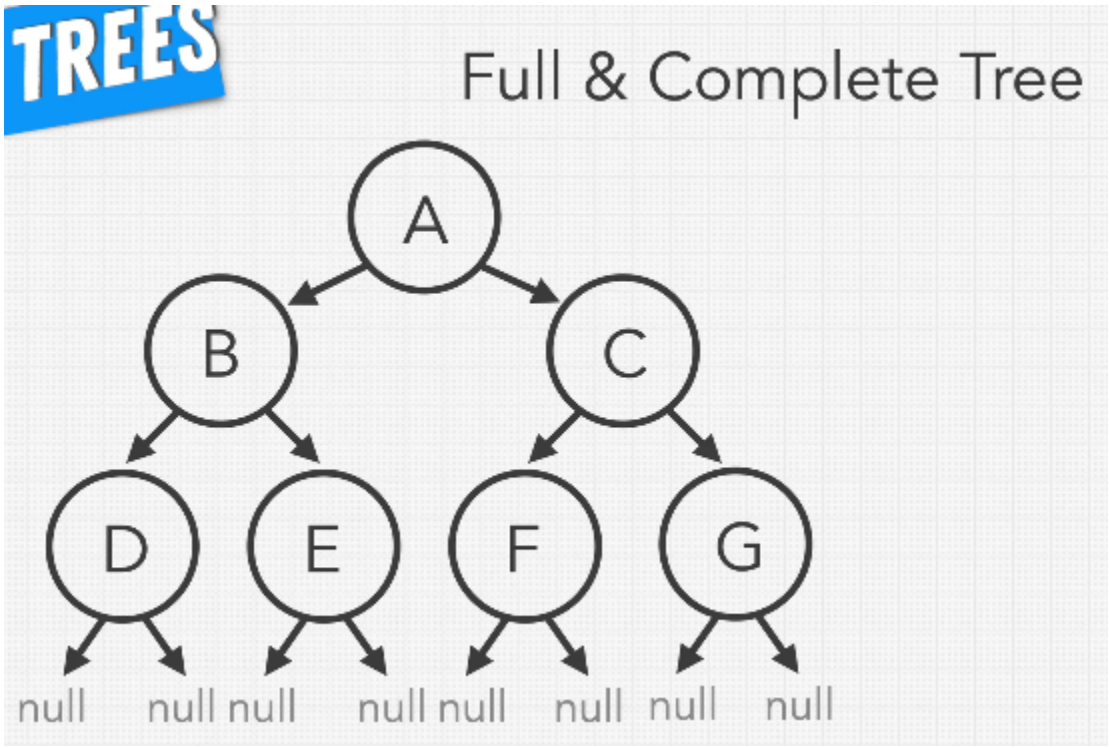
Complete tree -> every level is completely fill



But the final level can not be totally fill, it should be the most left posible and the rest of the right can be null



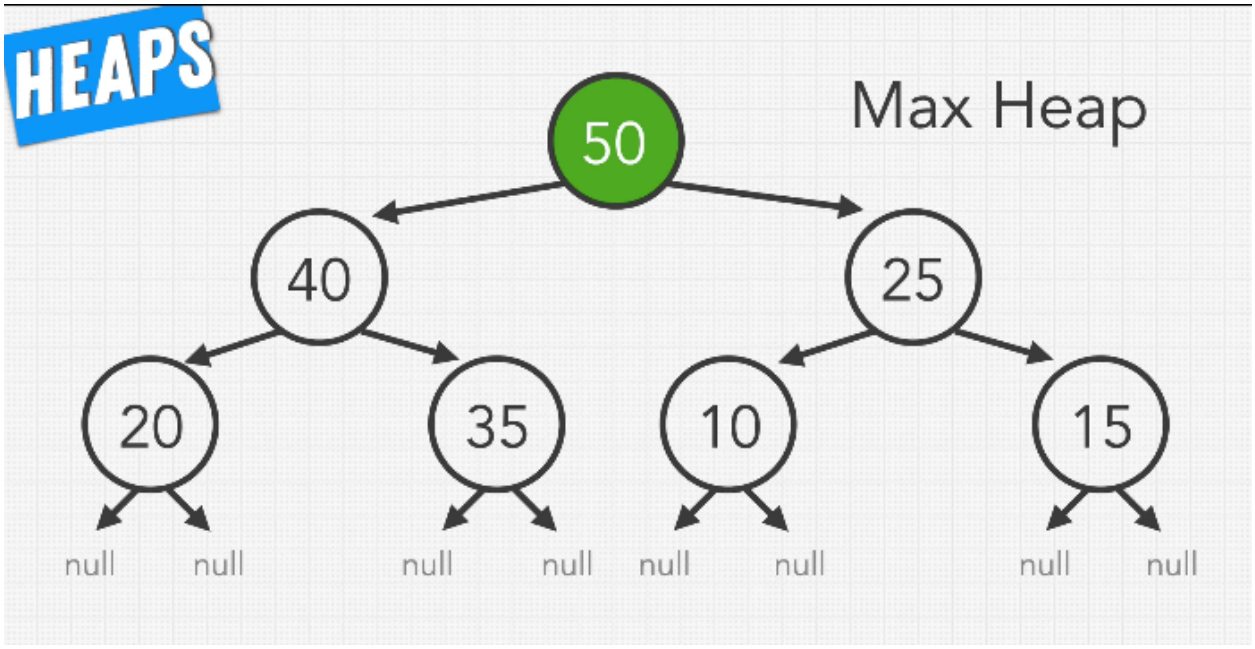
Full & complete is all



Heaps

Assembles a Complete binary tree

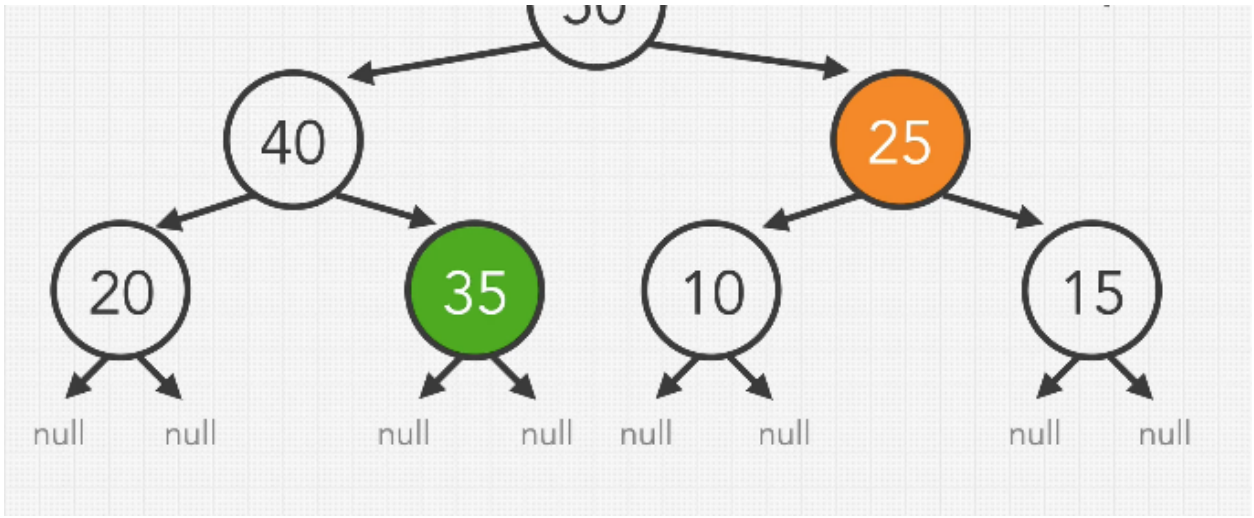
Push to the left as possible



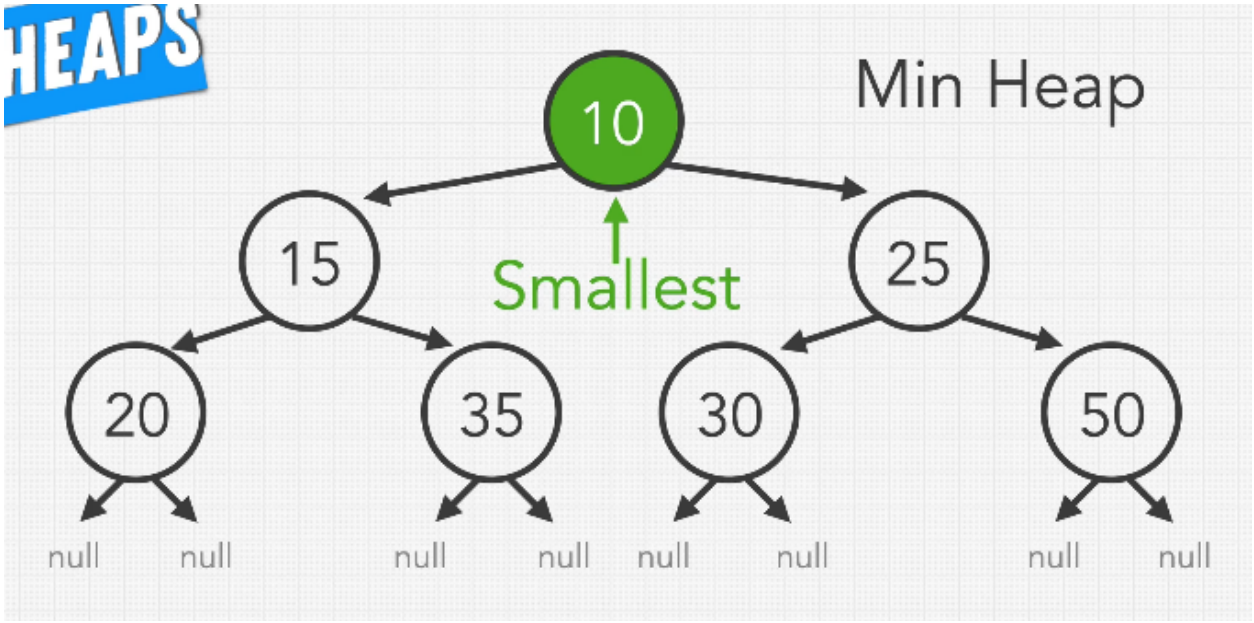
Max heap .- every single child should be smaller than parent

The root is the greatest value

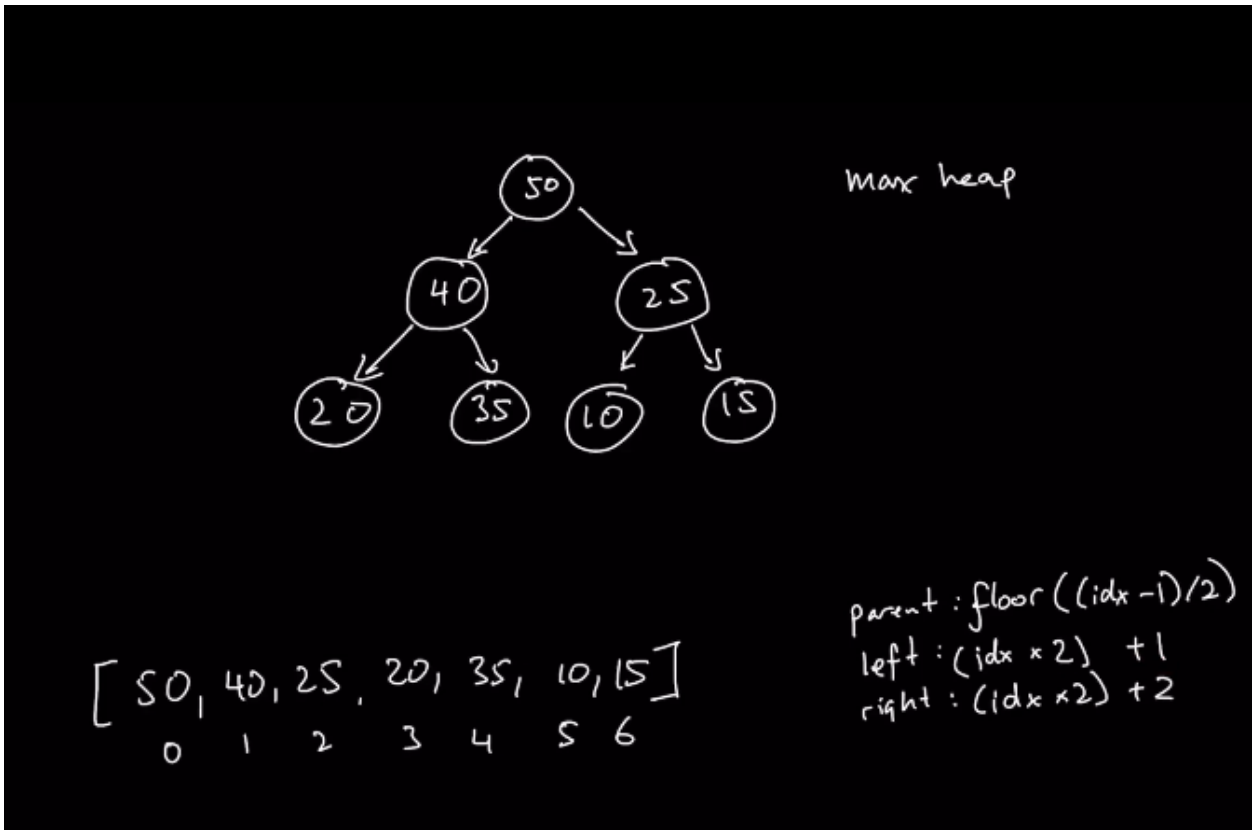
The second child of the root is not necessarily the third highest value



Min heap.- the children must be higher of his parent



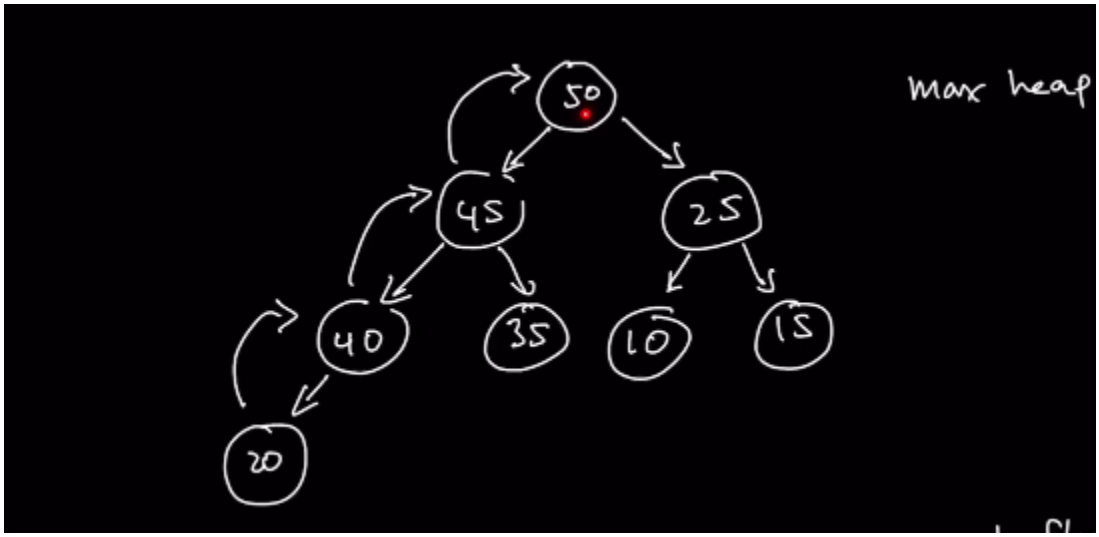
To find a parent of a index we use



It has this properties

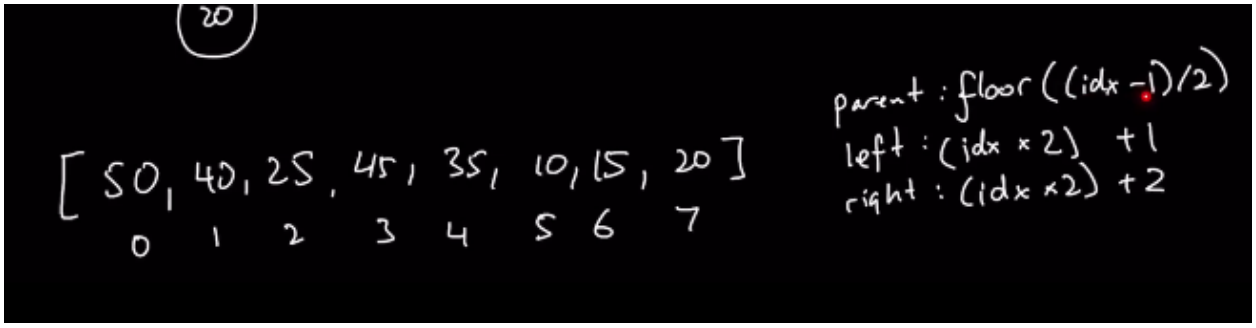
Inserting

To add an element we use “Sift up”

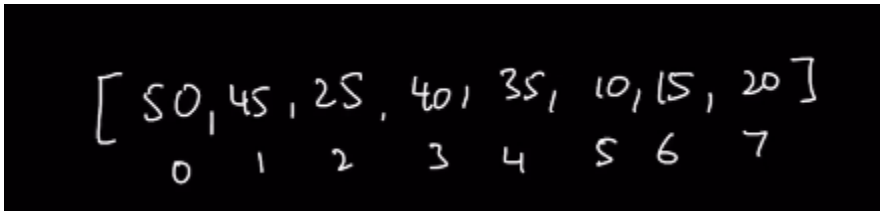


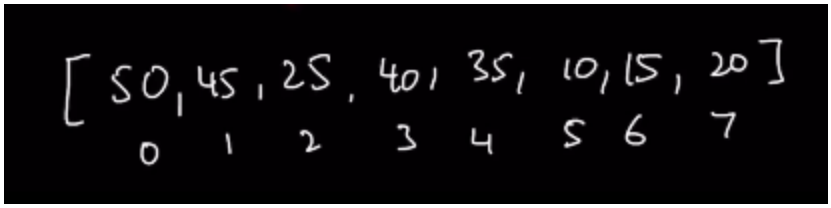
The 45 is being compared from bottom swapping positions

For array the same



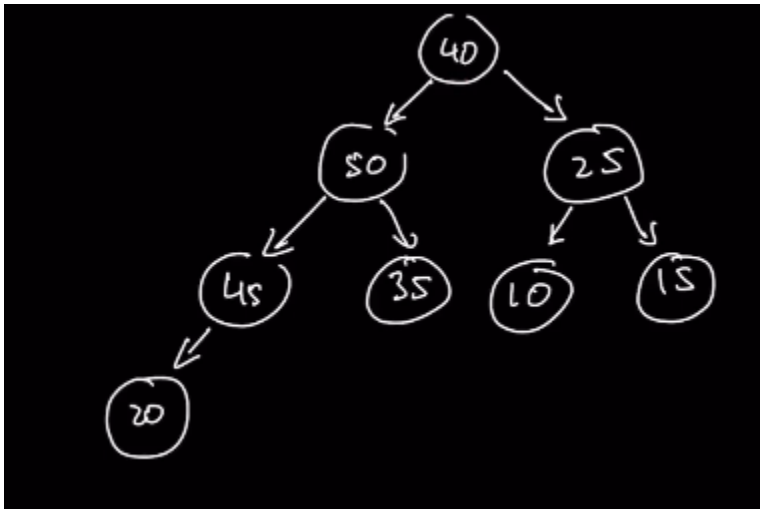
We swap using the parent formula





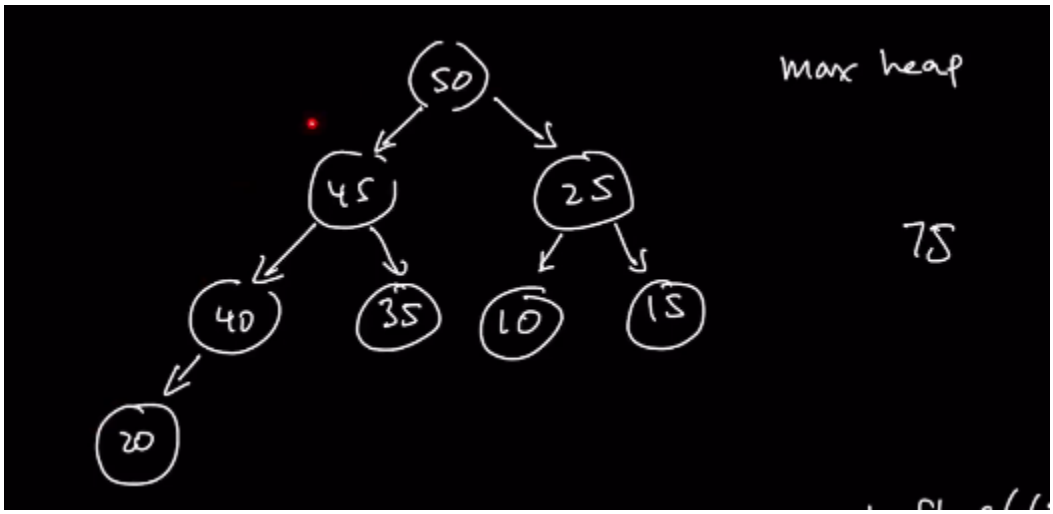
Remove

Example removing 75

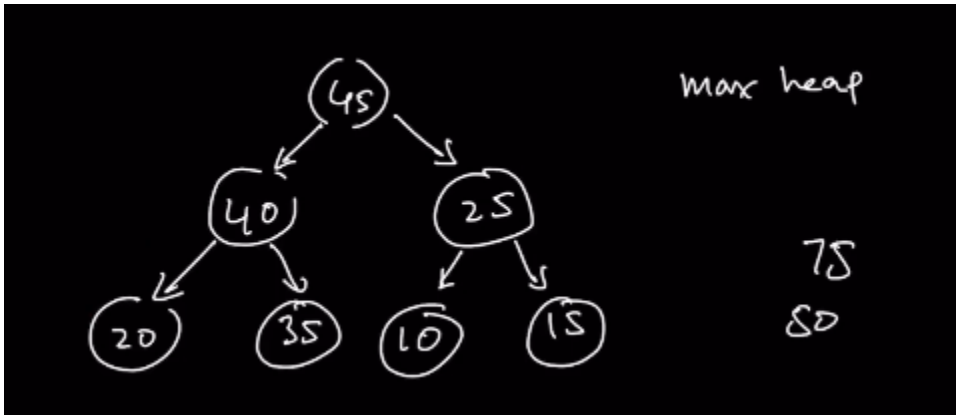


We get the most right value of the binary tree

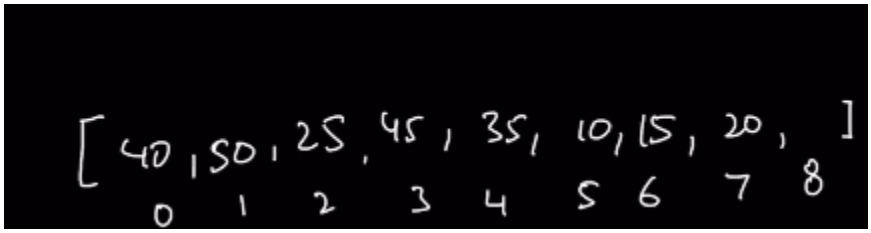
Comparing for the greatest we start from left

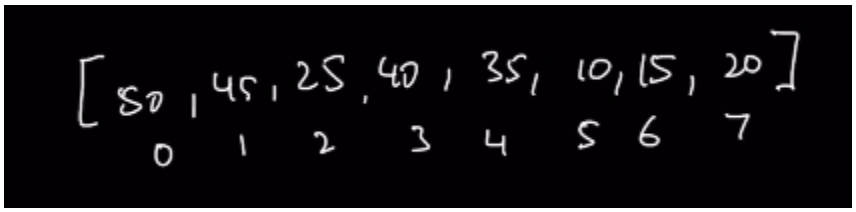


Removing 80:



For array

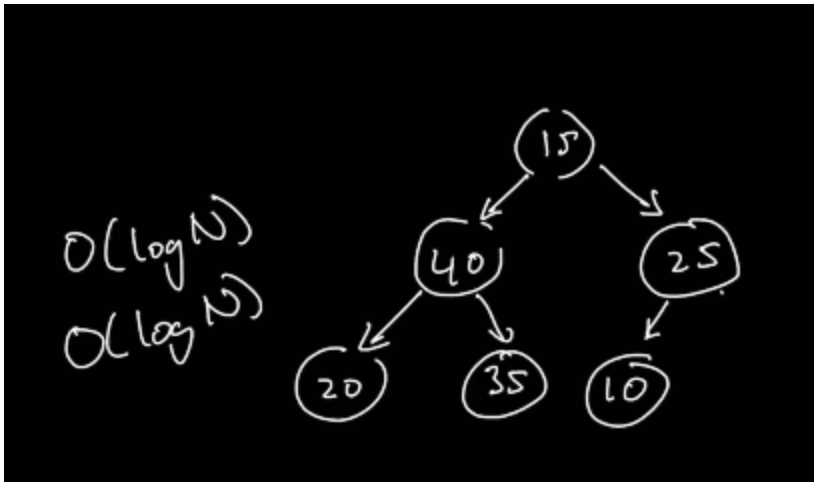




This is the result for 75

Priority Queue

Is like a queue but always gives the value with highest priority



In the implementation we can pass a function to get the min and max heap in the same class

```
class PriorityQueue {
    constructor( comparator = (a,b) => a > b ) {
        this._heap = [];
        this._comparator = comparator;
    }
    size() {
        return this._heap.length;
    }
    isEmpty() {
        return
```

```
isEmpty() {
  return this.size() === 0;
}
peek() {
  return this._heap[0];
}
_parent(idx) {
  return Math.floor((idx - 1) / 2);
}
```

```
_leftChild(idx) {
  return idx * 2 + 1;
}
_rightChild(idx) {
  return idx * 2 + 2;
}
```

```
_swap(i, j) {
  const temp = this._heap[i];
  this._heap[i] = this._heap[j];
  this._heap[j] = temp;
}
_compare(i, j) {
  return this._comparator(this._heap[i], this._heap[j]);
}
```

For insert:

```
push(value) {
  this._heap.push(value)
  this._siftUp();
  return this.size();
}
```

```
_siftUp() {
  let nodeIdx = this.size() - 1;
  while (nodeIdx > 0 && this._compare(nodeIdx, this._parent(nodeIdx))) {
      this._swap(nodeIdx, this._parent(nodeIdx));
      nodeIdx = this._parent(nodeIdx);
  }
}
```

```
pop() {
  if(this.size() > 1) {
    this._swap(0, this.size() - 1);
  }
  const poppedValue = this._heap.pop();
  this._siftDown();
  return poppedValue;
}
```

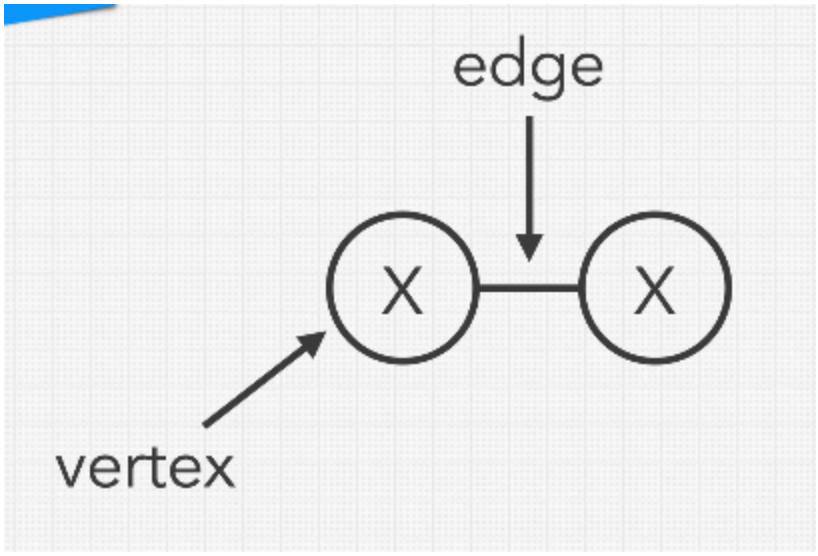
```
_siftDown() {
    let nodeIdx = 0;
    while (
      (this._leftChild(nodeIdx) < this.size() &&
      this._compare(this._leftChild(nodeIdx), nodeIdx)) ||
      (this._rightChild(nodeIdx) < this.size() &&
       this._compare(this._rightChild(nodeIdx), nodeIdx))
    ) {
```

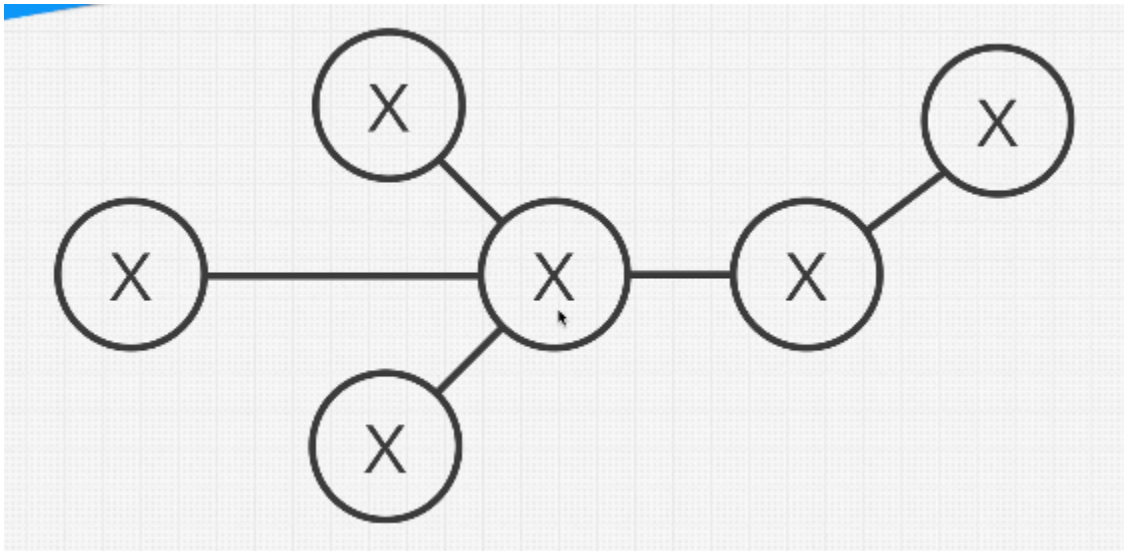
```
const greaterNodeIdx =
    this._rightChild(nodeIdx) < this.size() &&
    this.compare(this.rightChild(nodeIdx), this._leftChild(nodeIdx))
        ? this._rightChild(nodeIdx)
        : this._leftChild(nodeIdx)

this._swap(greaterChildIdx, nodeIdx);
nodeIdx = greaterChildIdx;
```

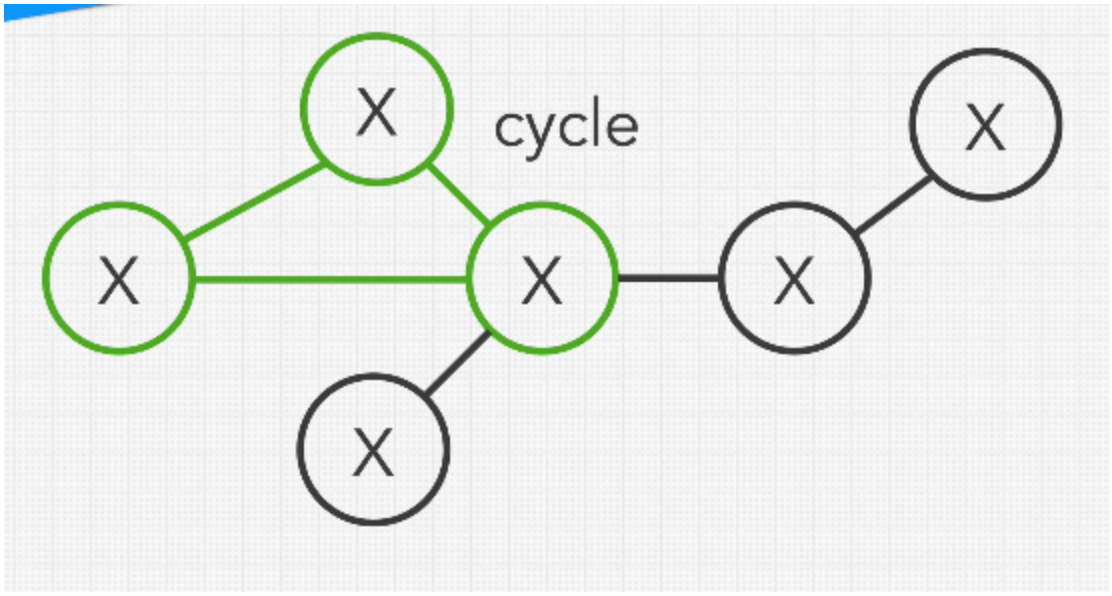
This is efficient by log(n) and perfect for API or something similar, is very fast.

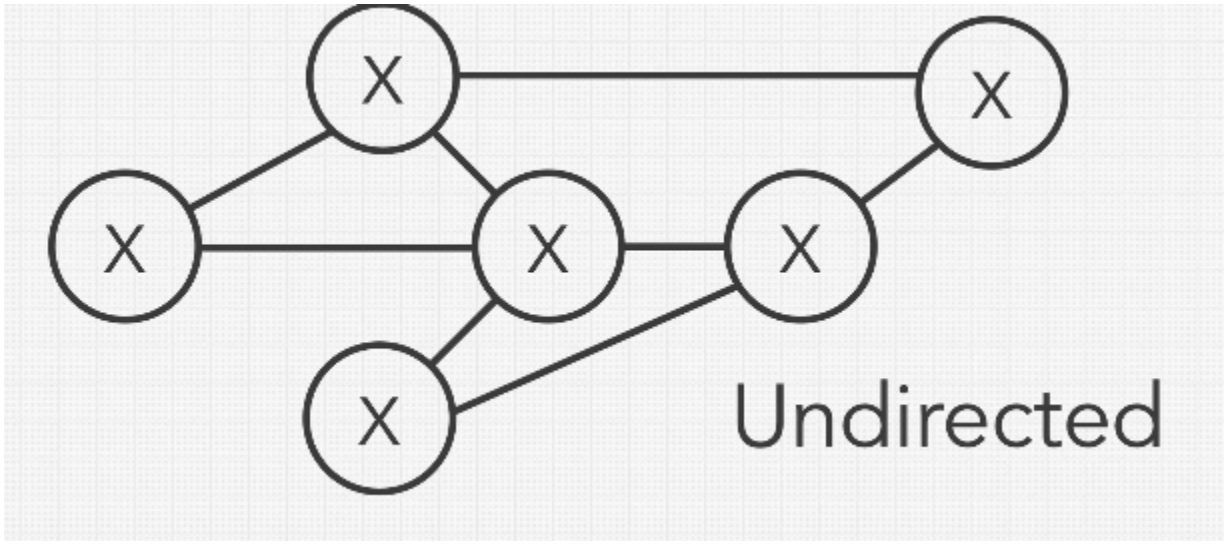
Graphs

Collection of nodes

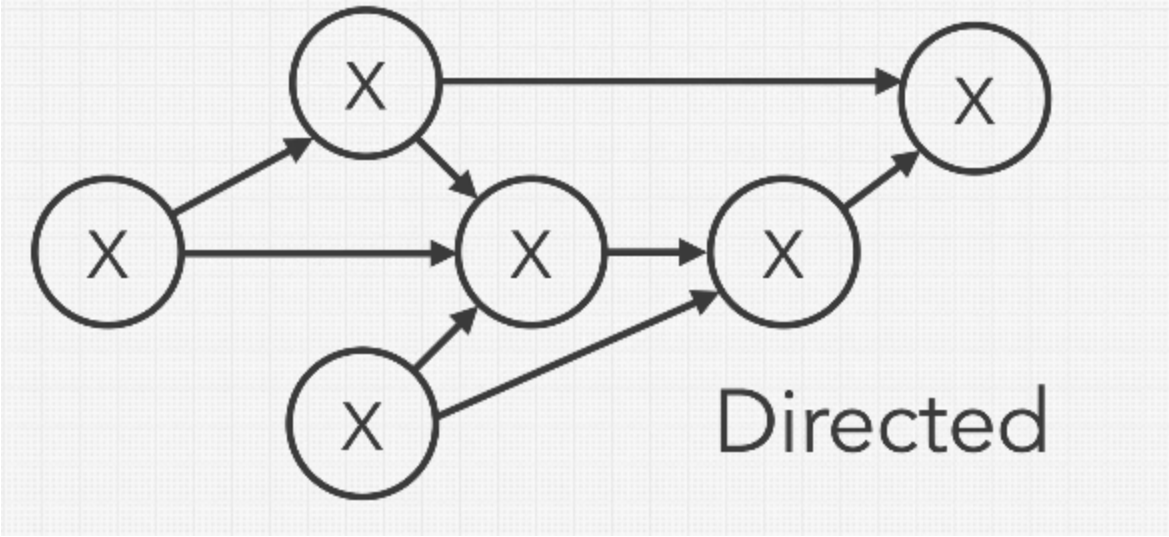

edge
X
X
vertex


X
X
X
X
X
X

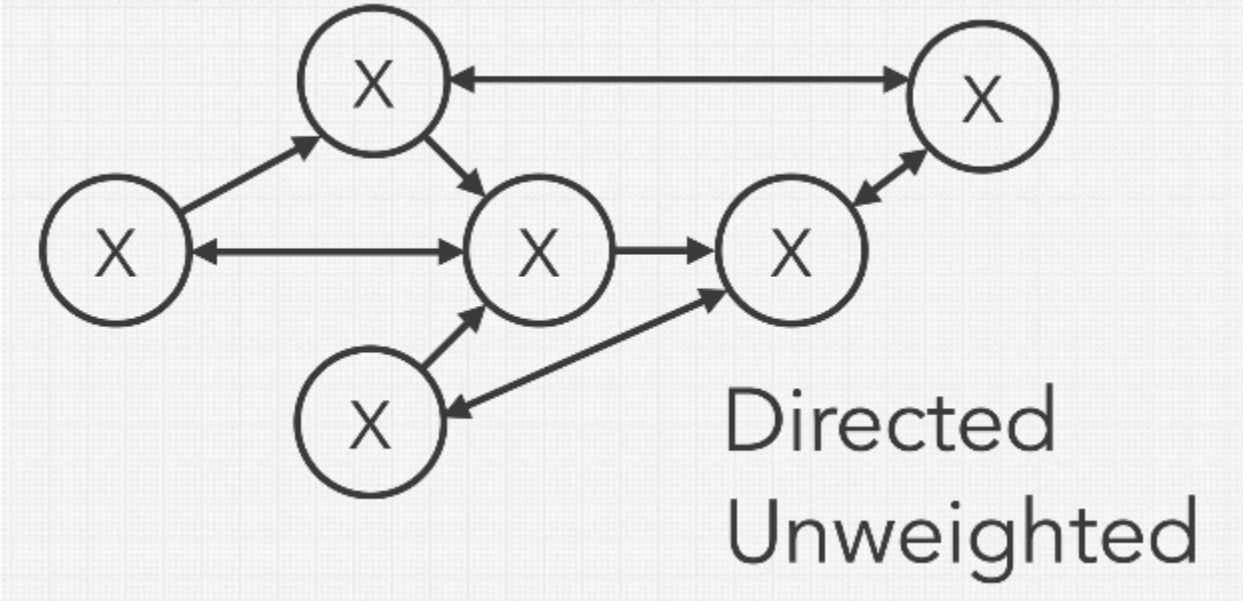


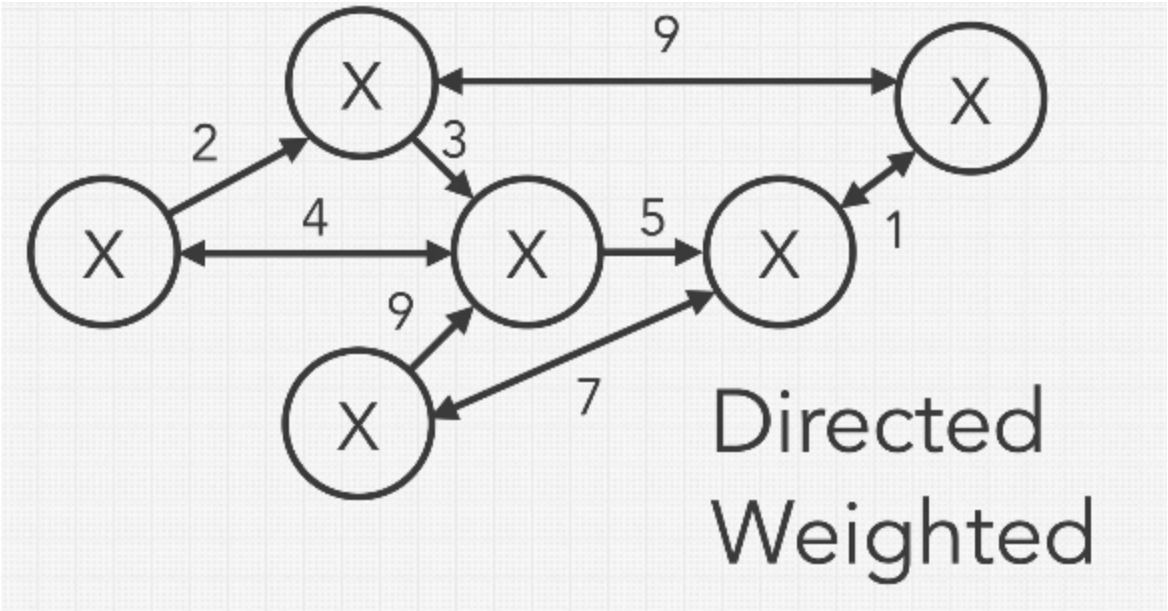


We traverse without problems from one node to another node



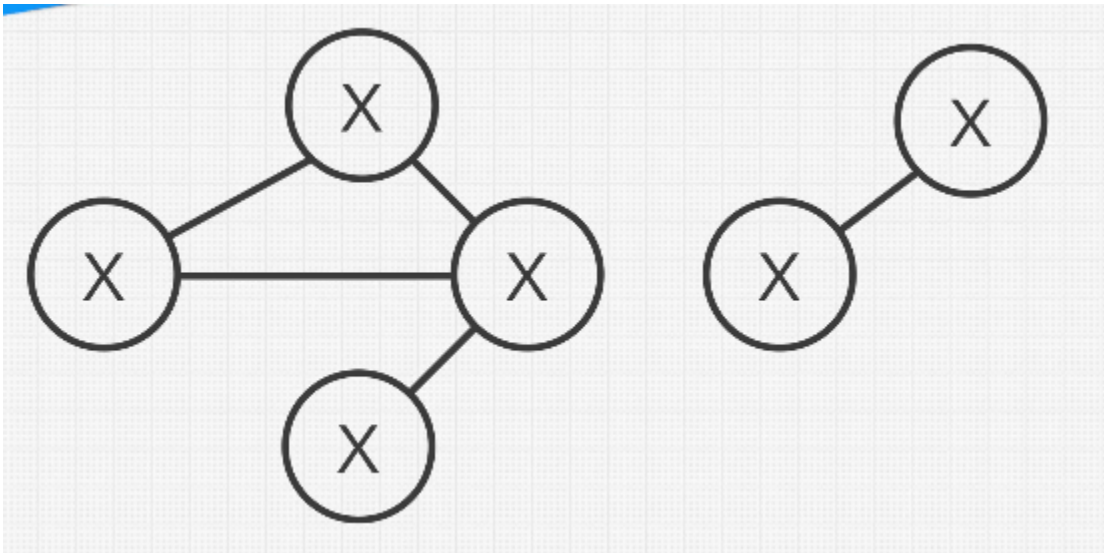
Some can go in both directions





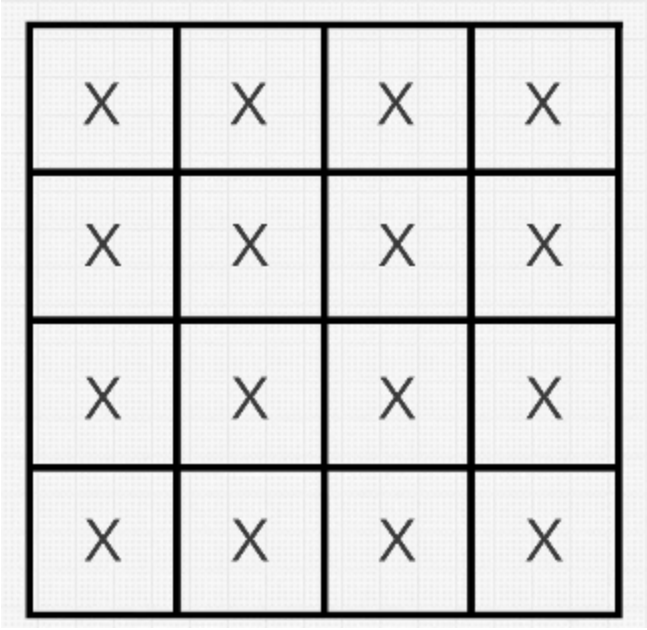
Undirected graph can be easily converted to directed

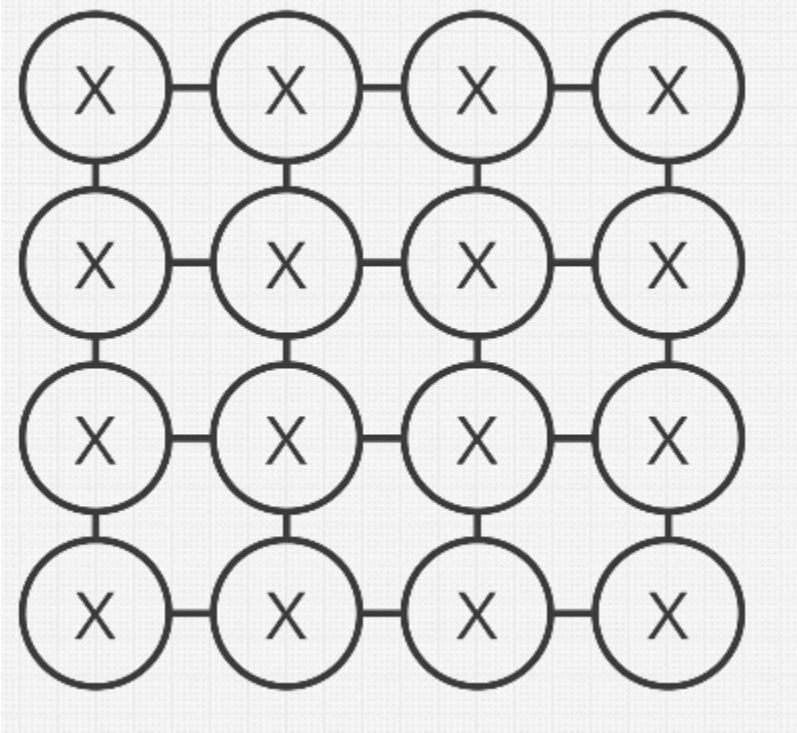
Graphs not necessarily have to be all connected



All binary trees are graphs but not all graphs are binary tree.

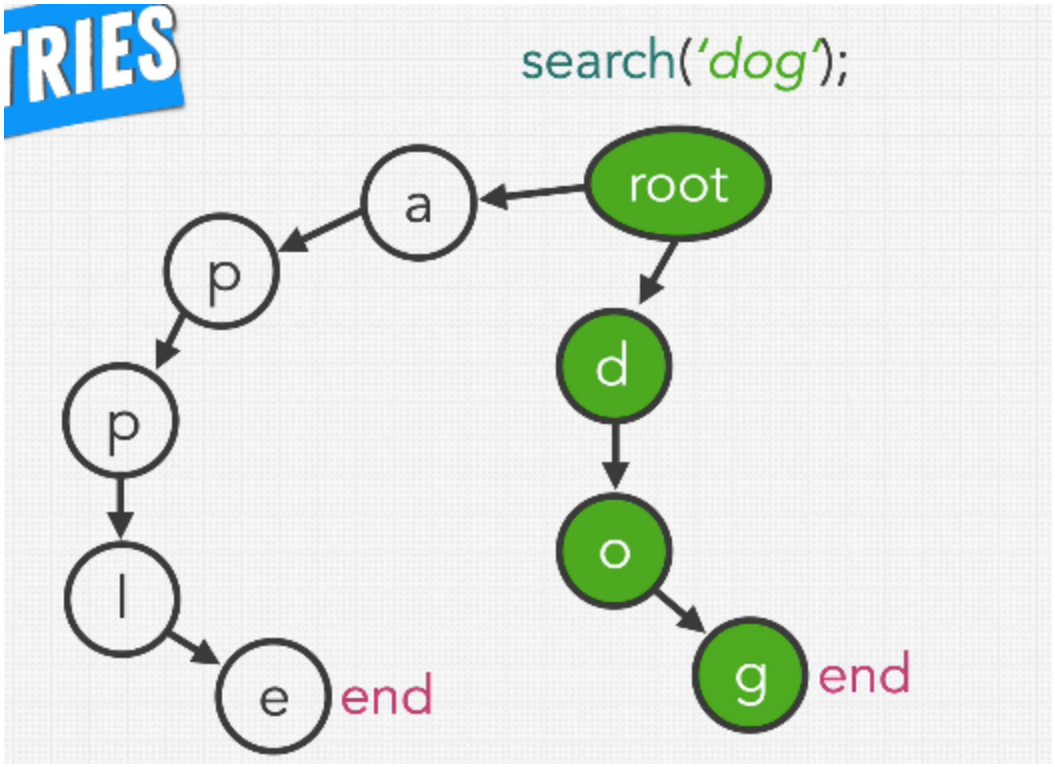
A matrix also is a graph



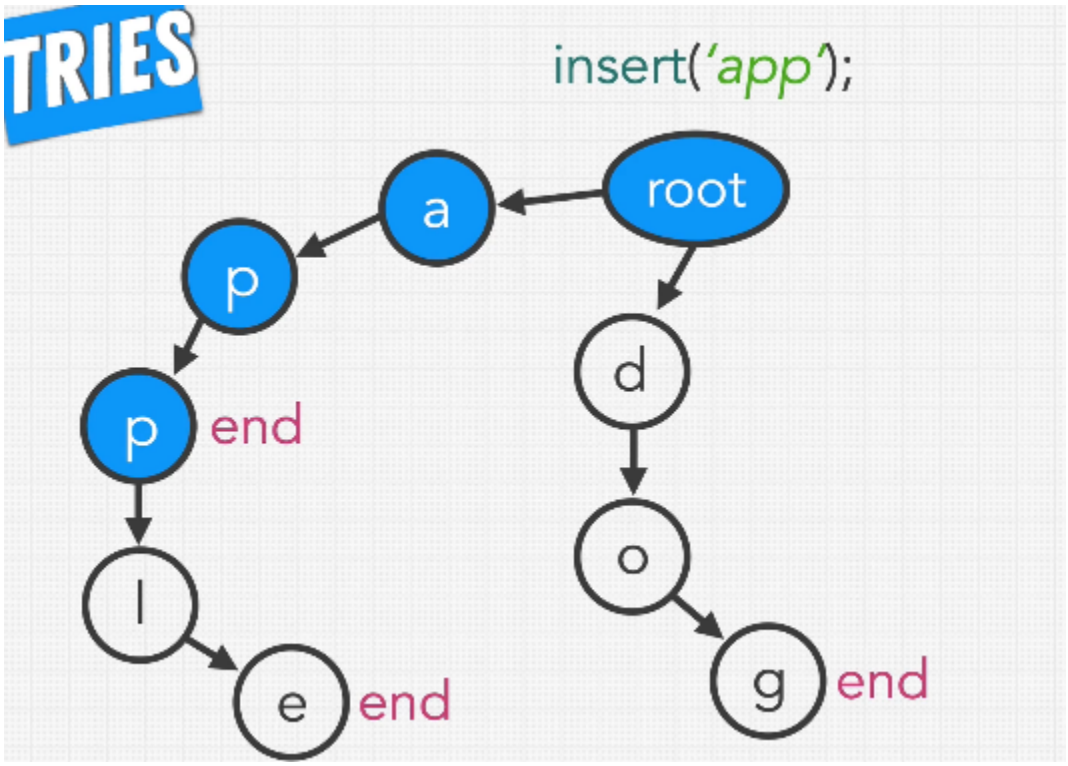


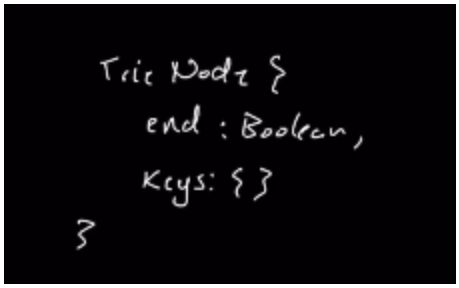
Tries

This hold strings, for quickle search of string

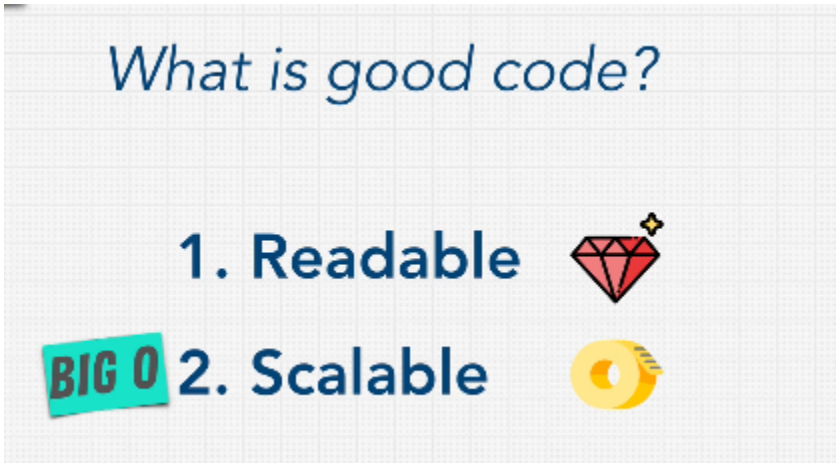


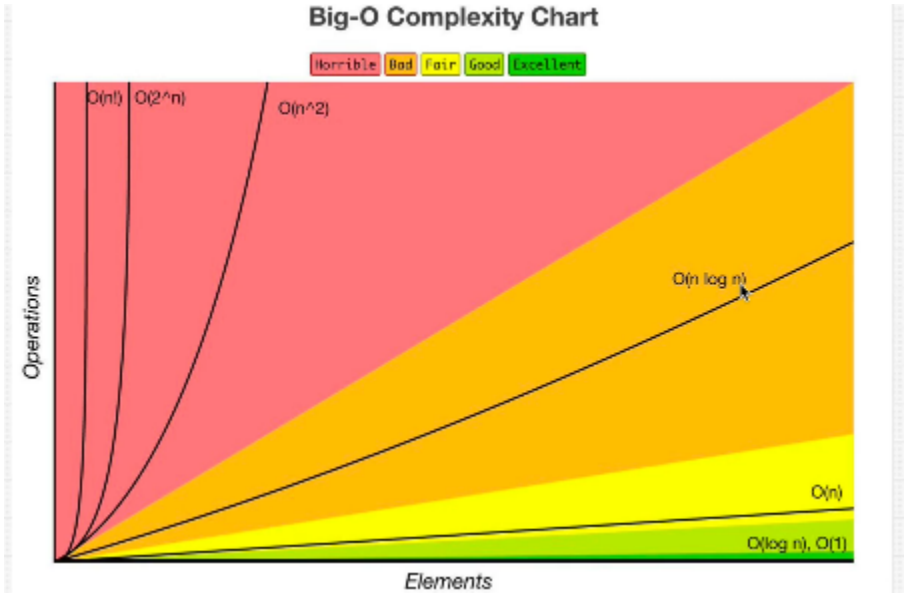
We mark the end



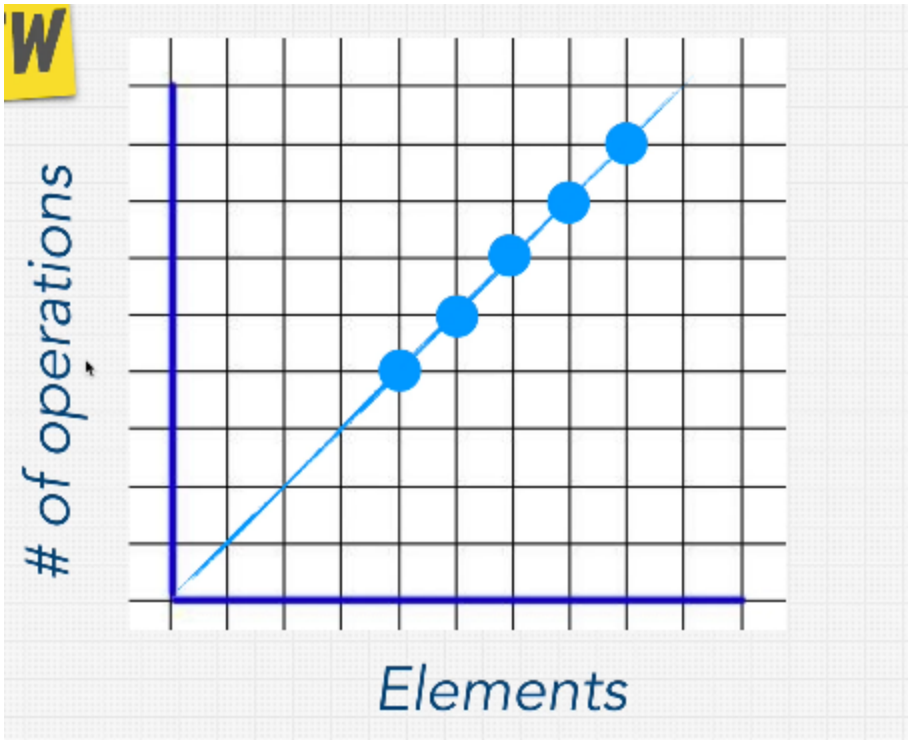


Big-O





This is linear time

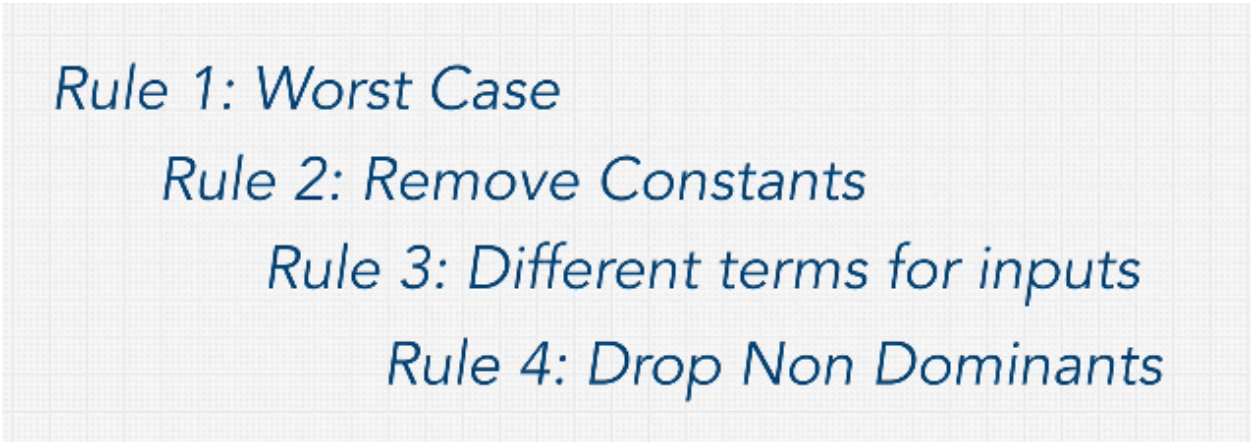



```
; // O(n) --> Linear Time
```

Example of counting operations

```
function funChallenge(input) {
  let a = 10; //O(1)
  a = 50 + 3; //O(1)

  for (let i = 0; i < input.length; i++) { //O(n)
    anotherFunction(); //O(n)
    let stranger = true; //O(n)
    a++; //O(n)
  }
  return a; //O(1)
}

BIG O(3 + 4n)
```

```
function anotherFunCahllenge(input) {
  let a = 5; //O(1)
  let b = 10;//O(1)
  let c = 50;//O(1)
  for (let i = 0; i < input; i++) {
    let x = i + 1; //O(n)
    let y = i + 2; //O(n)
    let z = i + 3; //O(n)
  }
  for (let j = 0; j < input; j++) {
    let p = j * 2; //O(n)
    let q = j * 2; //O(n)
  }
  let whoAmI = "I don't know"; //O(1)
}

BIG O(4 + 5n)
```

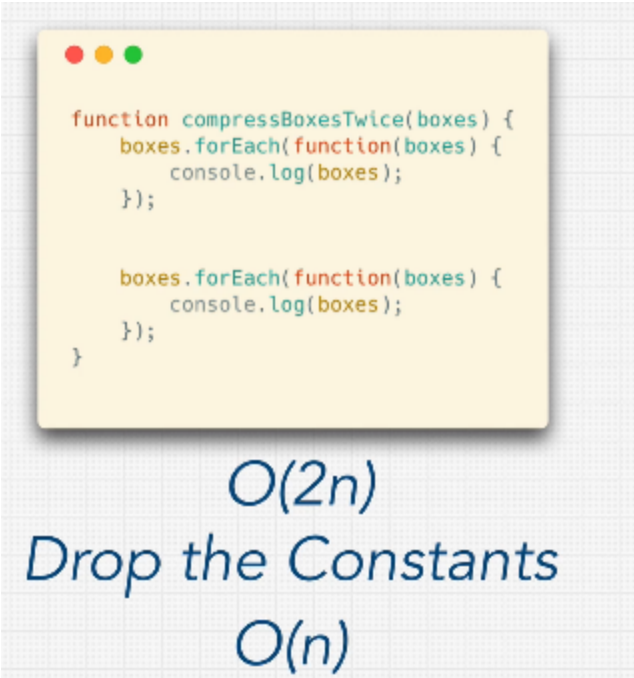


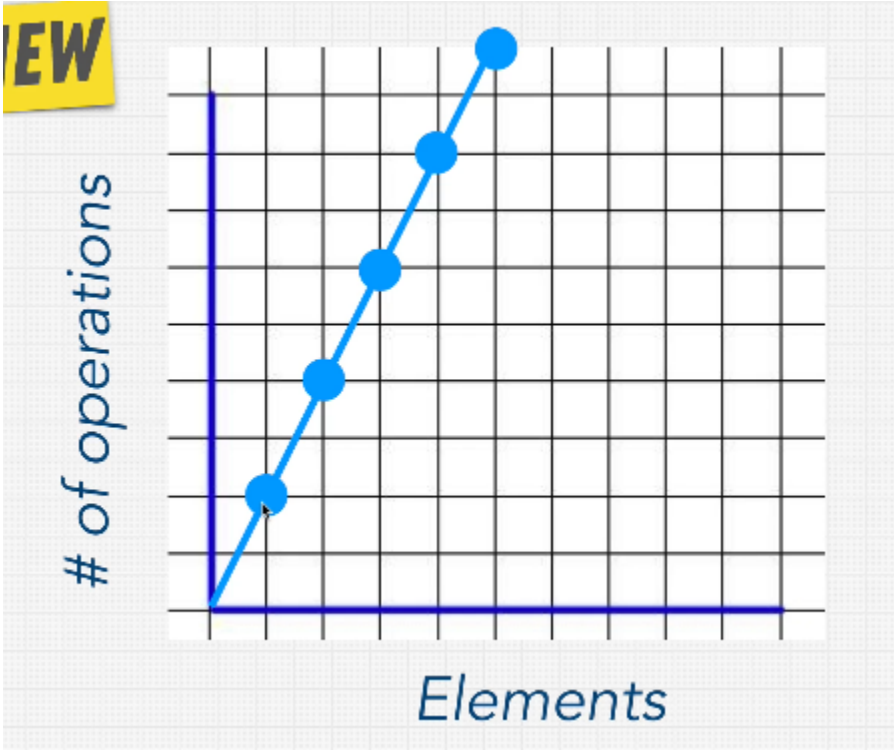
Rule 1: assume the maximum iterations even if exists control instructions like break inside a conditional

Rule 2: Drop constants

```
O(1 + n/2 + 100)
```

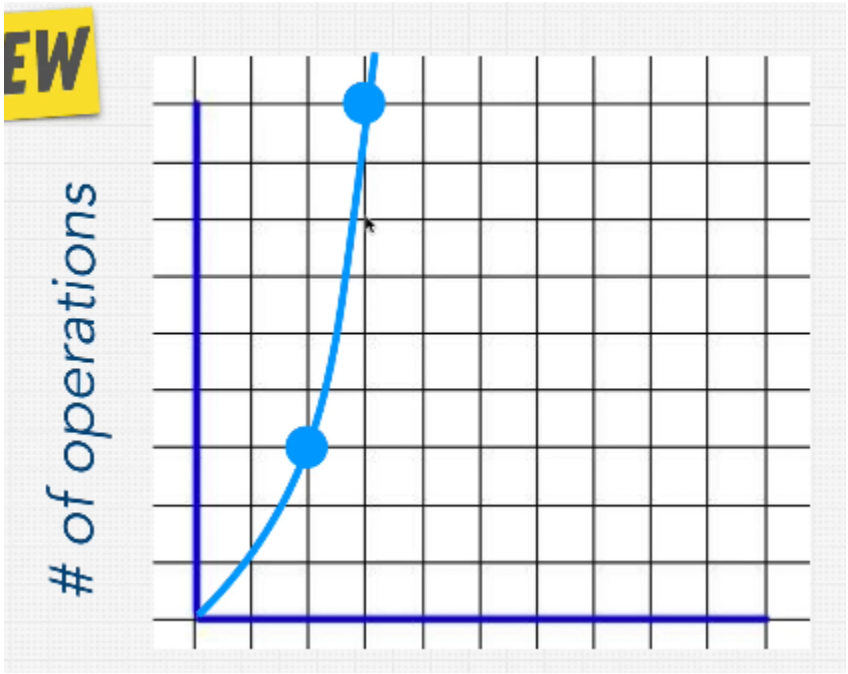
```
O(n)
```





Rule 3: keep the different size of inputs

```
function compressBoxesTwice(boxes, boxes2) {
    boxes.forEach(function(boxes) {
        console.log(boxes);
    });


    boxes2.forEach(function(boxes) {
        console.log(boxes);
    });
}

O(a + b)
```




```
function logAllPairsOfArray(array) {
  for (let i = 0; i < array.length; i++) {
    for (let j = 0; j < array.length; j++) {
      console.log(array[i], array[j])
    }
  }
}

logAllPairsOfArray(boxes)

O(n^2)
```

Rule 4: drop non dominants

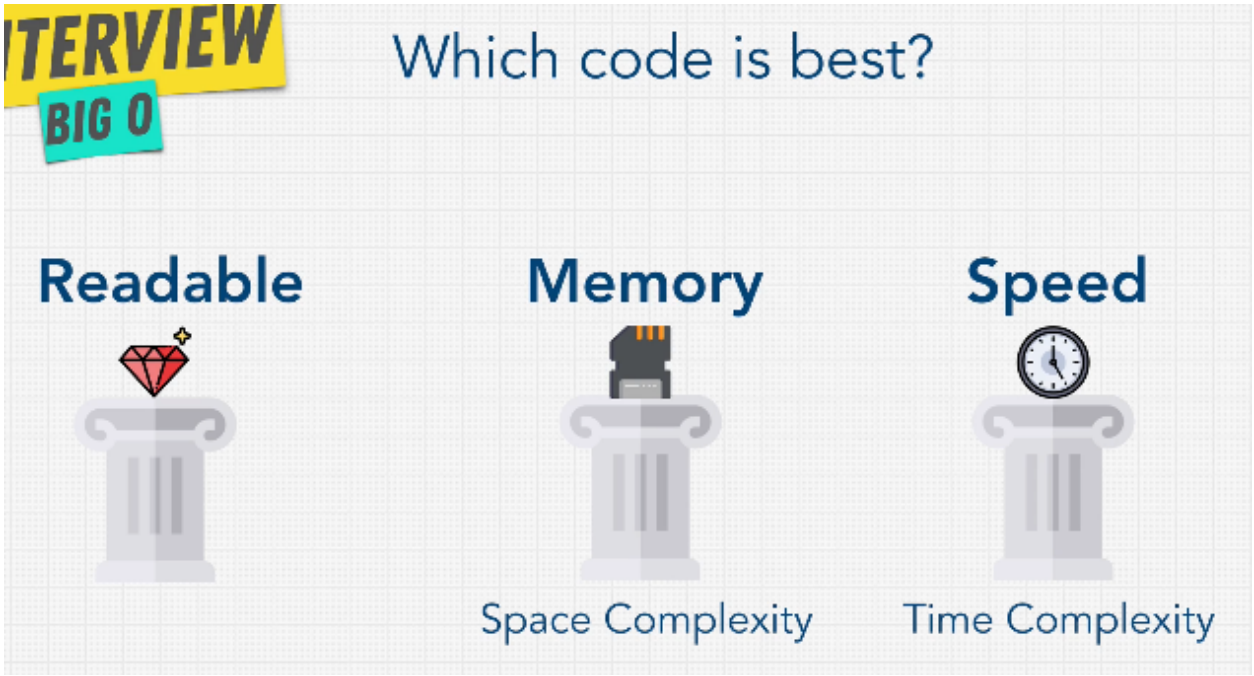
```
O(n + n^2)
```

```
O(x^2)
```

Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

N! -> you have a loop for every element



```
-What causes Space complexity?-
Variables
Data Structures
Function Call
Allocations
```

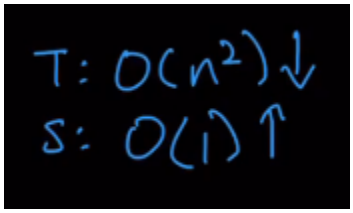
```
function booooo(n) {
  for (let i = 0; i < n.length; i++) {
    console.log('boooooo!');
  }
}

booooo([1,2,3,4,5]) // O(1)

function arrayOfHiNTimes(n) {
  let hiArray = [];
  for (let i = 0; i < n; i++) {
    hiArray[i] = 'hi';
  }
  return hiArray;
}

arrayOfHiNTimes(6) // O(n)
```

Methods

Tradeoff between Space complexity and Time

Convert the functions of the loop with hashmap or other data structure

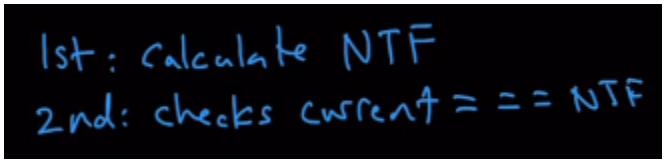
A good option is to make the space complexity worst and the time complexity better.



¿Can we remove a loop of the two loops?

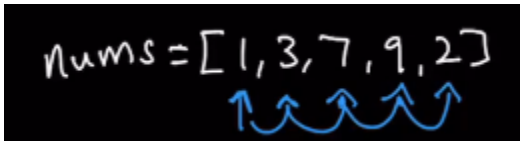
Para pasar un 2 loop to two loops we should figure out what is doing each loop

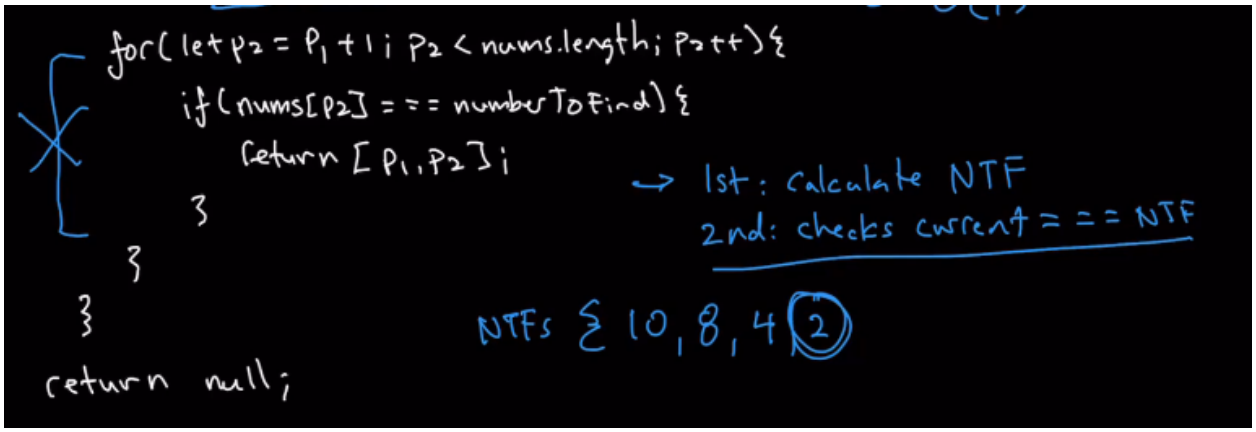
NTF: number to find



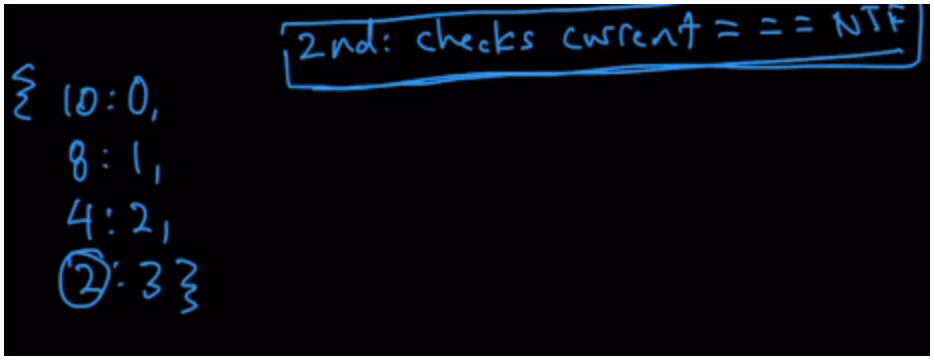
We can use a hashmap

We can first compute the number to find for each number

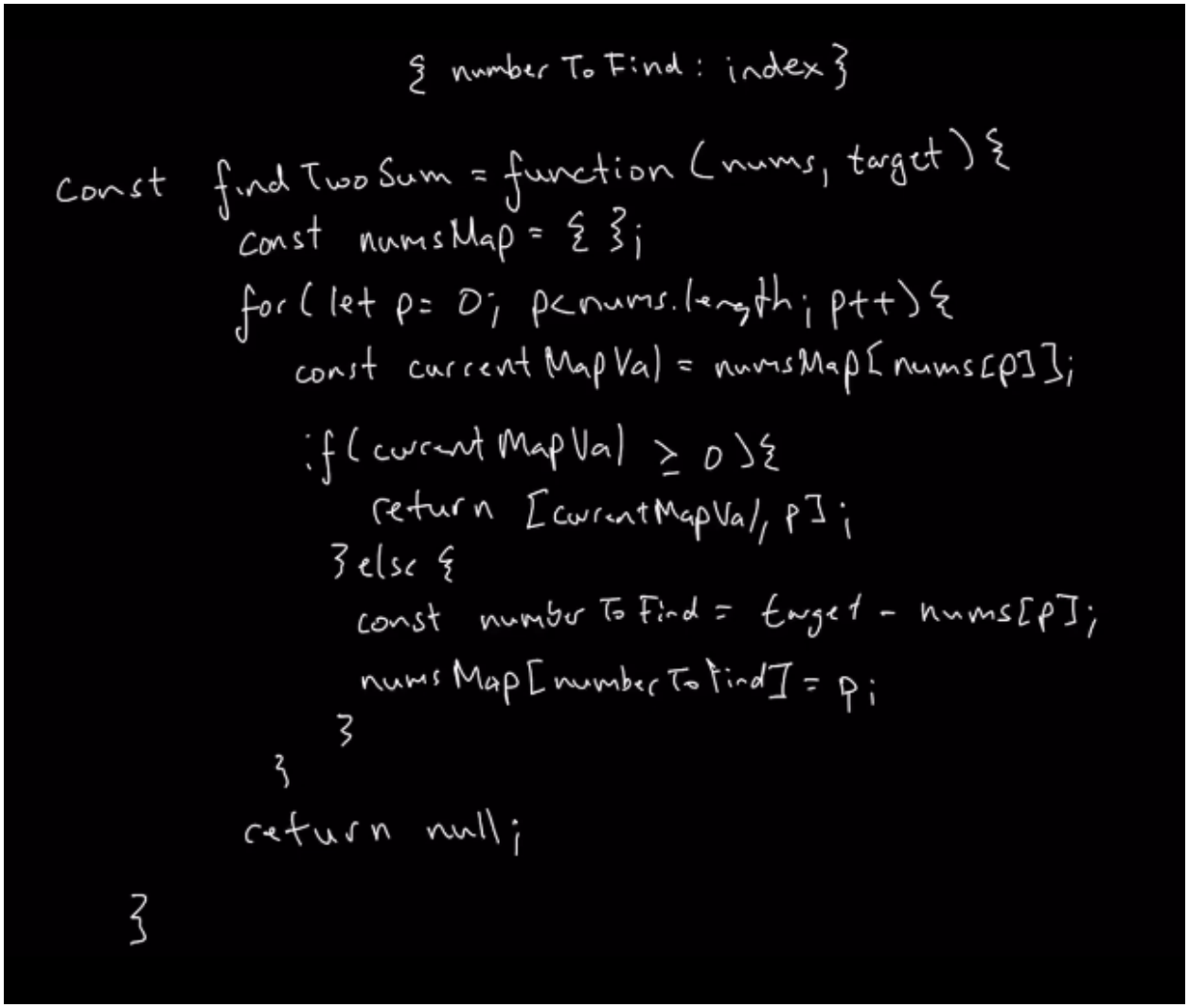




Now we will do the task of second for loop with a hash map

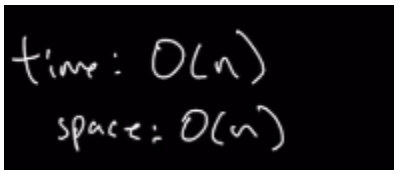


We should check if the value exists, we return the answer, else we generate the solution



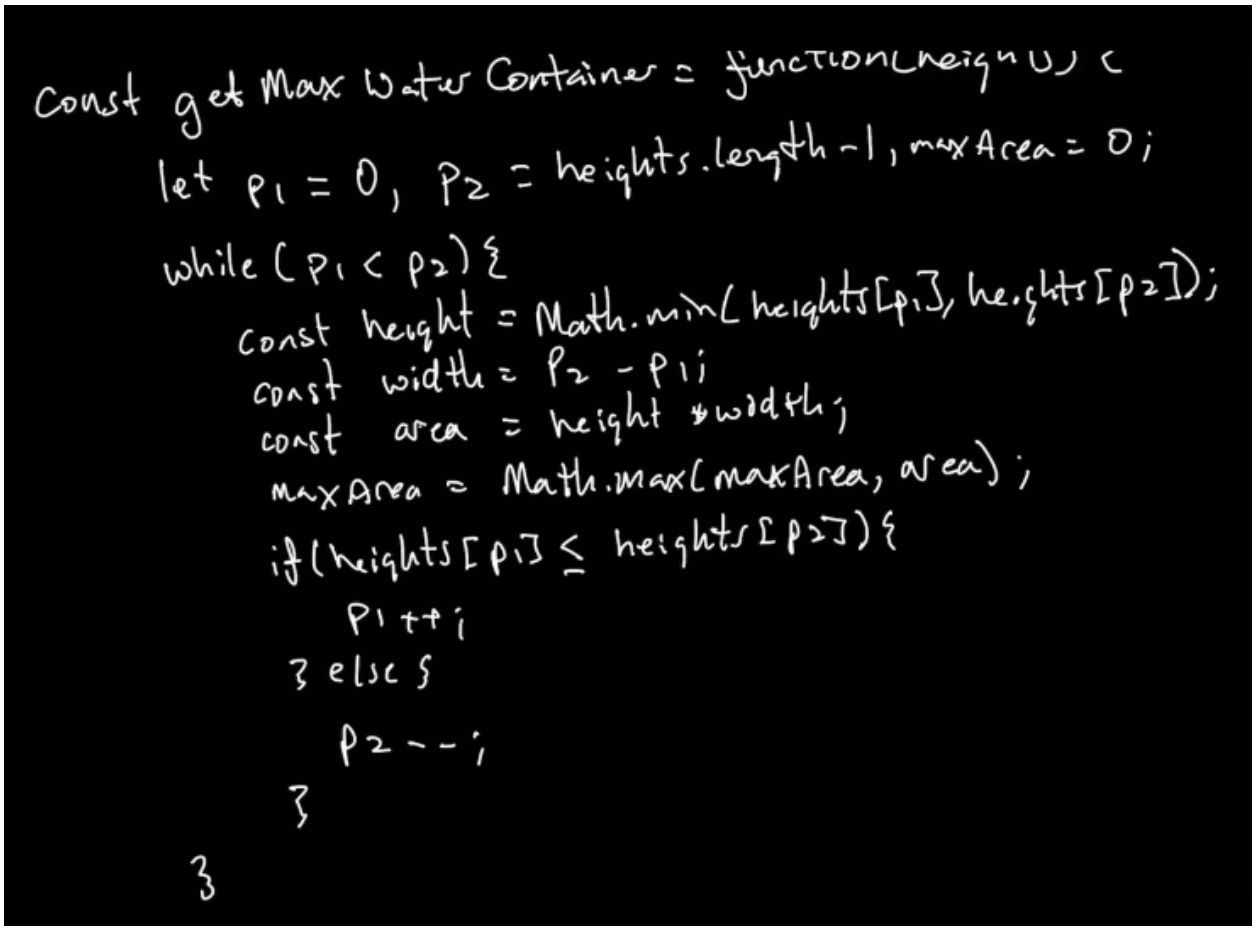
Then we test this optimized solution

The space complexity has grown



Two pointers

Use just a loop, but with reference changing to the pointers

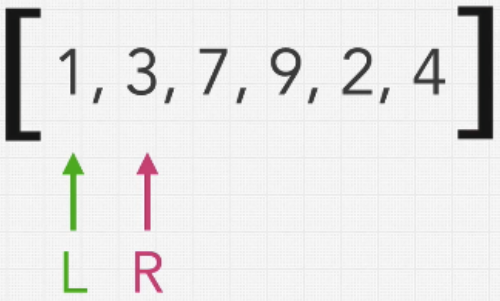


Then return the área

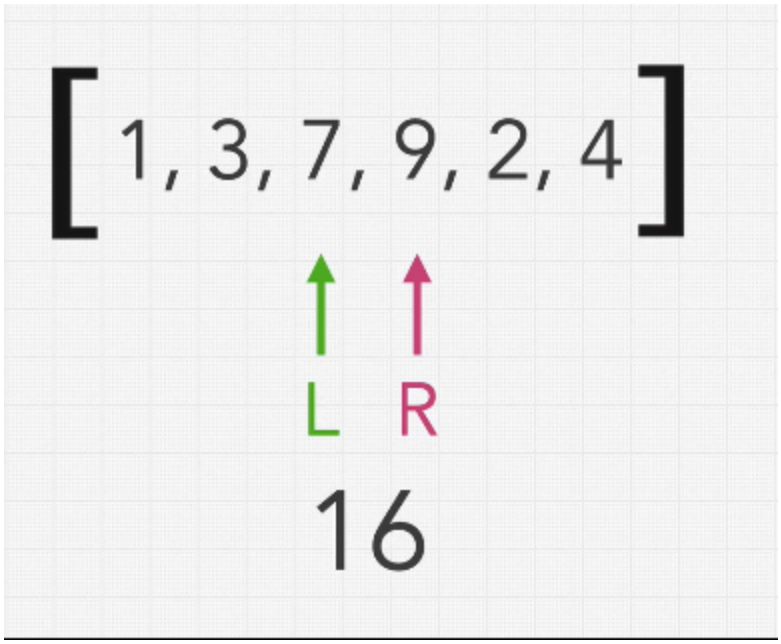
Sliding window

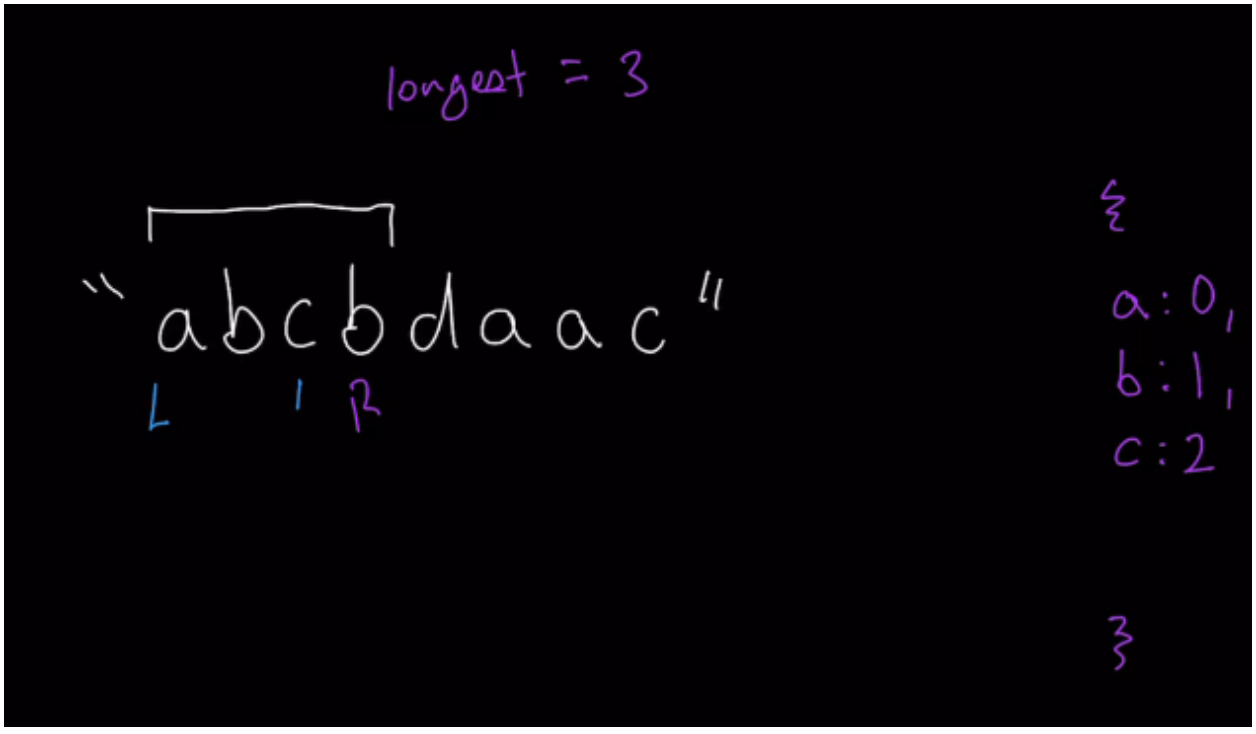
Form a window over some **portion** of **sequential** data, then **move** that window throughout the data to **capture** different parts of it.

Given an array of integers, find two contiguous integers that form the greatest sum.

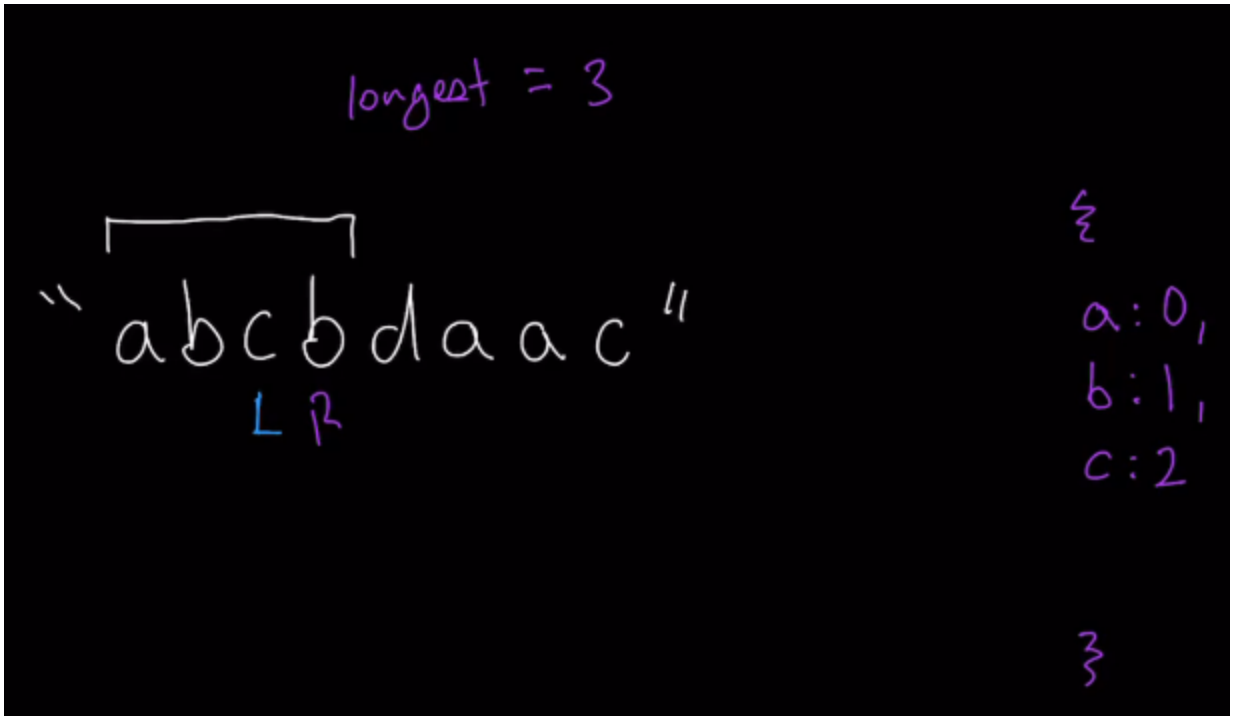


4

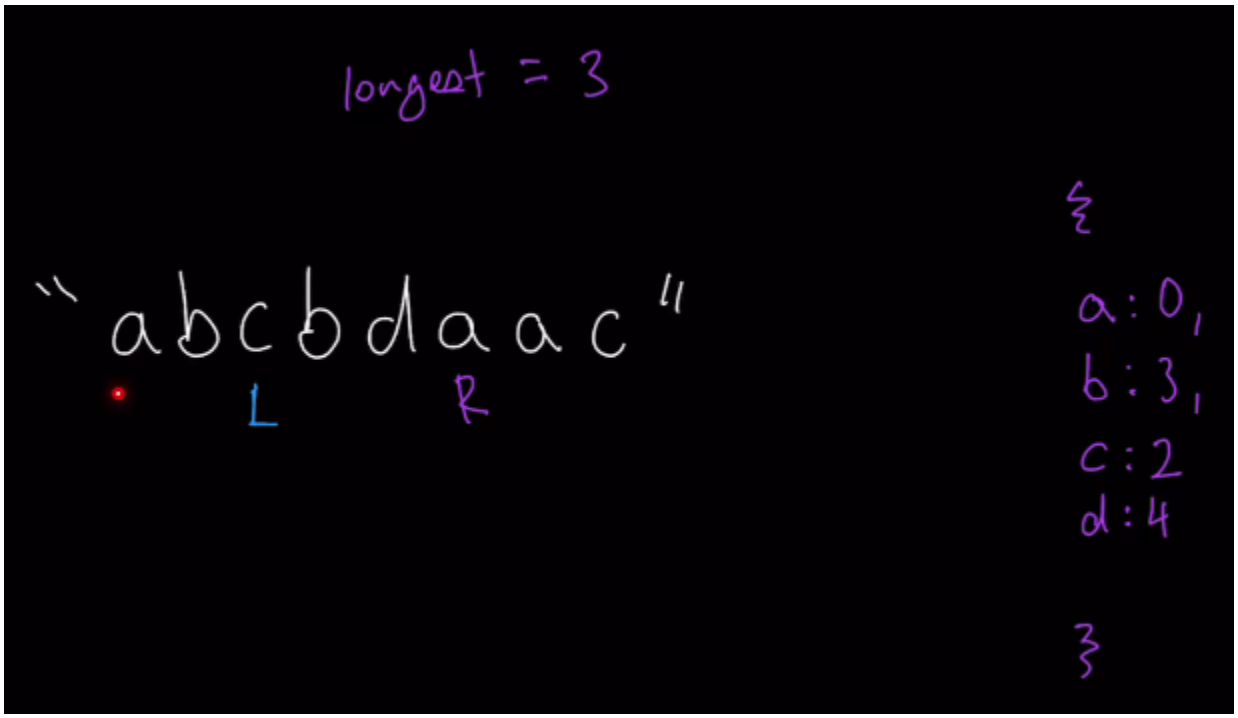


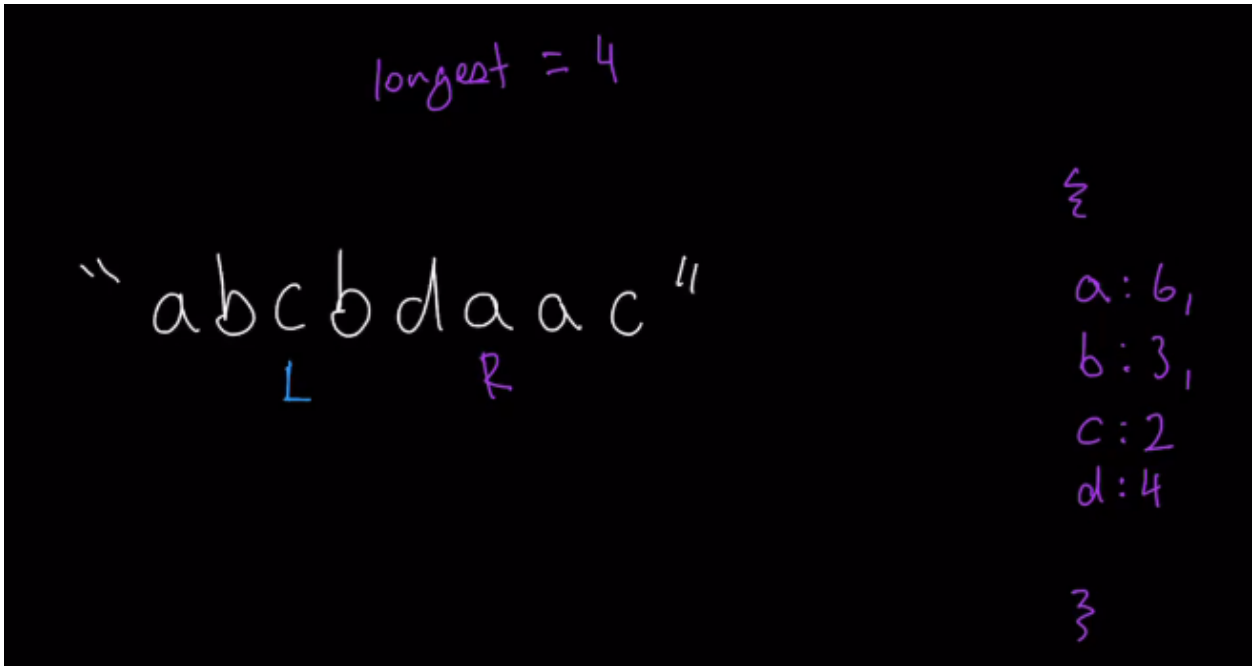


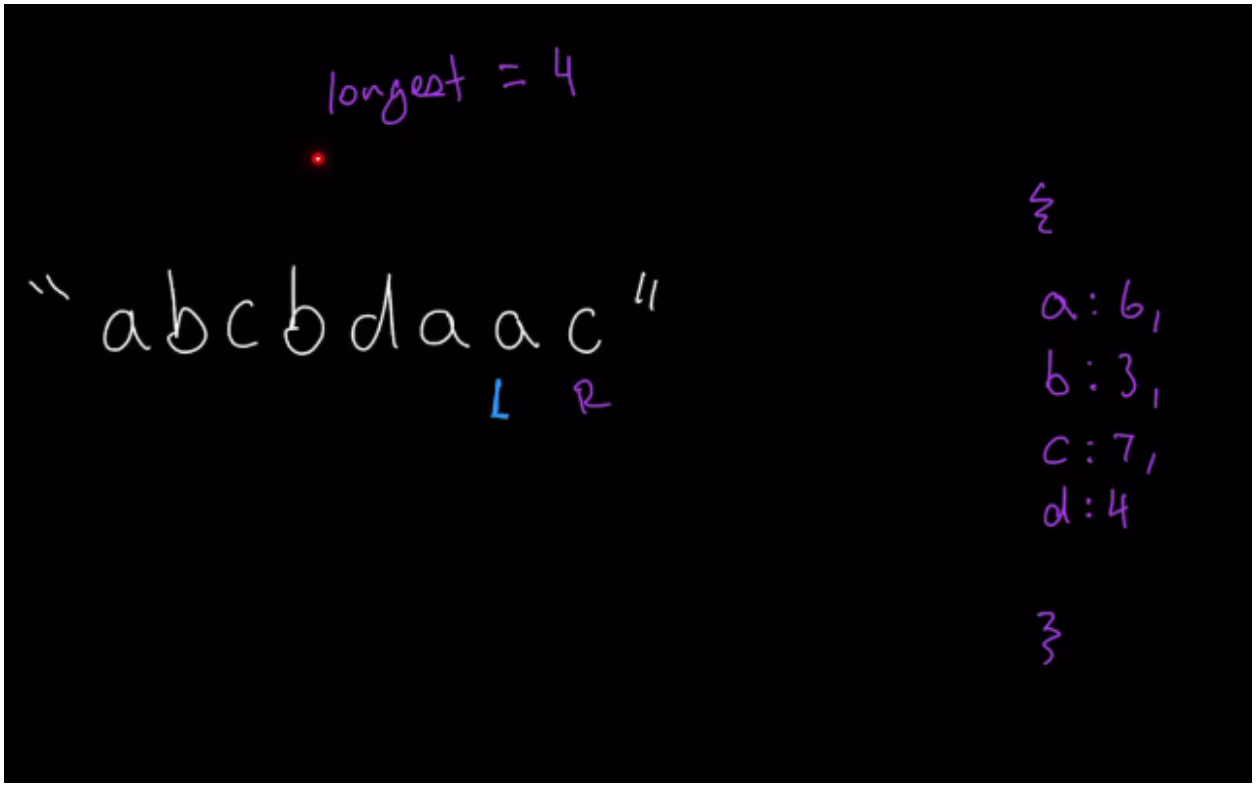
We can derivate a conclusion, the elements after first b are unique, and before are elements with b so will not work



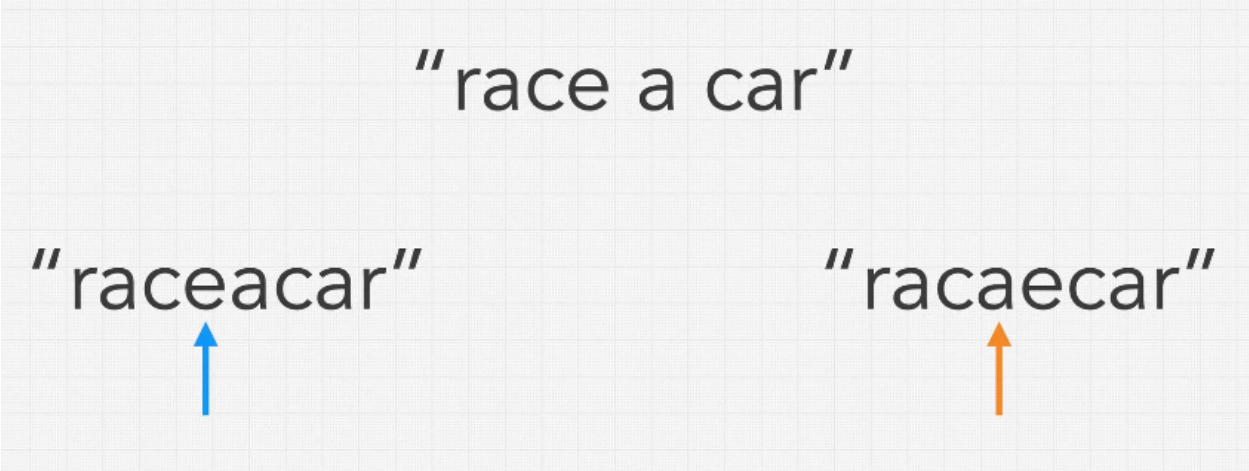
The index is 0 of a is less than L so we haven't seen already




longest = 4
"abcbdaac"
L
R
{
a: 6,
b: 3,
c: 2
d: 4
}


longest = 4
"abcbdaac"
L
R
{
a: 6,
b: 3,
c: 7,
d: 4
}

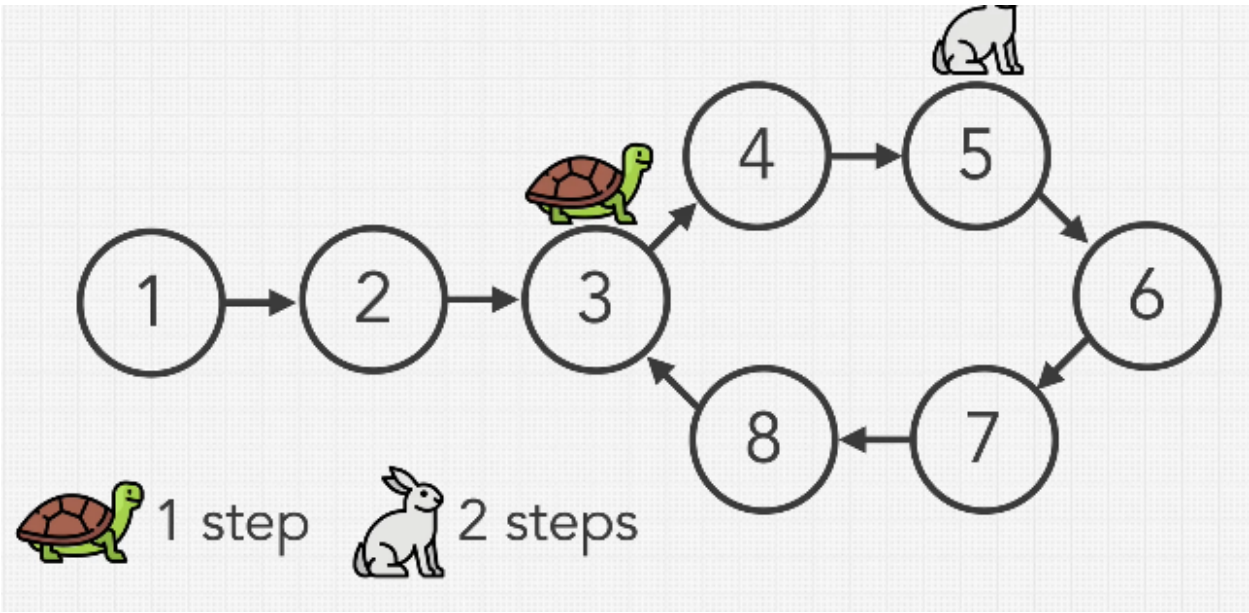
Pointers in palindrome



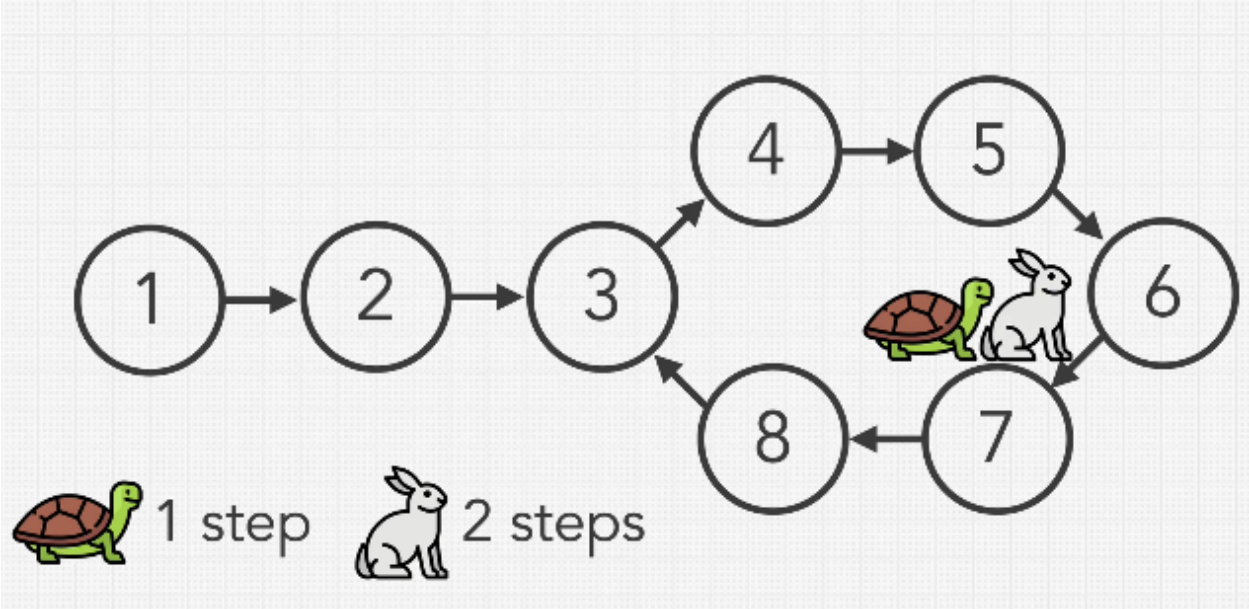
Not every tecnic works in every palindrome problem

This three are important

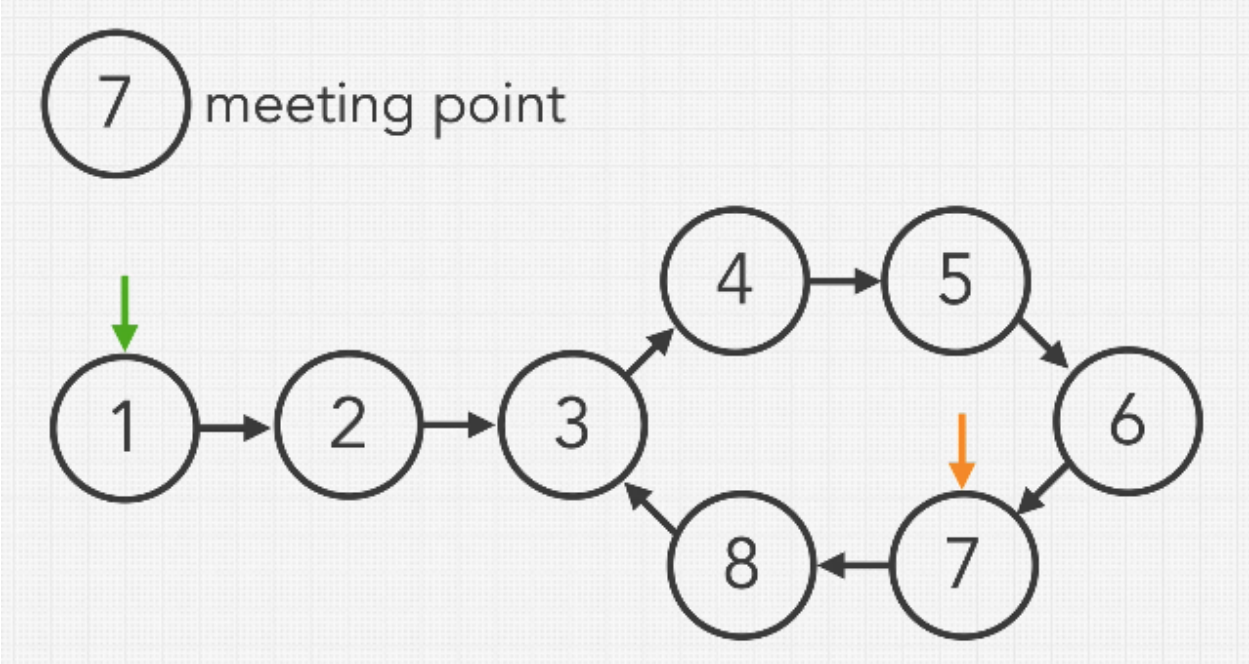
Cycle detection. Floyd's tortoise and hare



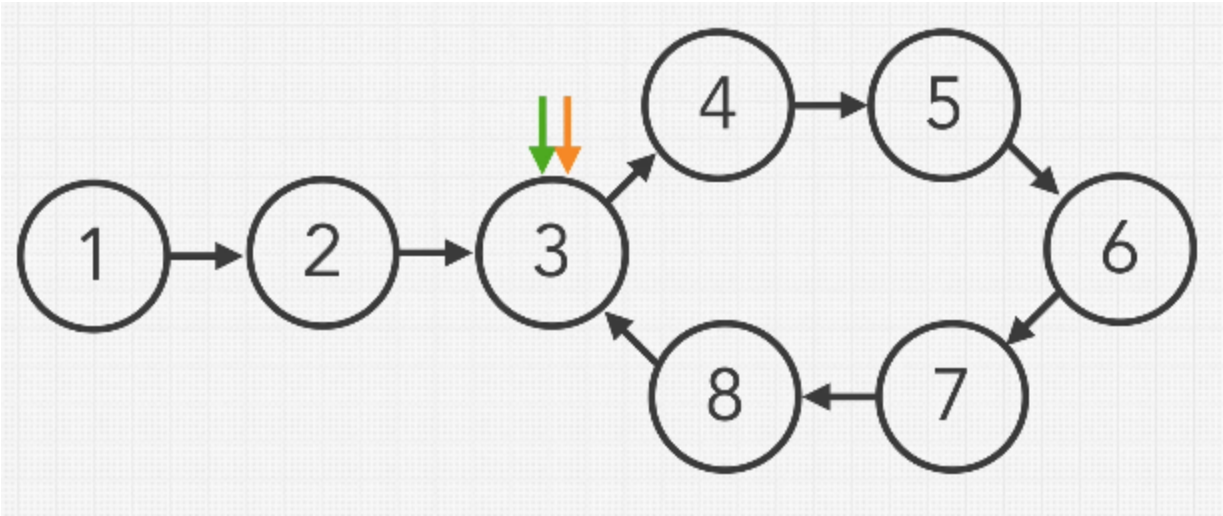
We repeat it until the turtle and rabbit are in the same node



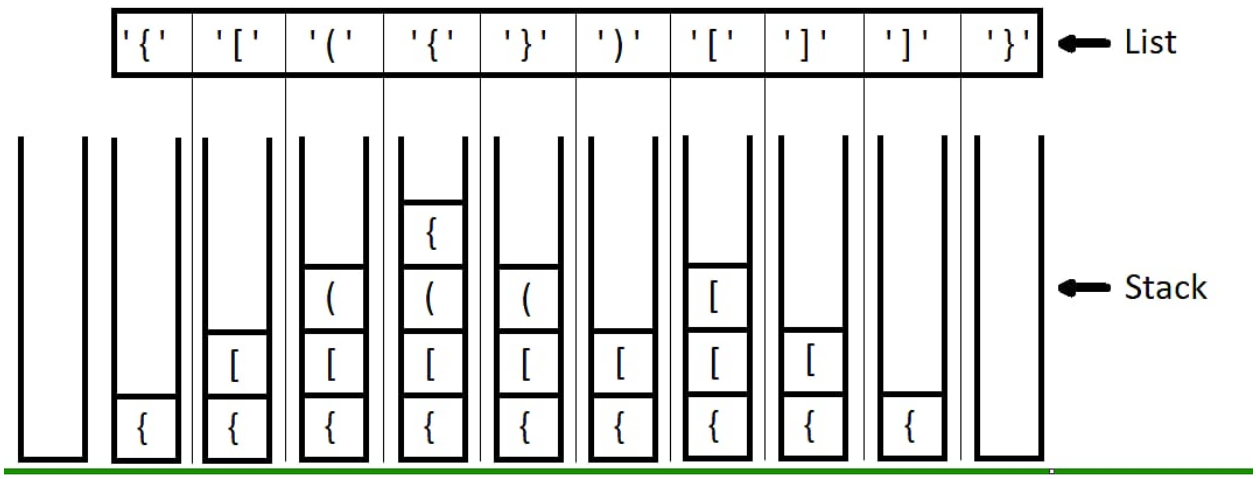
These are pointers moving at different speed



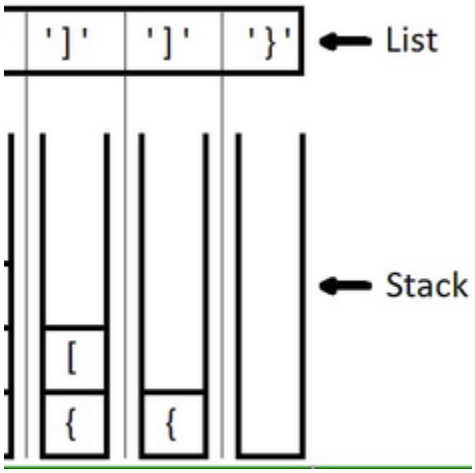
To get the loop we move one at time



Stack

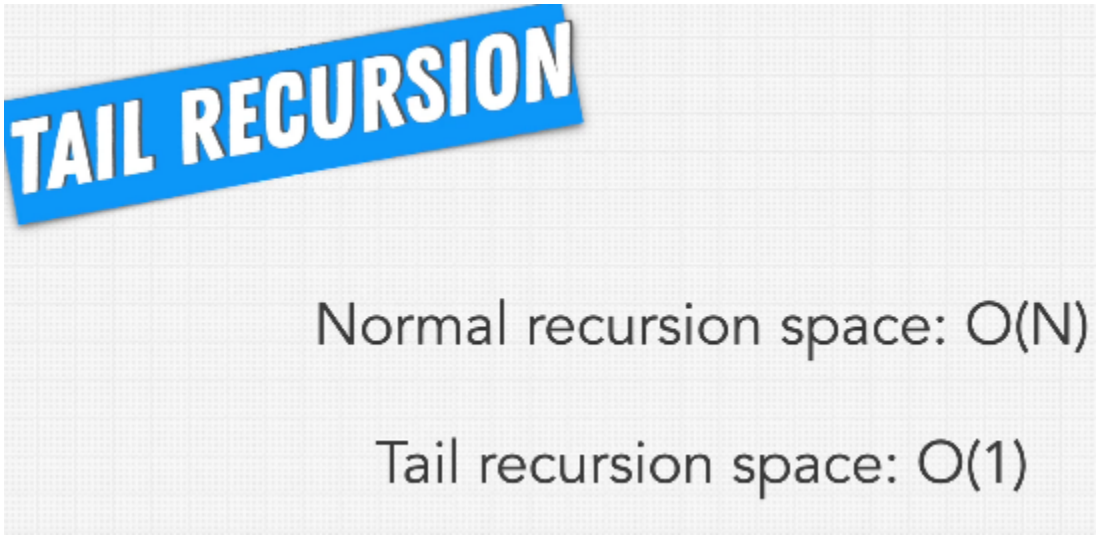


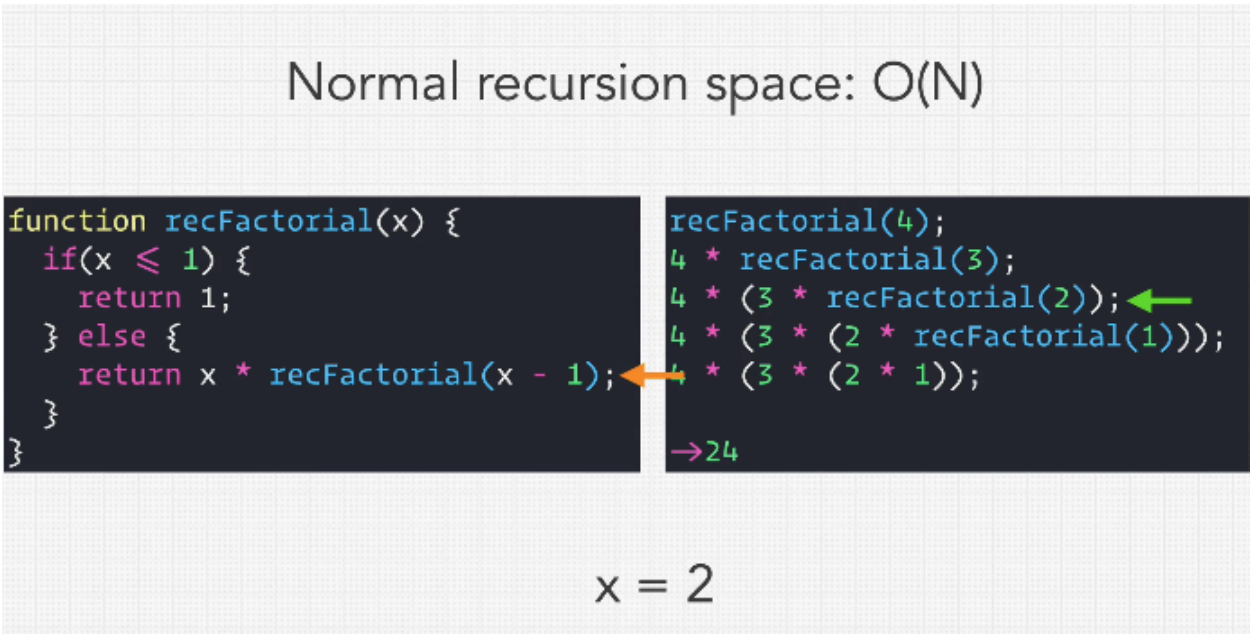
Queue



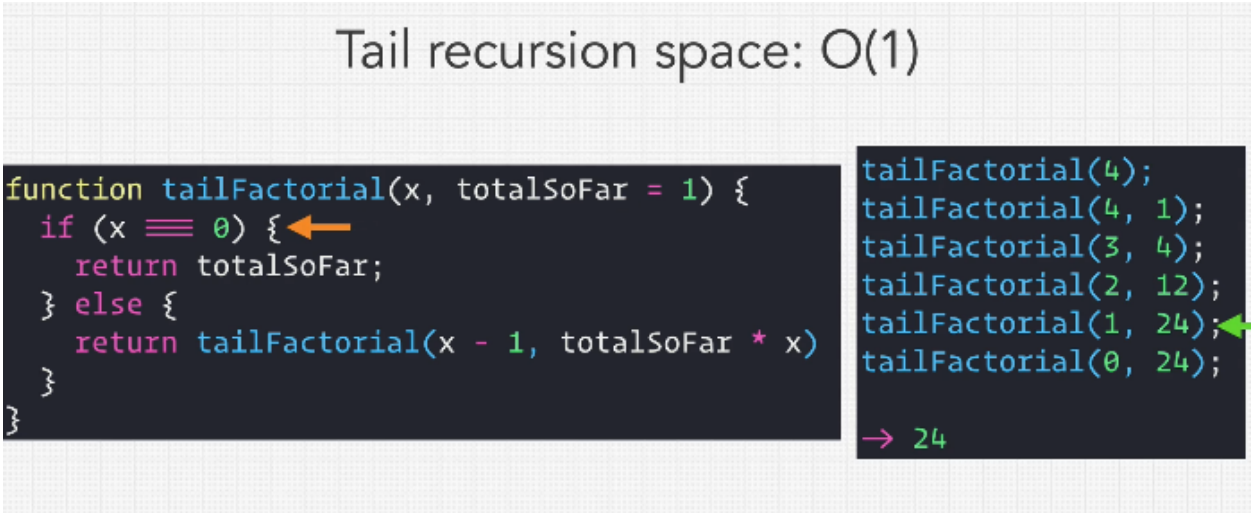
Queue also is useful as a stack, for instance if the parenthesis should close but it’s not important that close inside other other closers.

Tail recursion





This four values are stored in some place of memory



Here we only store a value in memory and is passed through recursion

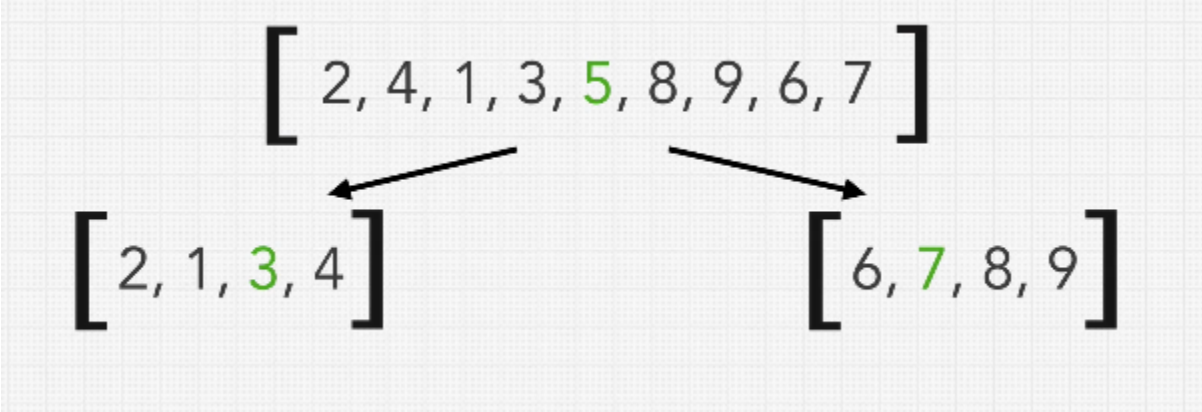
Divide and conquer

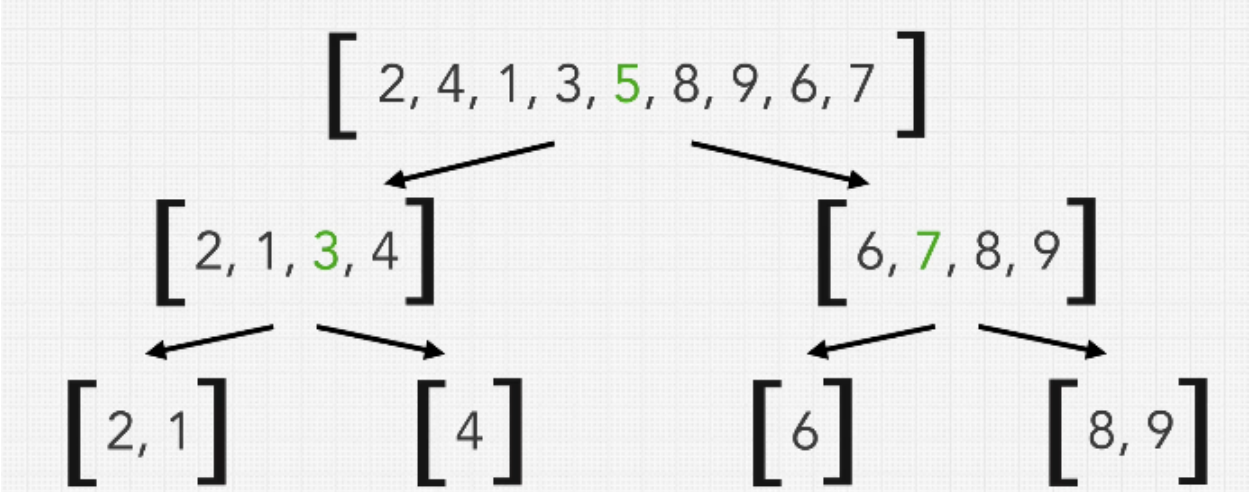
1. Multi-branched recursion.

2. Breaks a problem into multiple smaller but same sub-problems.

3. Combines the solutions of sub-problems into the solution for the original problem.

Multibranched solution:



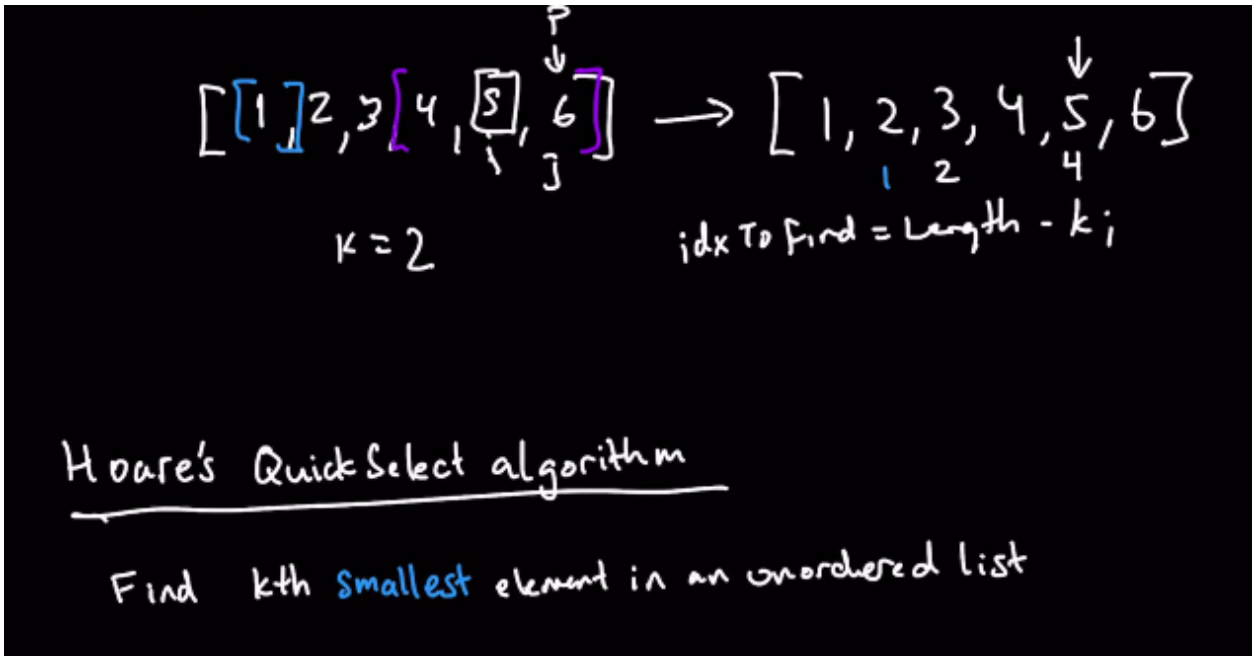


2. Breaks a problem into multiple smaller but same sub-problems.

3. Combines the solutions of sub-problems into the solution for the original problem.

[1, 2, 3, 4, 5, 6, 7, 8, 9]

Hoare's Quick select algorithm



We choose what branch (left or right) of the recursion will be sorted, so it's a modification of quicksort

```
const quickSelect = function(array, left, right, idxToFind) {
  if(left < right) {
    const partitionIdx = partition(array, left, right);
    if(partitionIdx = IdxTo find) {
      return array[partitionIdx];
    } else if(indexTo find < partitionIdx) {
      return quickSelect(array, left, partitionIdx - 1, idxTo find);
    } else {
      return quickSelect(array, partitionIdx + 1, right, idxTo find);
    }
  }
const partition = fuction(array, left, right) {
  const pivotElement = array[right];
```

```
  }
const partition = fuction(array, left, right) {
  const pivotElement = array[right];
  let partitionIdx = left;
  for(let j = left; j < right; j++) {
    if(array[j] < pivotElement) {
      swap(array, partitionIdx, j);
      partitionIdx++;
    }
  }
  swap(array, partitionIdx, right);
```

$n + n/2 + n/4 + n/8 \ldots$

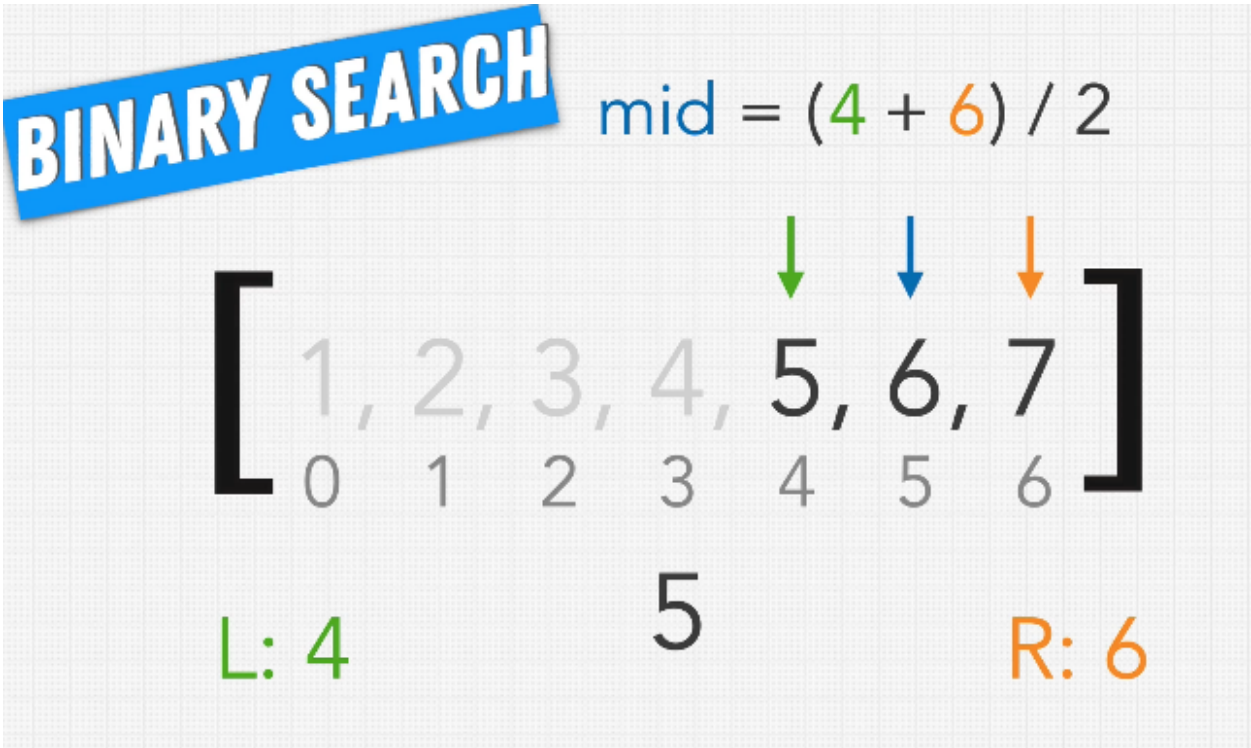
$O(2N)$

$O(N)$

Using just a part of the sorting algorithm can be faster than using all algorithm if the problem doesn't requires that the array is sorted.

Binary search

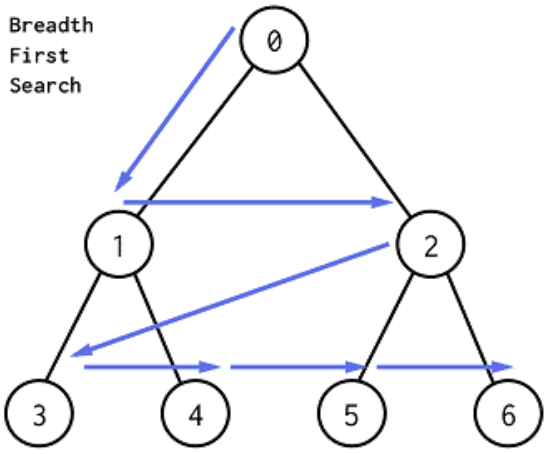
Linear search is go one by one element. With T(N)

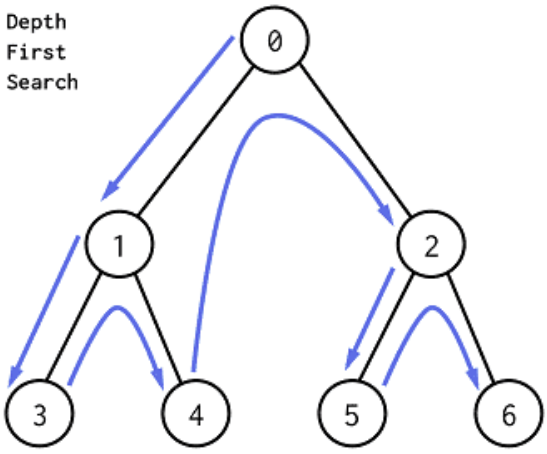


This is a drastic improvement.

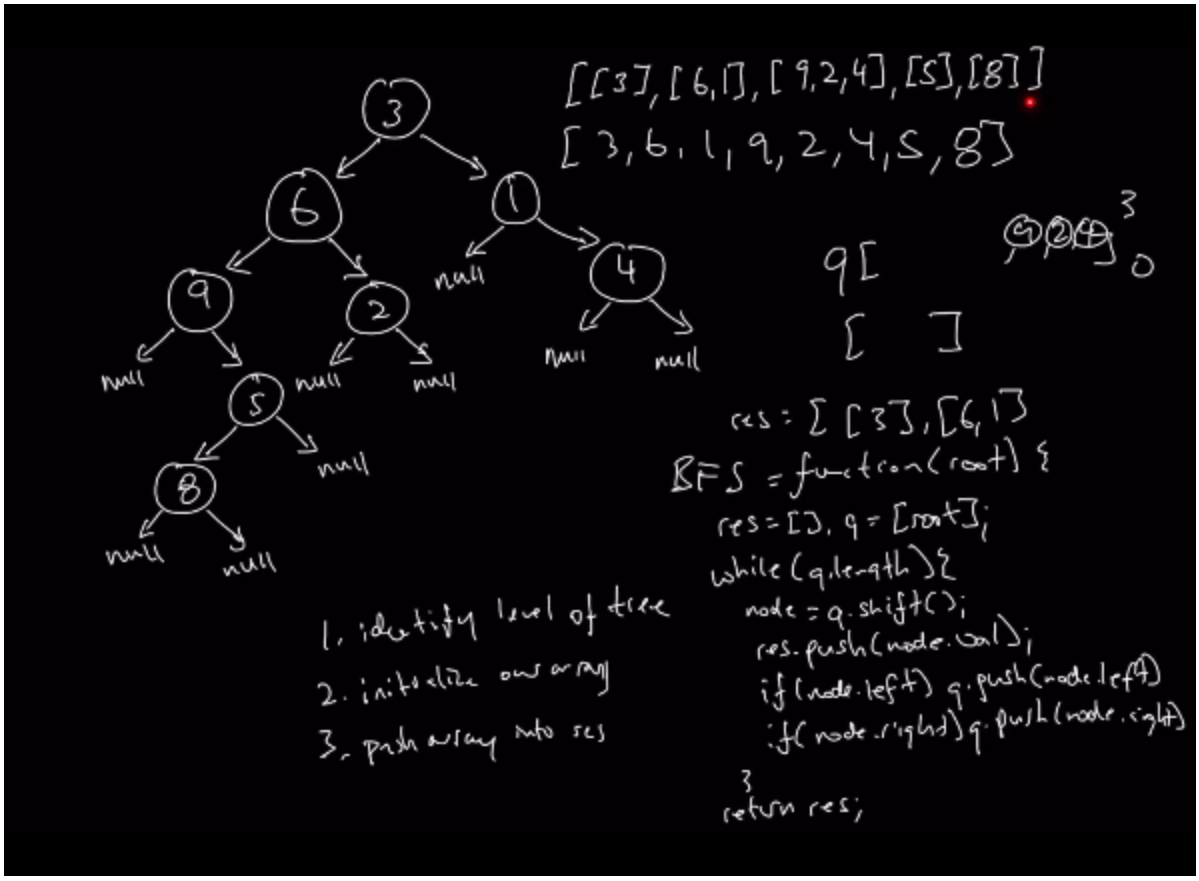
```
const binarySearch = function (array, target) {
    let left = 0, right = array.length - 1;
    while (left <= right) {
        const mid = Math.floor((left + right) / 2);
        const foundVal = array[mid];
        if (foundVal === target) {
            return mid;
        } else if (foundVal < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

BFS - Breadth First Search and DFS - Depth First Search

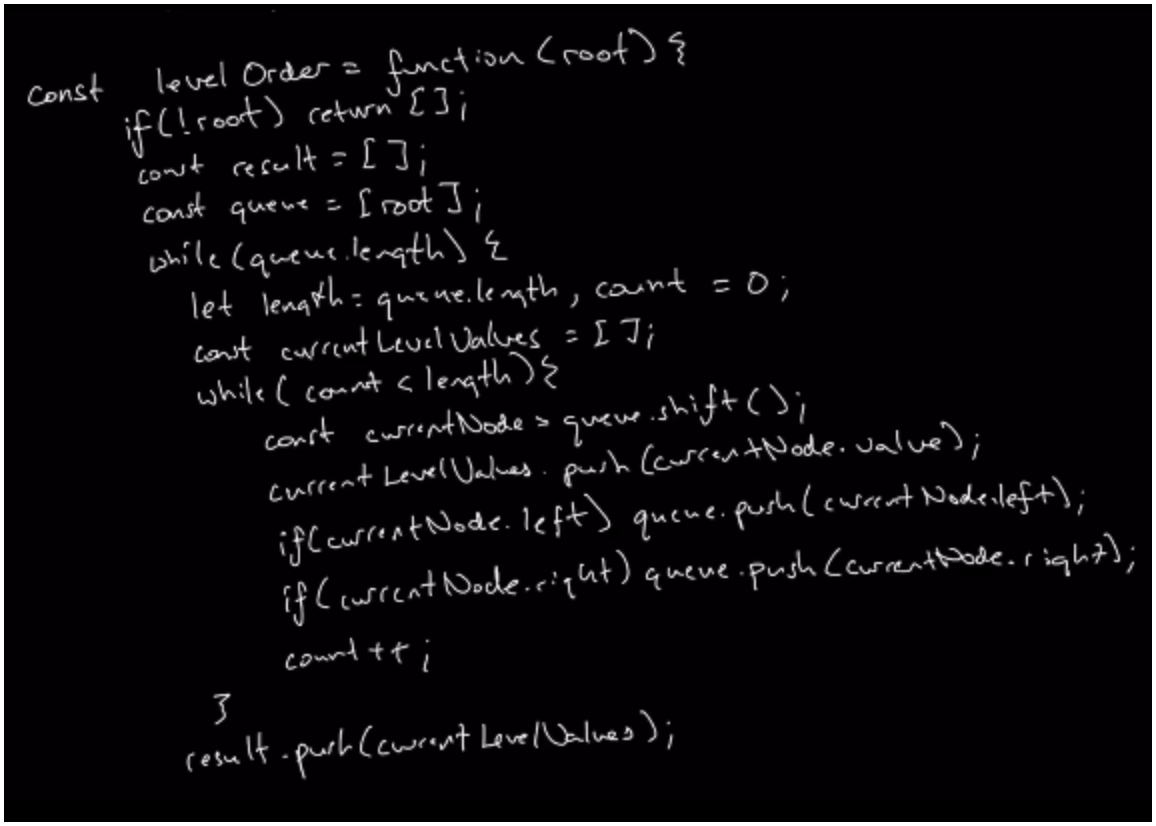




BFS:
This is level order

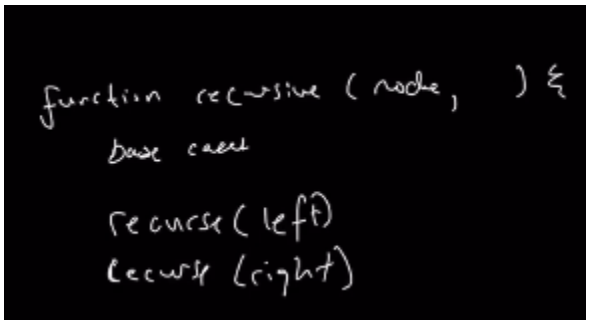


We iterate through an array that grows in the time

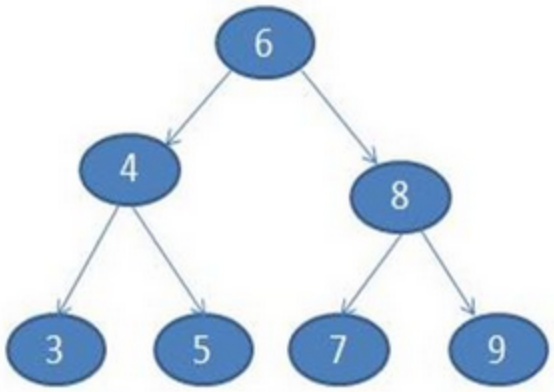


At the end return the result

DFS:



Binary tree, preorder | inorder | postorder



Preorder: Root-Left-Right

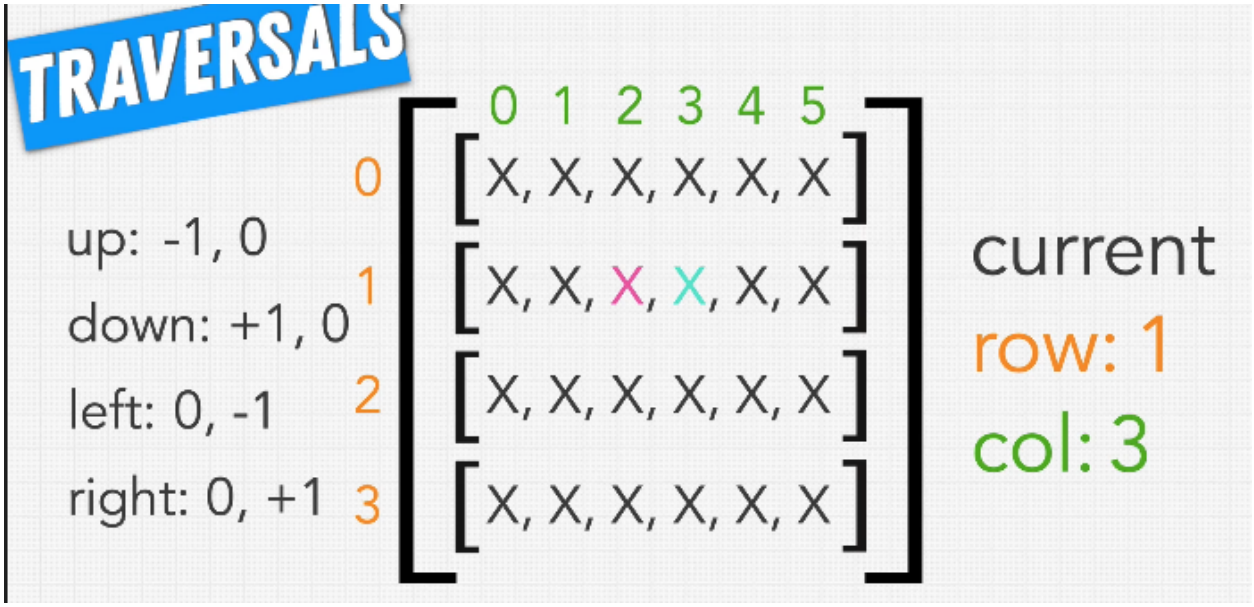
Inorder: Left-Root-Right

Postorder: Left-Right-Root

Preorder : 6, 4, 3, 5, 8, 7, 9
Inorder : 3, 4, 5, 6, 7, 8, 9
Postorder: 3, 5, 4, 7, 9, 8, 6

But the order of left and right can be interchanged like mirror.
Preorder: Root-right-left
Inorder: right-root-left
Postorder: right-left-root

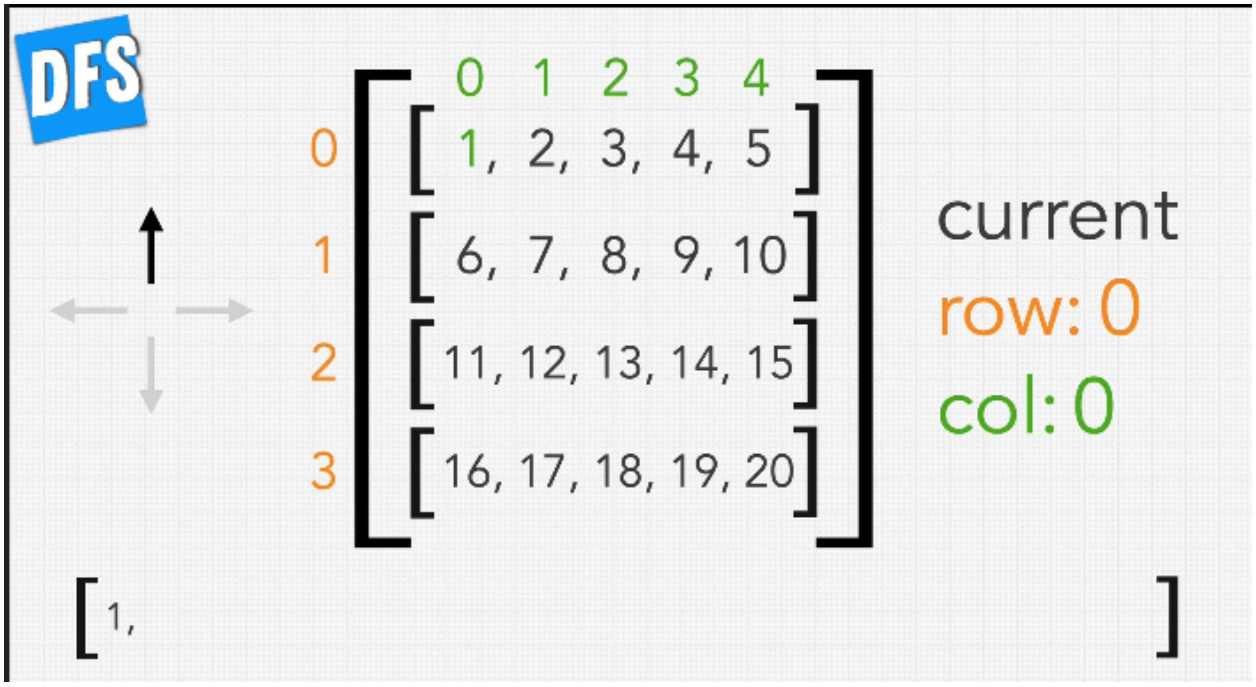
2-D Arrays traversals



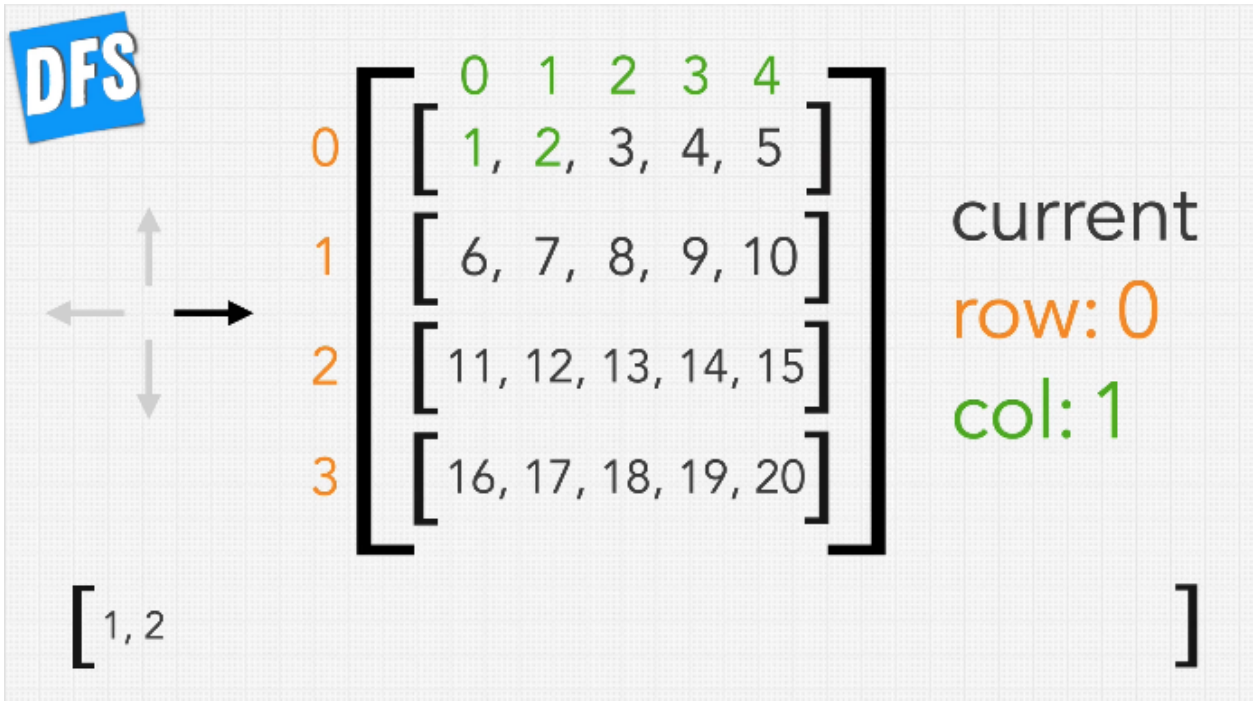
2-D Arrays BFS and DFS

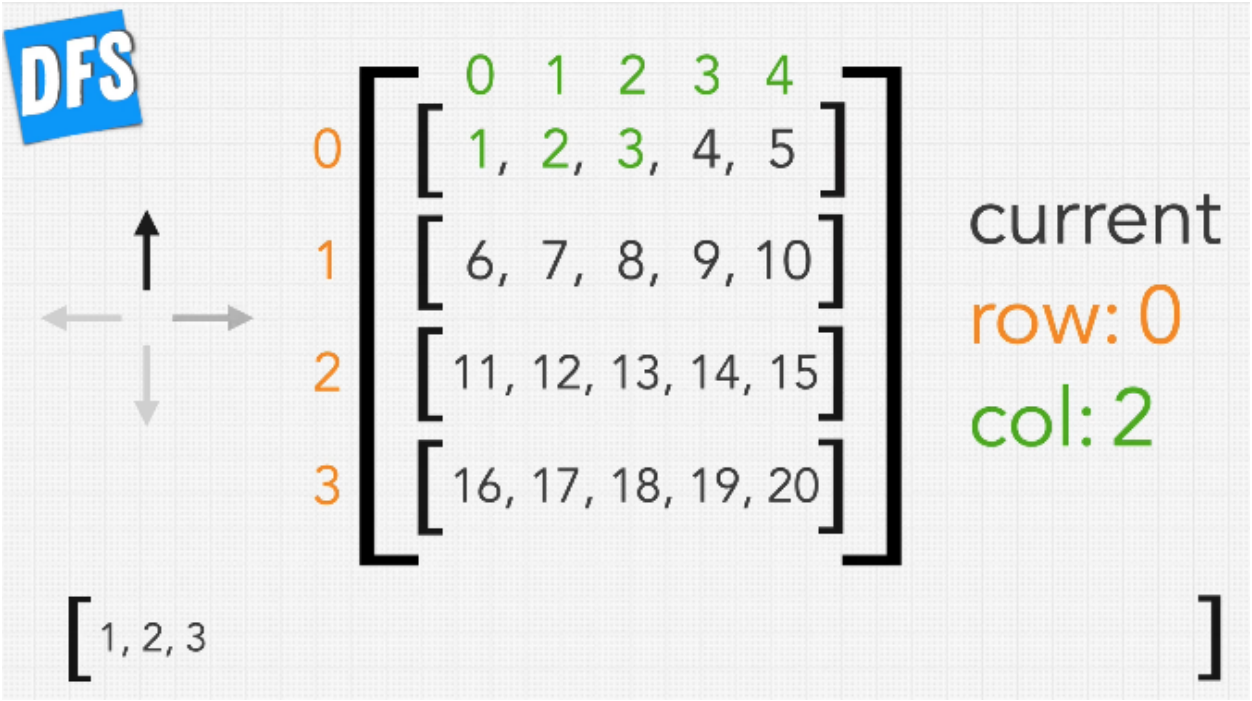
DFS

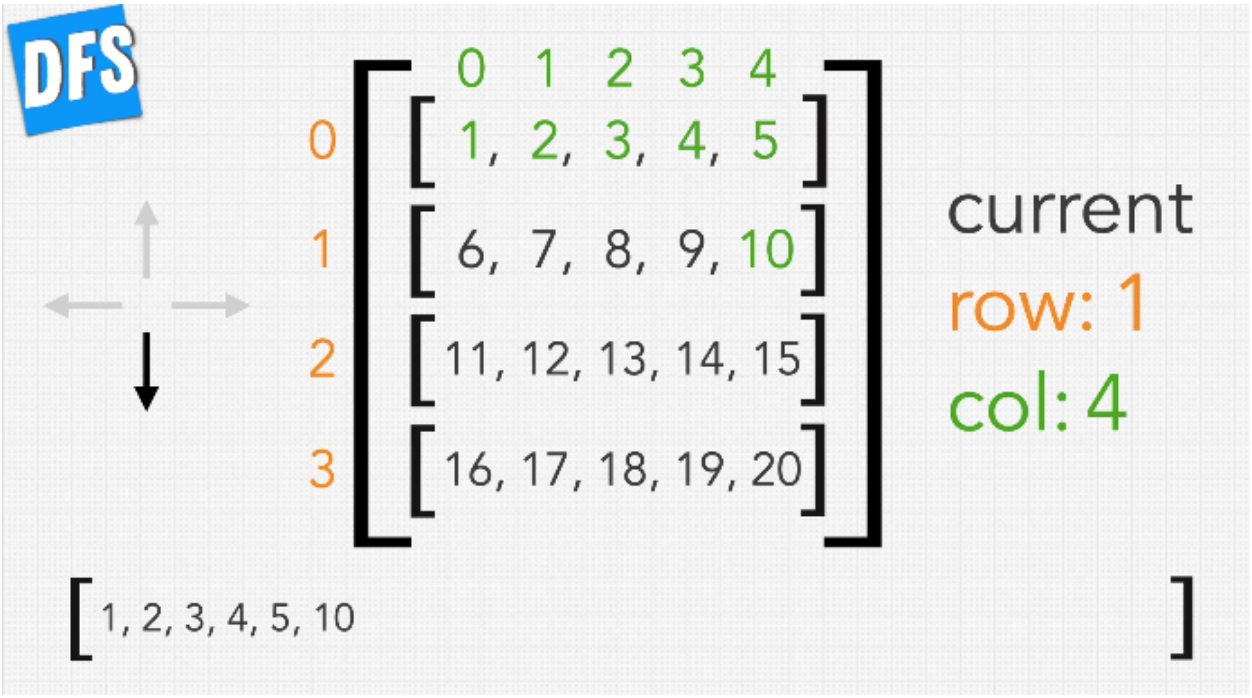
DFS we explore one of the four directions (in binary tree is two)

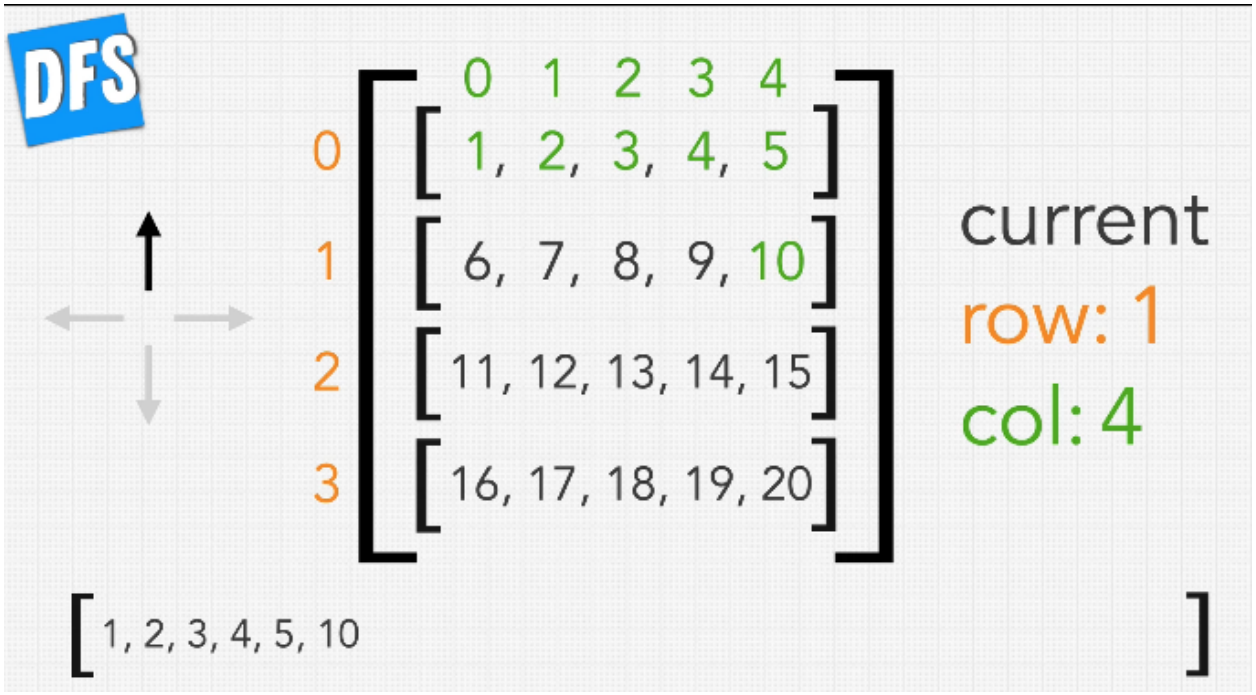


If exist wall, continue

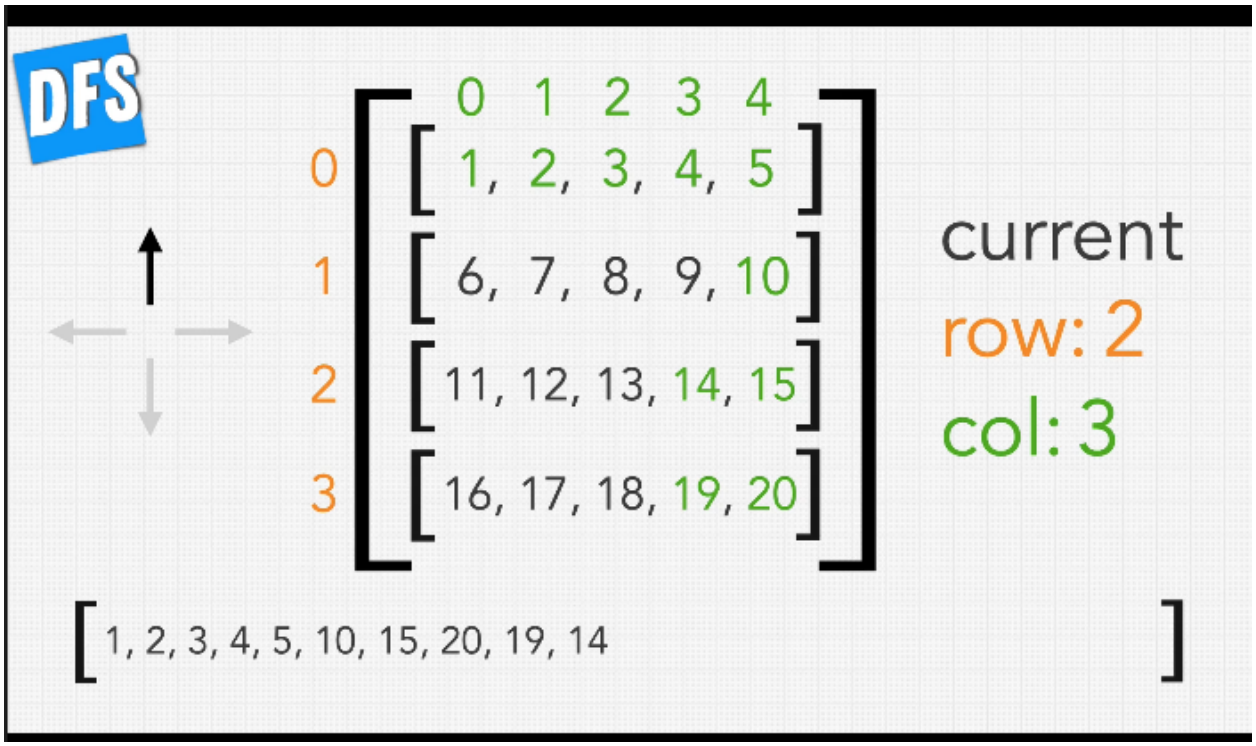


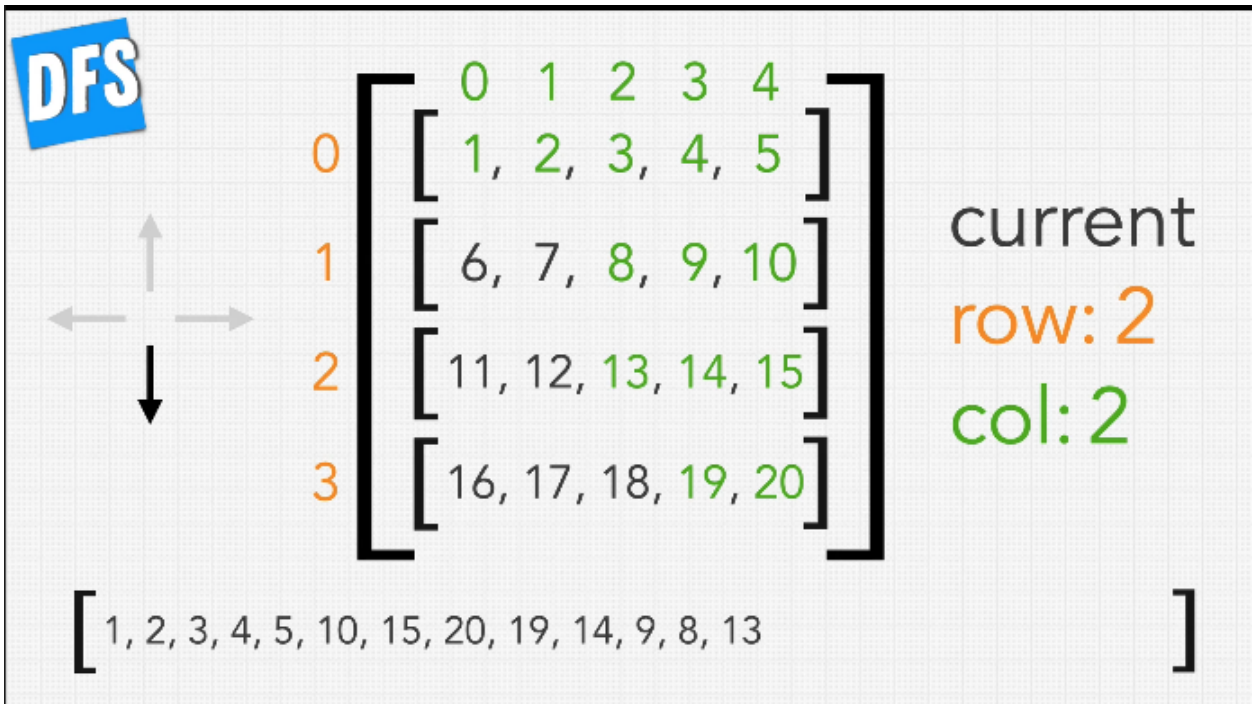

DFS
0 1 2 3 4
0 [1, 2, 3, 4, 5]
1 [6, 7, 8, 9, 10]
2 [11, 12, 13, 14, 15]
3 [16, 17, 18, 19, 20]
current
row: 0
col: 2
[1, 2, 3
]

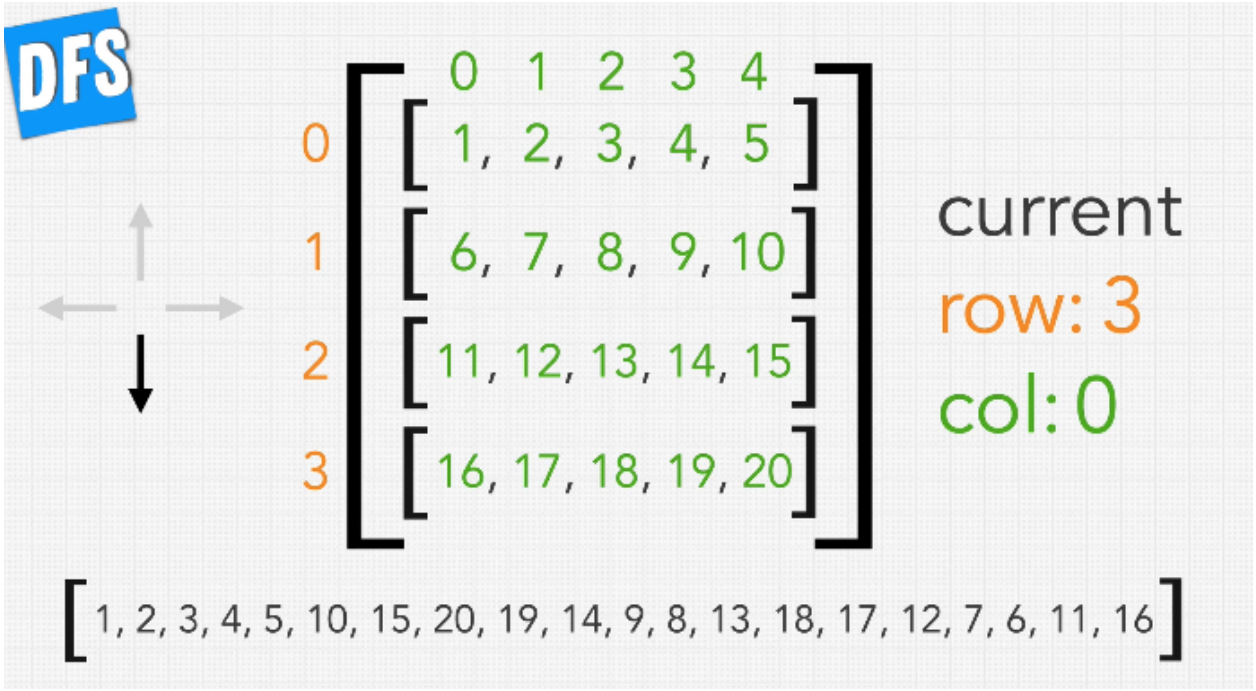

DFS
0 1 2 3 4
0 [1, 2, 3, 4, 5]
1 [6, 7, 8, 9, 10]
2 [11, 12, 13, 14, 15]
3 [16, 17, 18, 19, 20]
current
row: 1
col: 4
[1, 2, 3, 4, 5, 10
]



In this case 5 is already explored







Using DFS we can iterate throught the matrix even in espilar with “right, down, left, up”
This use a recursive method

```
const traversalDFS = function(matrix) {
  const seen = new Array(matrix.length).fill(0)
                .map(() => new Array(matrix[0].length)
                .fill(false));

  const values = [];

  dfs(matrix, 0, 0, seen, values);
  return values;
}
```

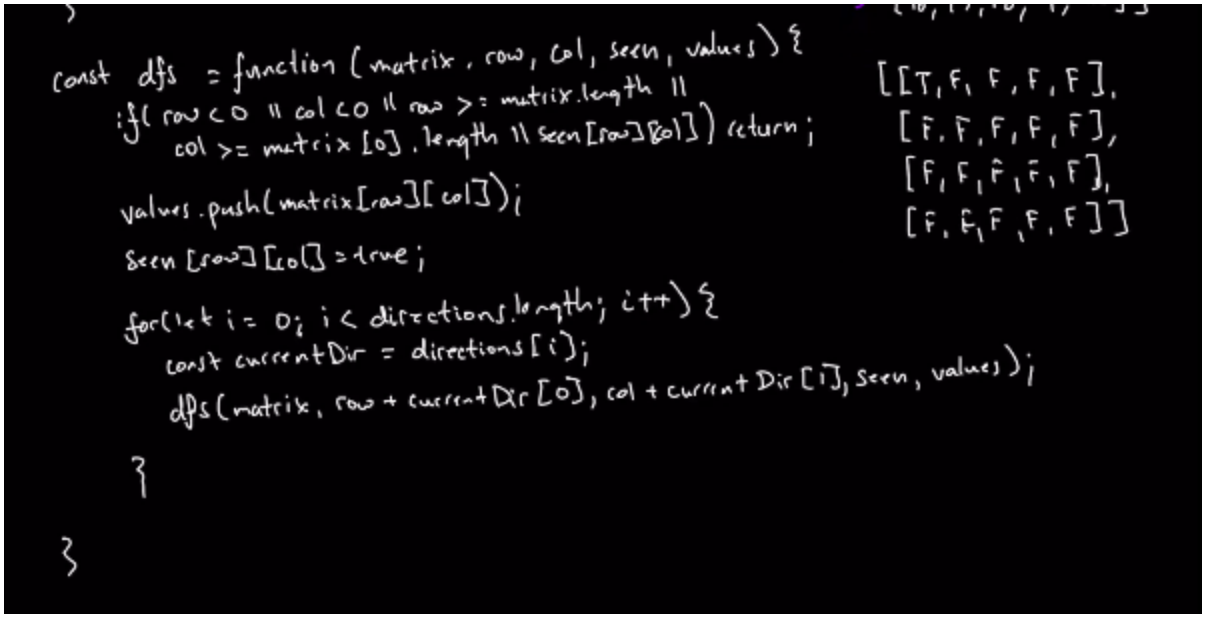
```
[[F, F, F, F, F],
 [F, F, F, F, F],
 [F, F, F, F, F],
 [F, F, F, F, F]]
```

```
const directions = [[-1, 0], // up
                    [0, 1], // right
                    [1, 0], // down
                    [0, -1]] // left
```

```
    0  1  2  3  4
0 [[1, 2, 3, 4, 5],
1  [6, 7, 8, 9, 10],
2  [11, 12, 13, 14, 15],
3  [16, 17, 18, 19, 20]]
```

```
const traversalDFS = function(matrix) {
  const seen = new Array(matrix.length).fill(0)
                .map(() => new Array(matrix[0].length)
                .fill(false));
```

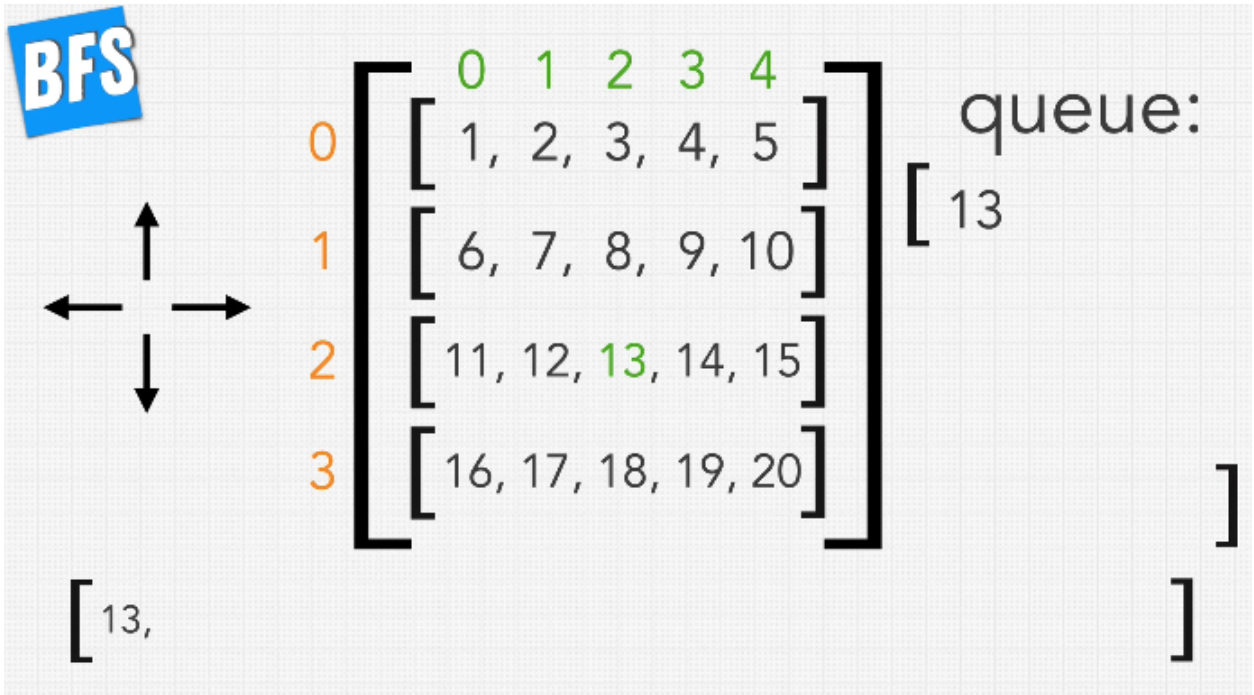
```
[[F, F, F, F, F],
 [F, F, F, F, F],
 [F, F, F, F, F],
 [F, F, F, F, F]]
```

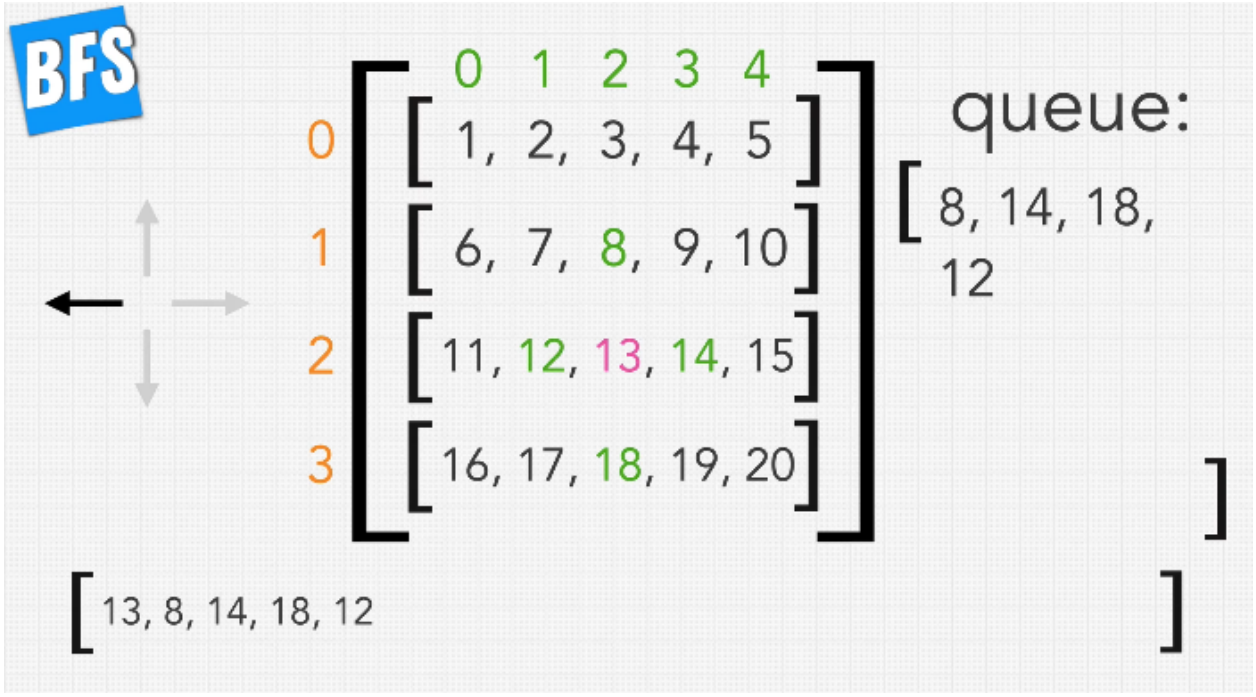


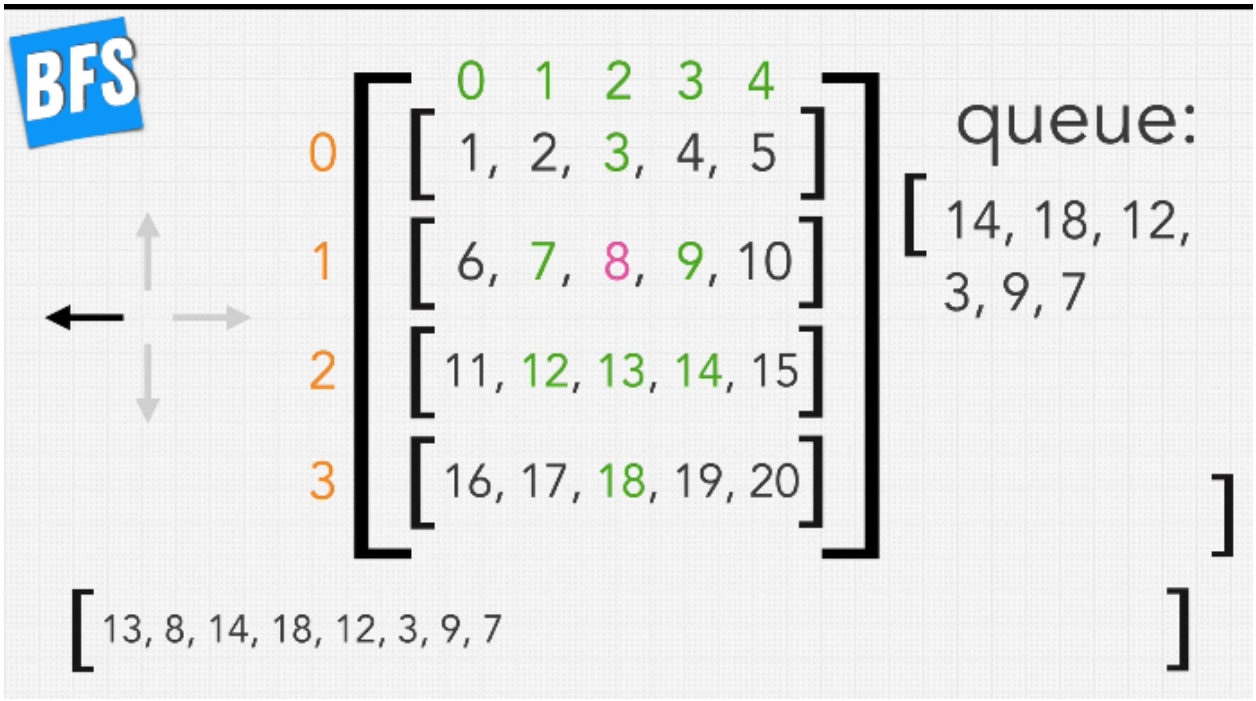
Depending on the order we de directions We can go in a lot of different patterns directions.

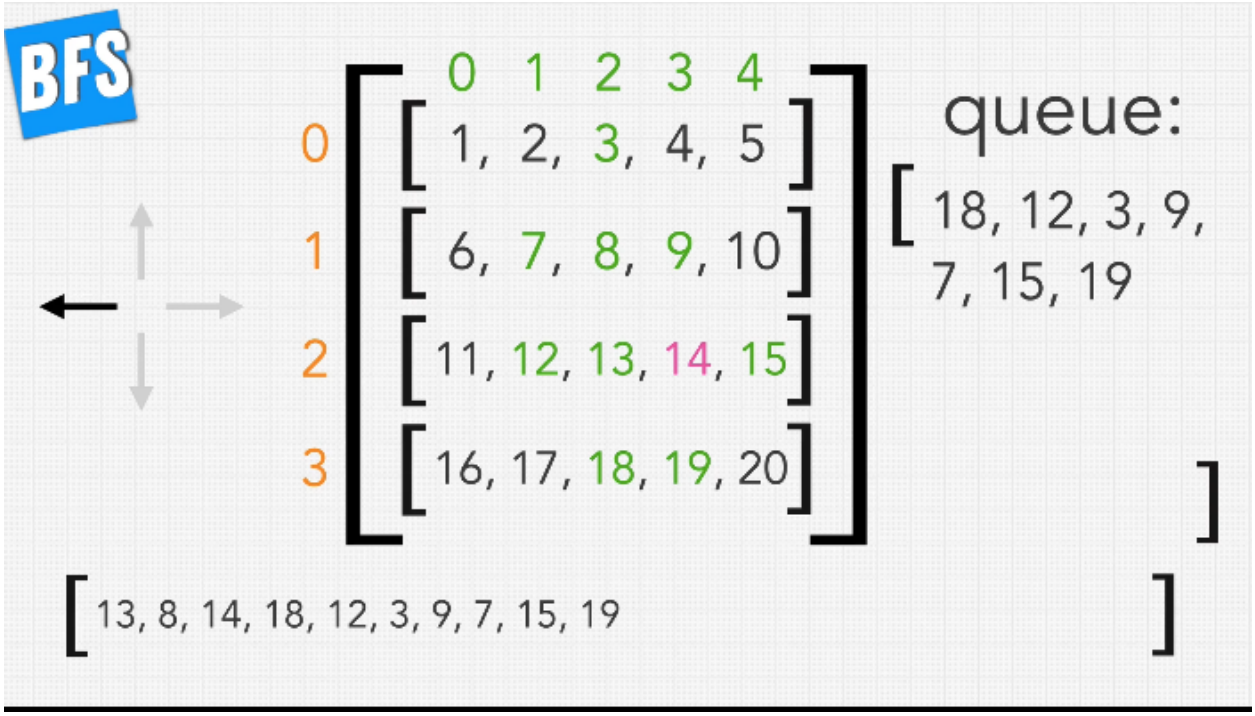
BFS

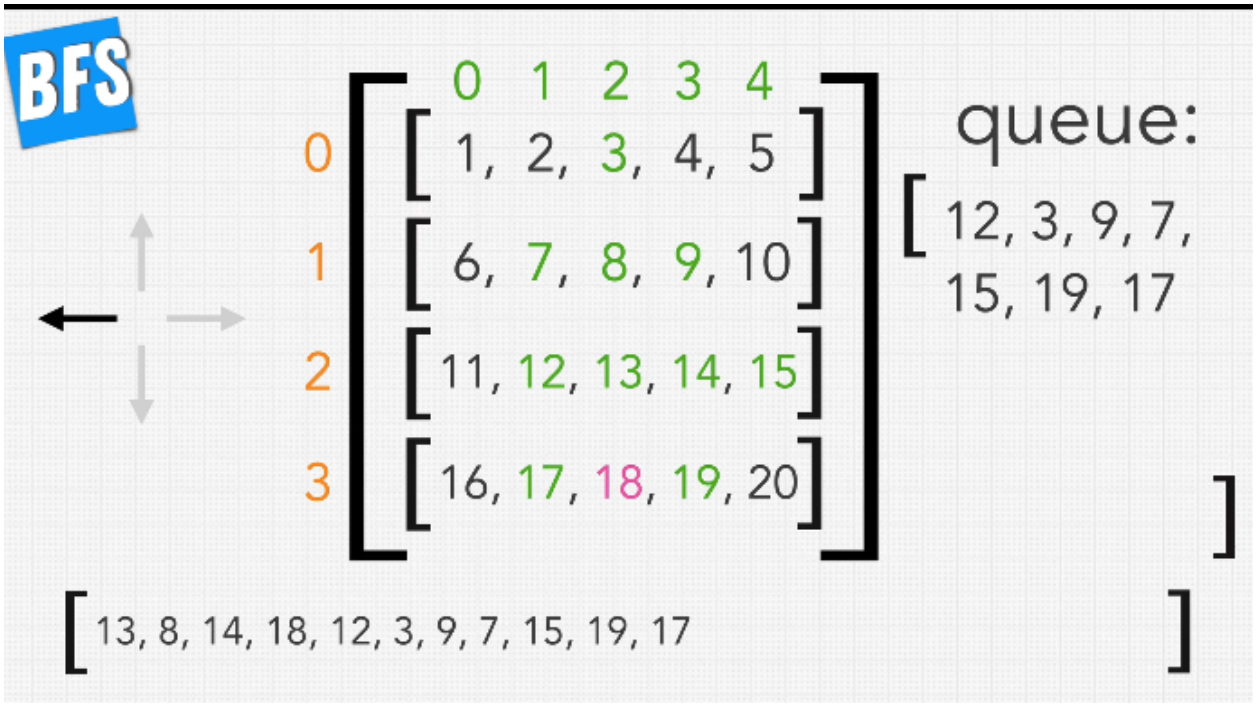
We chose a starting point



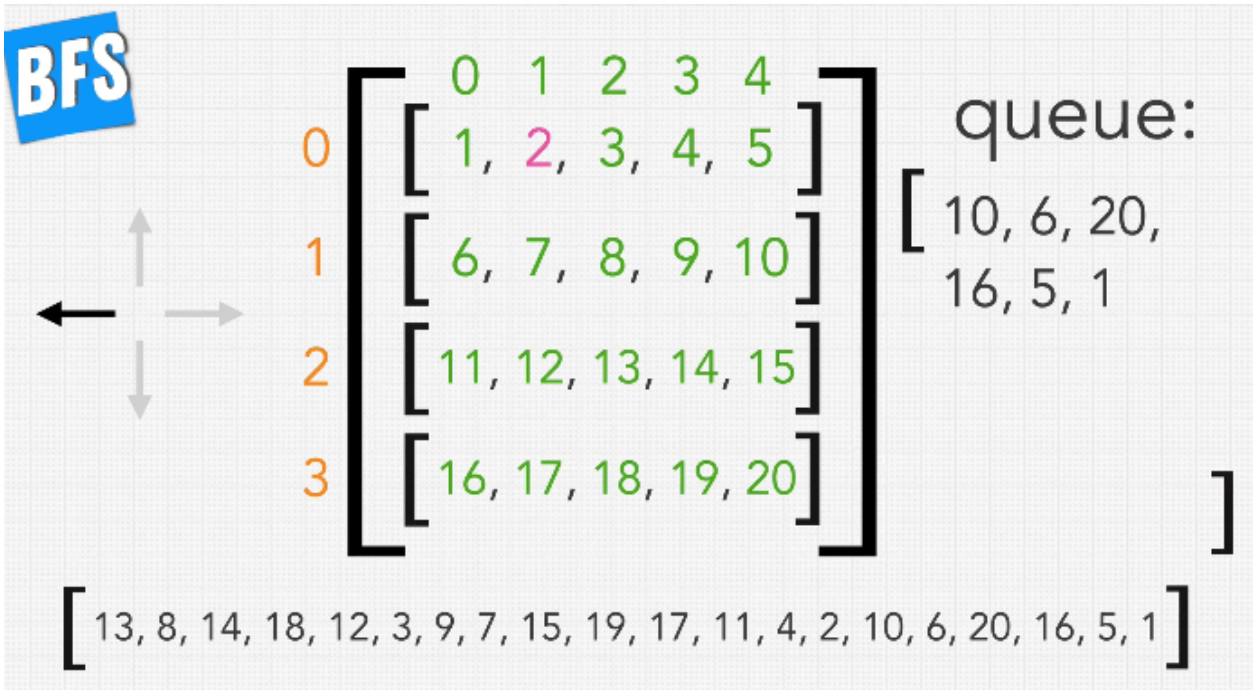

BFS
0 1 2 3 4
0 [1, 2, 3, 4, 5]
1 [6, 7, 8, 9, 10]
2 [11, 12, 13, 14, 15]
3 [16, 17, 18, 19, 20]
queue:
[8, 14, 18, 12
]
[13, 8, 14, 18, 12


BFS
0 1 2 3 4
0 [1, 2, 3, 4, 5]
1 [6, 7, 8, 9, 10]
2 [11, 12, 13, 14, 15]
3 [16, 17, 18, 19, 20]
queue:
[14, 18, 12,
3, 9, 7
]
[13, 8, 14, 18, 12, 3, 9, 7

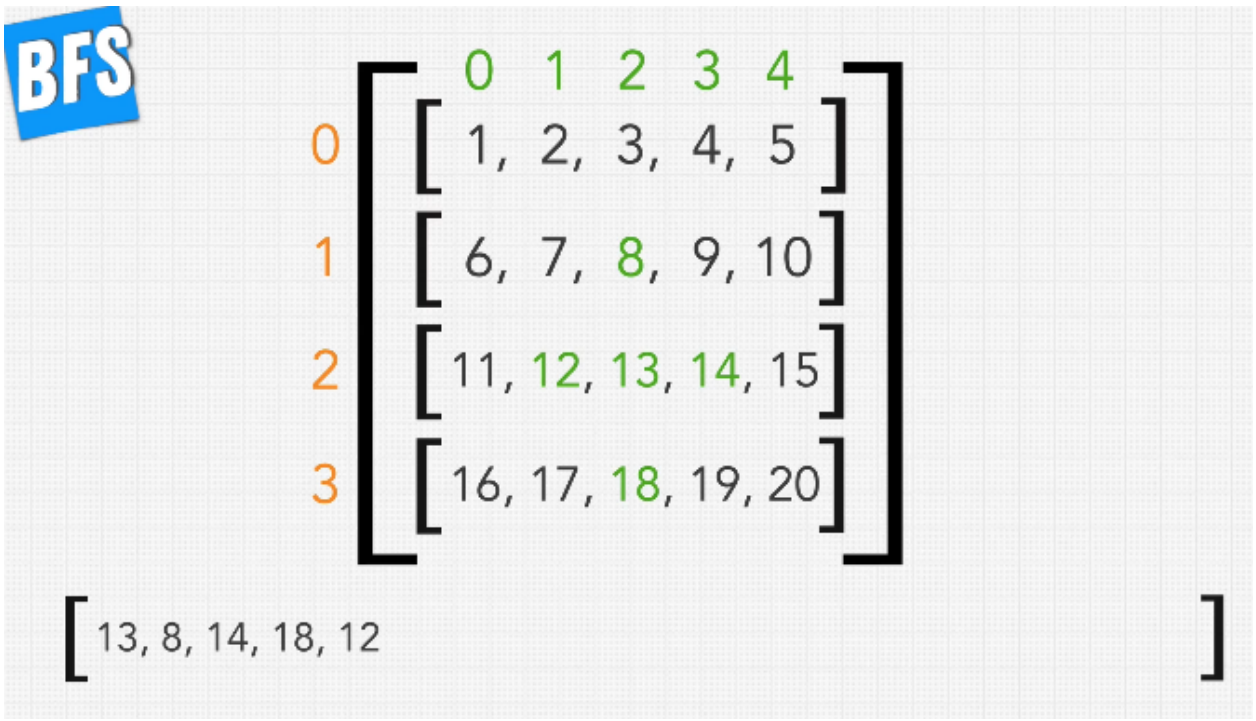




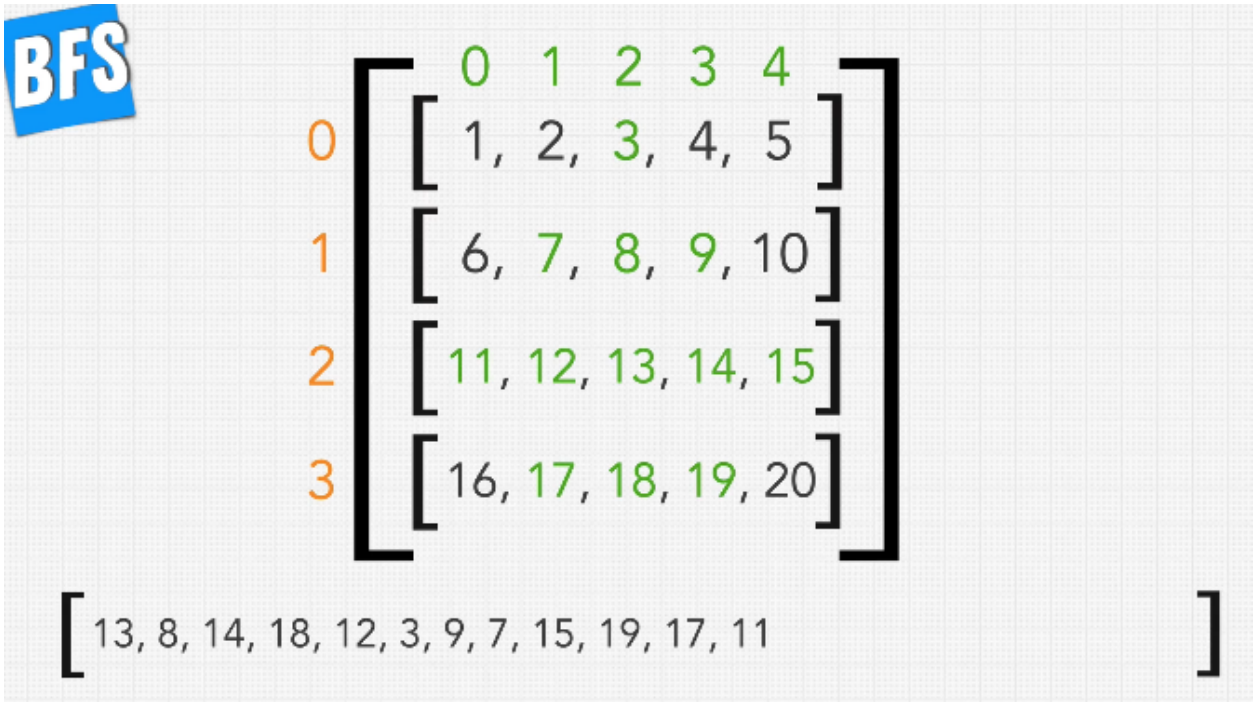
With this we expand the search in a ring pattern starting from the chosen point

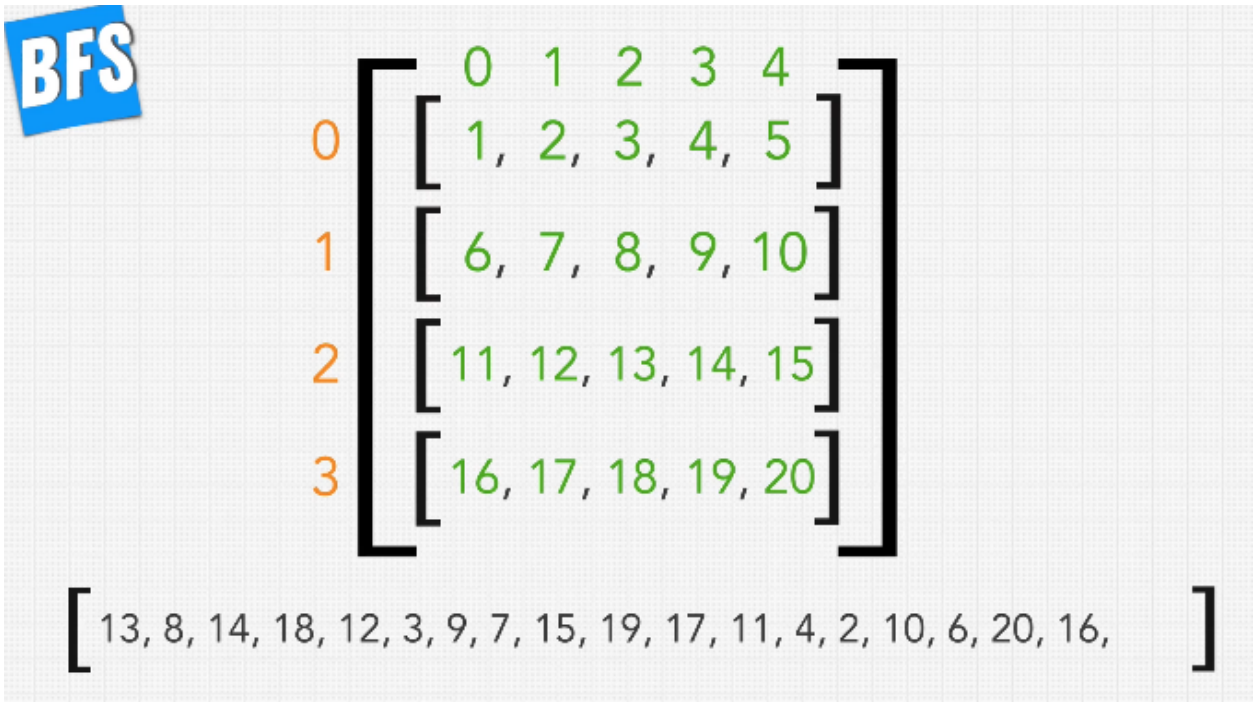


This is the ring



The same ring expands





That shape is expanding

```
const directions = [[-1,0], up
                    [0,1], right
                    [1,0], down
                    [0,-1]] left
```

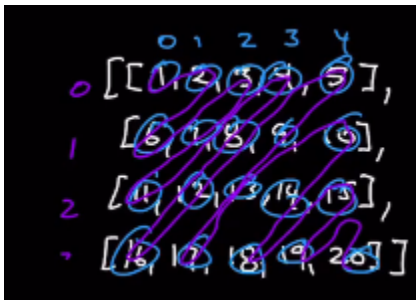
```
    0  1  2  3  4
0 [[1, 2, 3, 4, 5],
1  [6, 7, 8, 9, 10],
2  [11,12,13,14,15],
3  [16,17,18,19,20]]
```

```
const traversalBFS = function(matrix) {
  const seen = new Array(matrix.length).fill(0)
                .map(() => new Array(matrix[0].length).fill(false));

  const values = [];
  const queue = [[0,0]];
```

```
while (queue.length) {
    const currentPos = queue.shift();
    const row = currentPos[0];
    const col = currentPos[1];
    if(row < 0 || row >= matrix.length || col < 0 || col >= matrix[0].length
        || seen[row][col]) {
        continue;
    }

    seen[row][col] = true;
    values.push(matrix[row][col]);
```

```
    for (let i = 0; i < directions.length; i++) {
      const currentDir = directions[i];
      queue.push([row + currentDir[0], col + currentDir[1]]);
    }
  }

  return values.
```

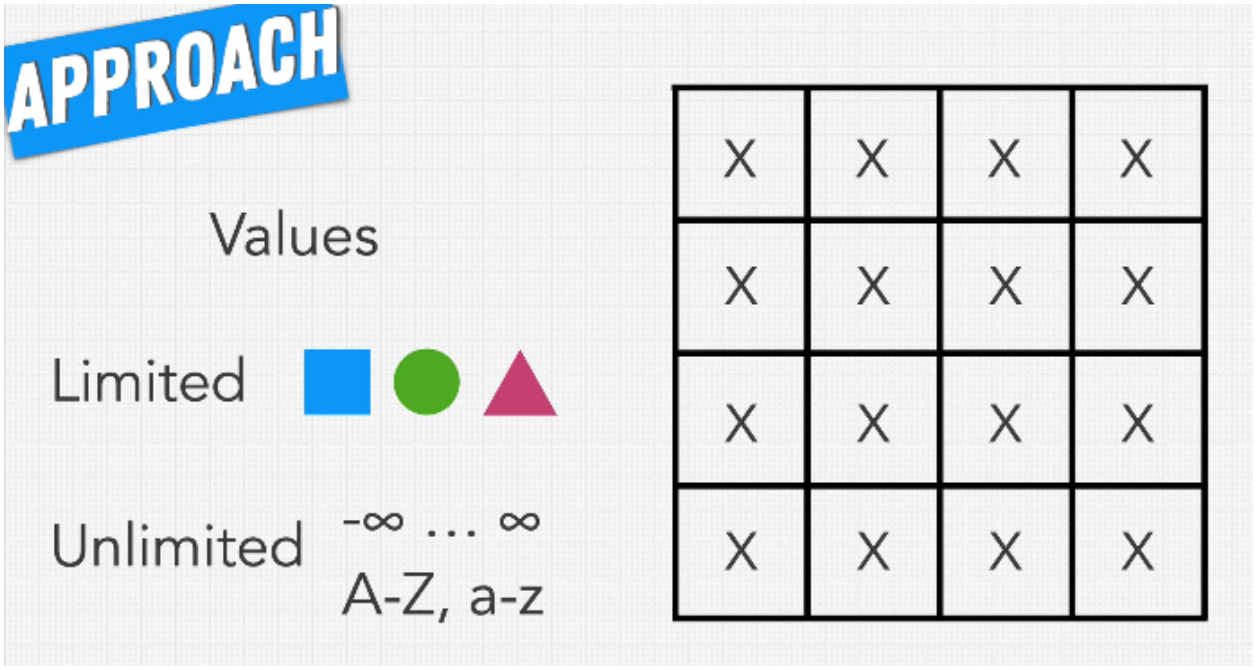
This append to the queue in this way



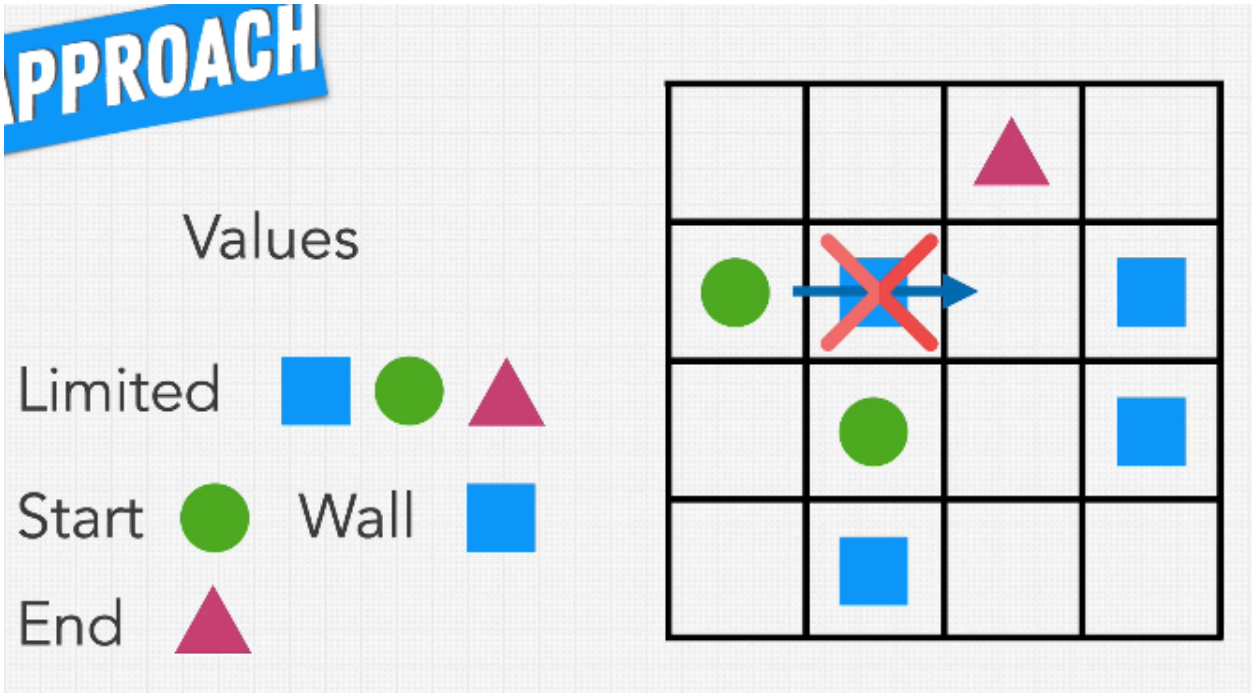
DFS and BFS only cares about adjacent values

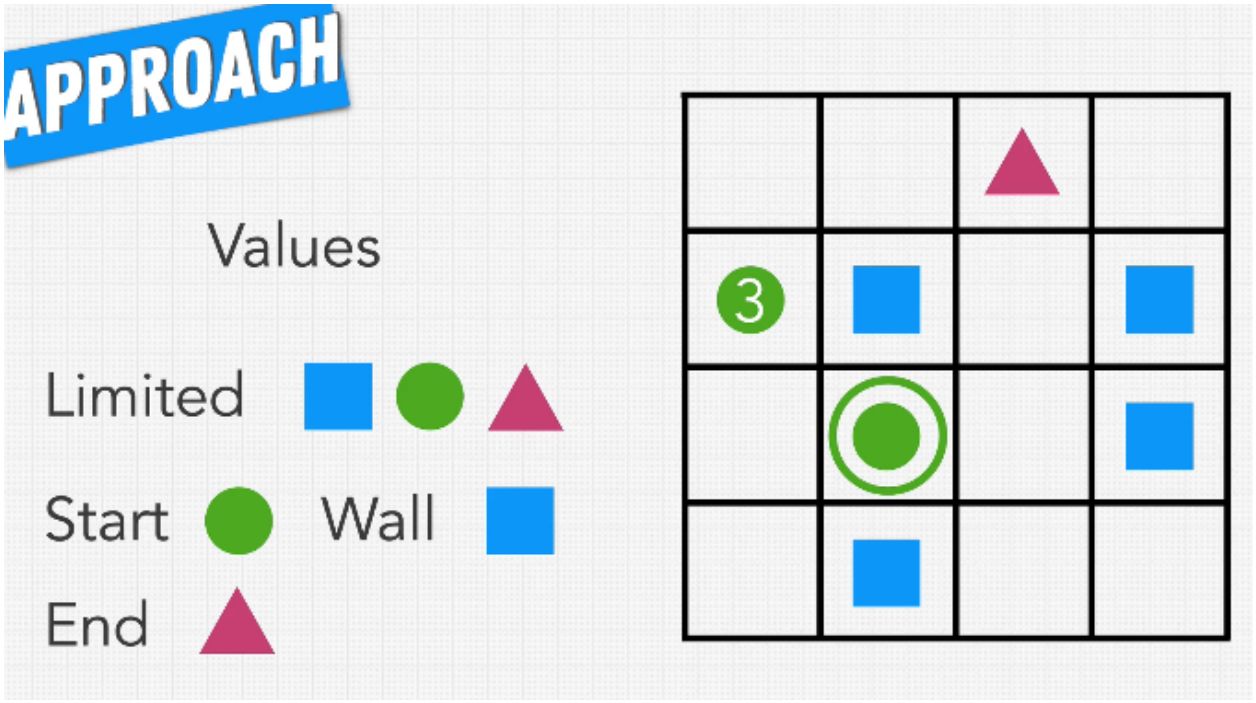
2-D Matrix approach

First think in what values will exist in matrix



- Is there a relationship between the values?



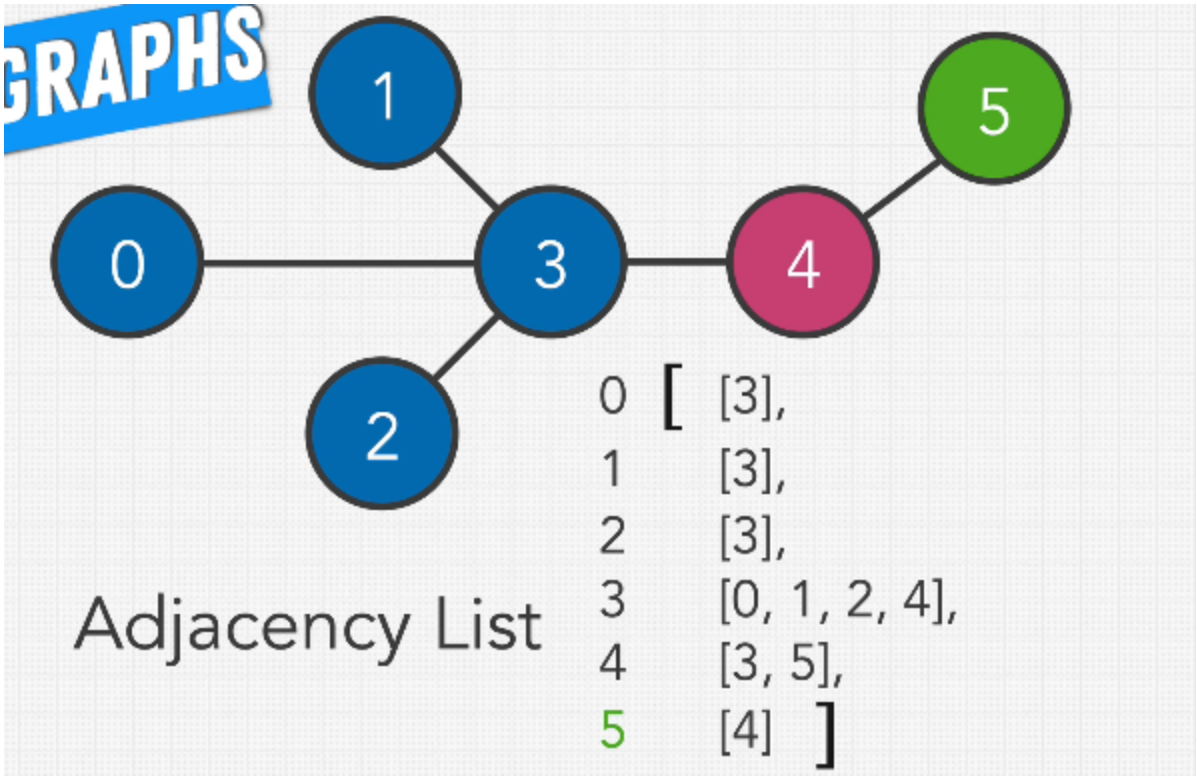


In sequential order we get the total steps for each element

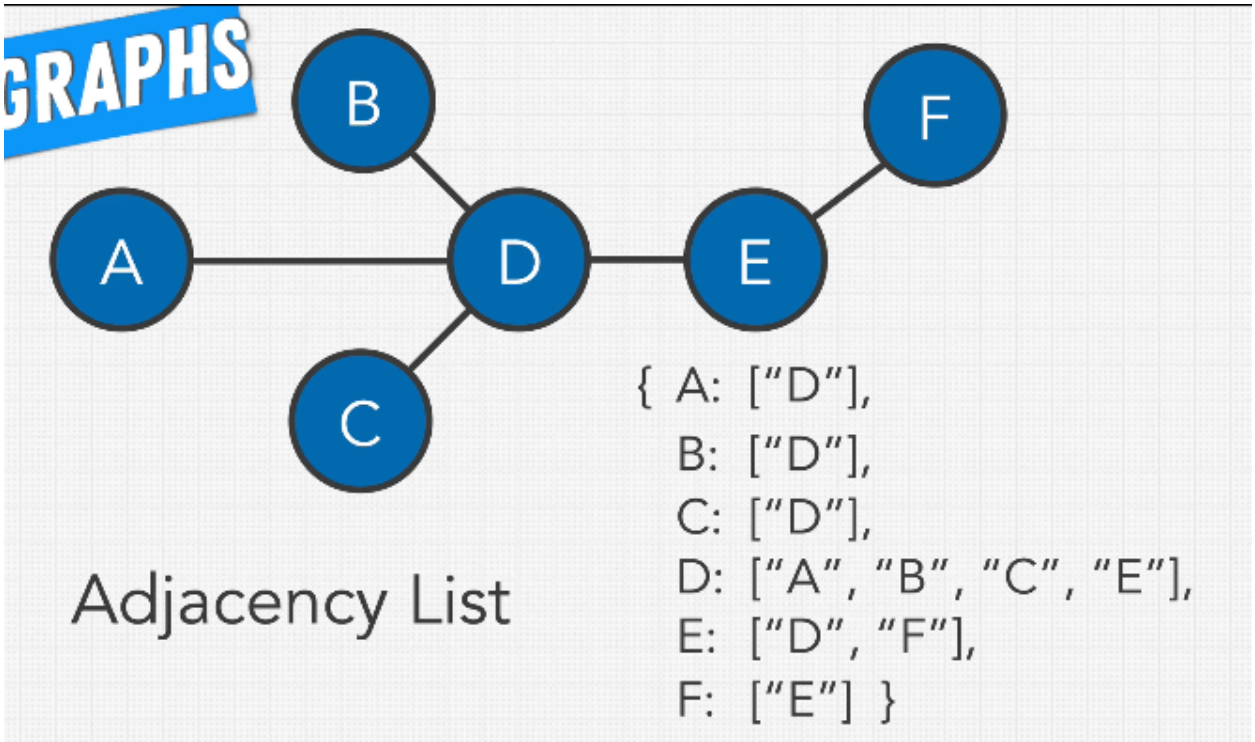
- Is there a subproblem that we can solve again and again to get the answer?

Graphs - Adjacency list

Every index corresponds to a node with the same name identifier

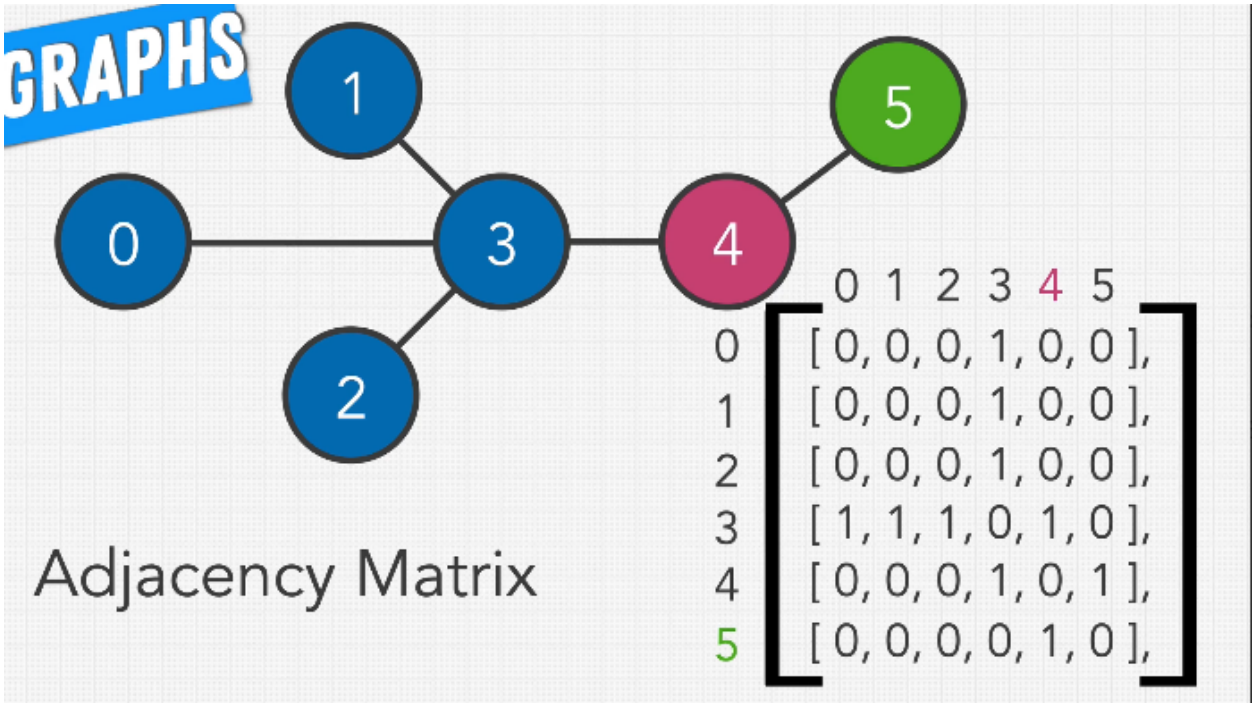

GRAPHS
1
5
0
3
4
2
Adjacency List
0 [[3],
1 [3],
2 [3],
3 [0, 1, 2, 4],
4 [3, 5],
5 [4]]

Any name:


GRAPHS
B
F
A
D
E
C
Adjacency List
{ A: ["D"],
B: ["D"],
C: ["D"],
D: ["A", "B", "C", "E"],
E: ["D", "F"],
F: ["E"] }

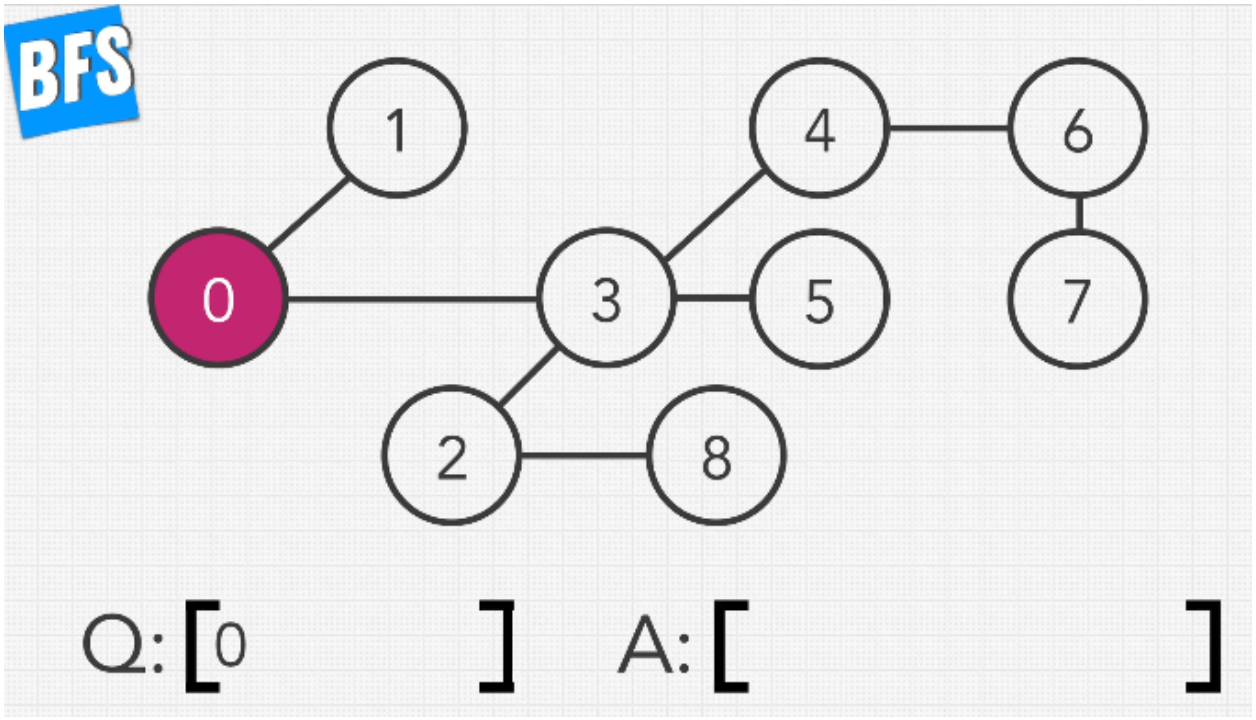
We can easily continue traverse with this representation A -> D, D=[A, B, C, E], and so on

Graphs - Adjacency matrix

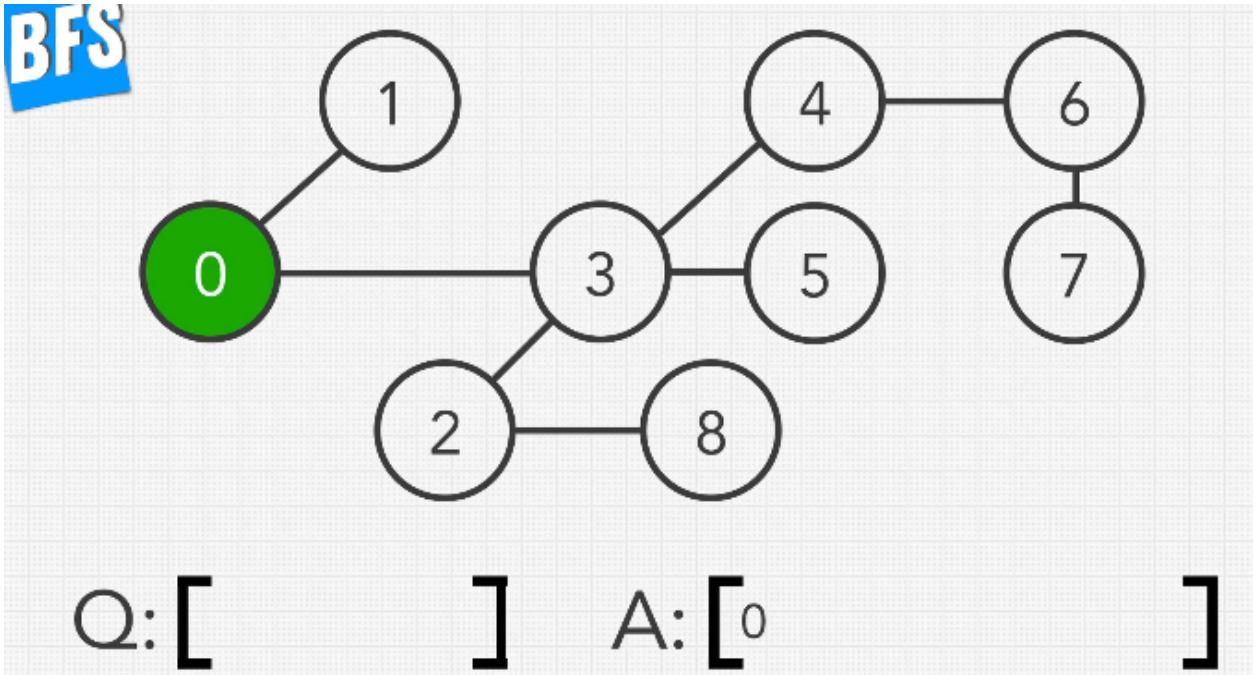


It uses more space

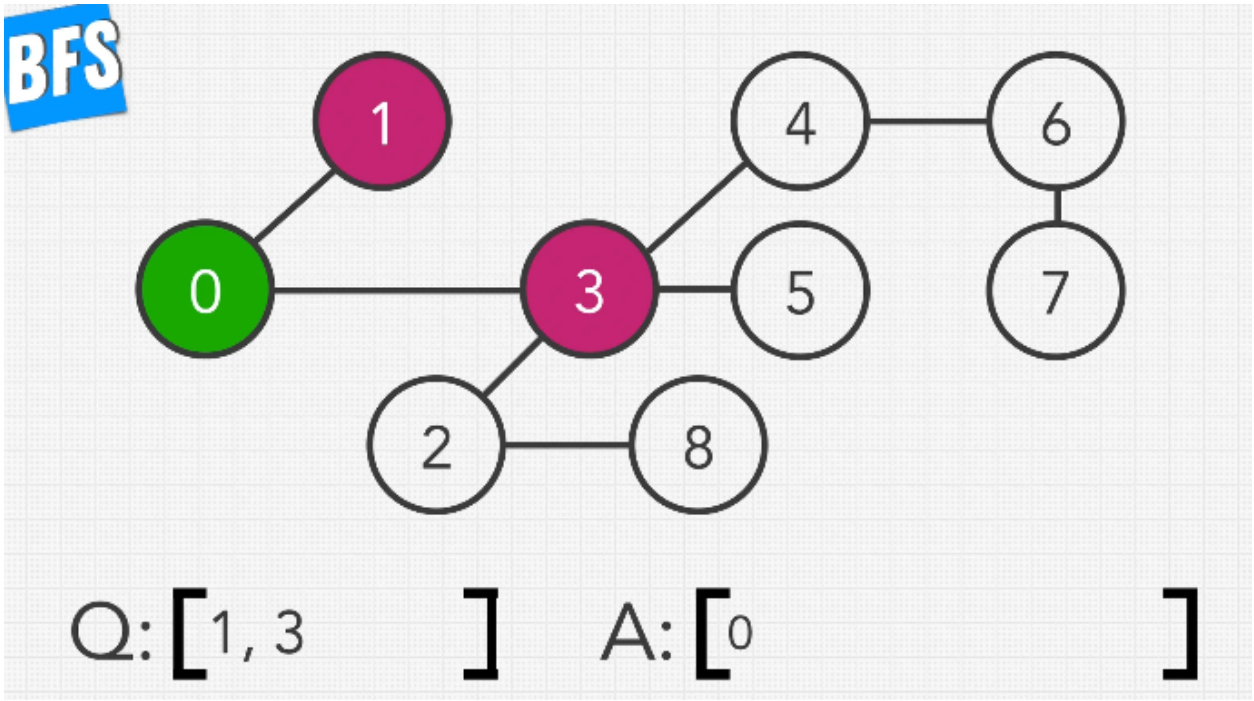
Graphs BFS

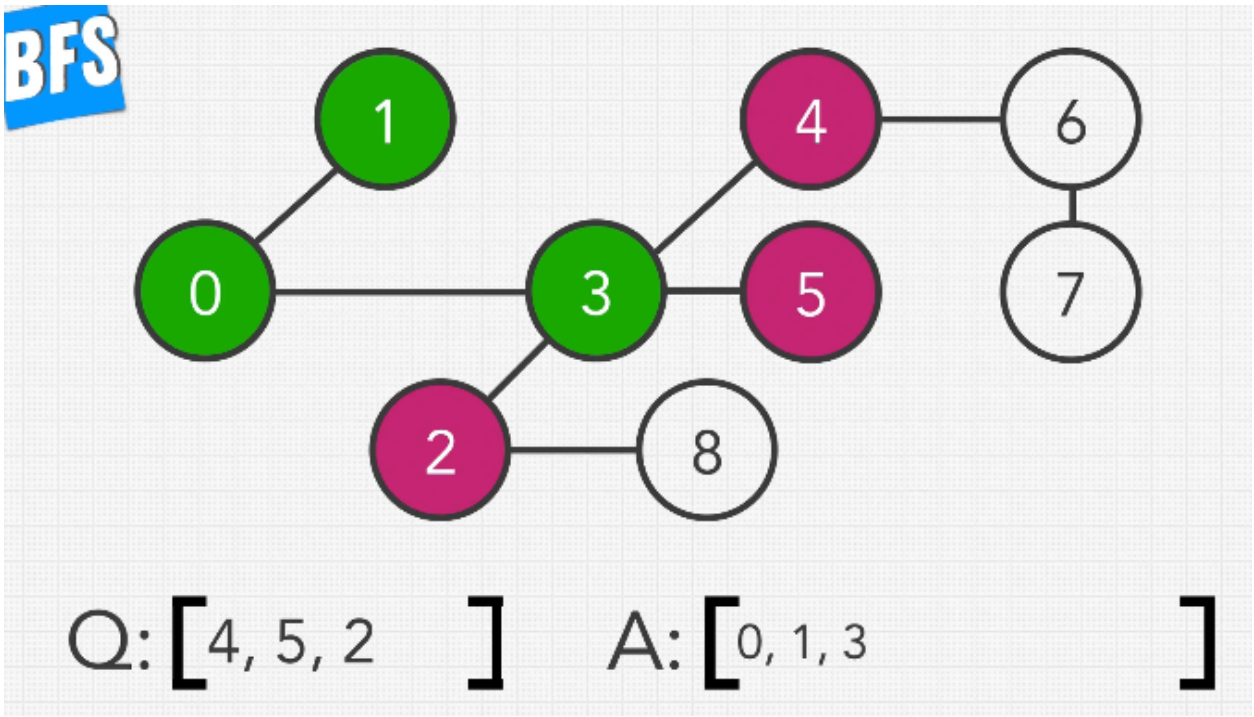


We select a starting point

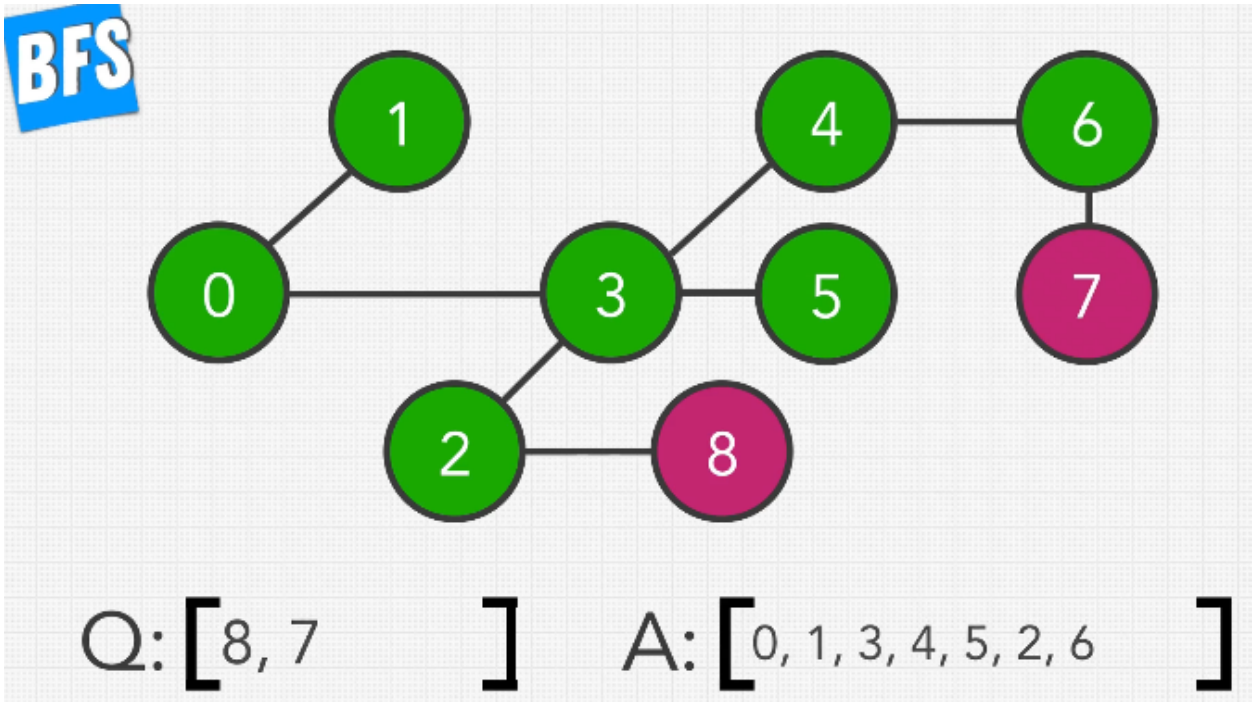


We process this value

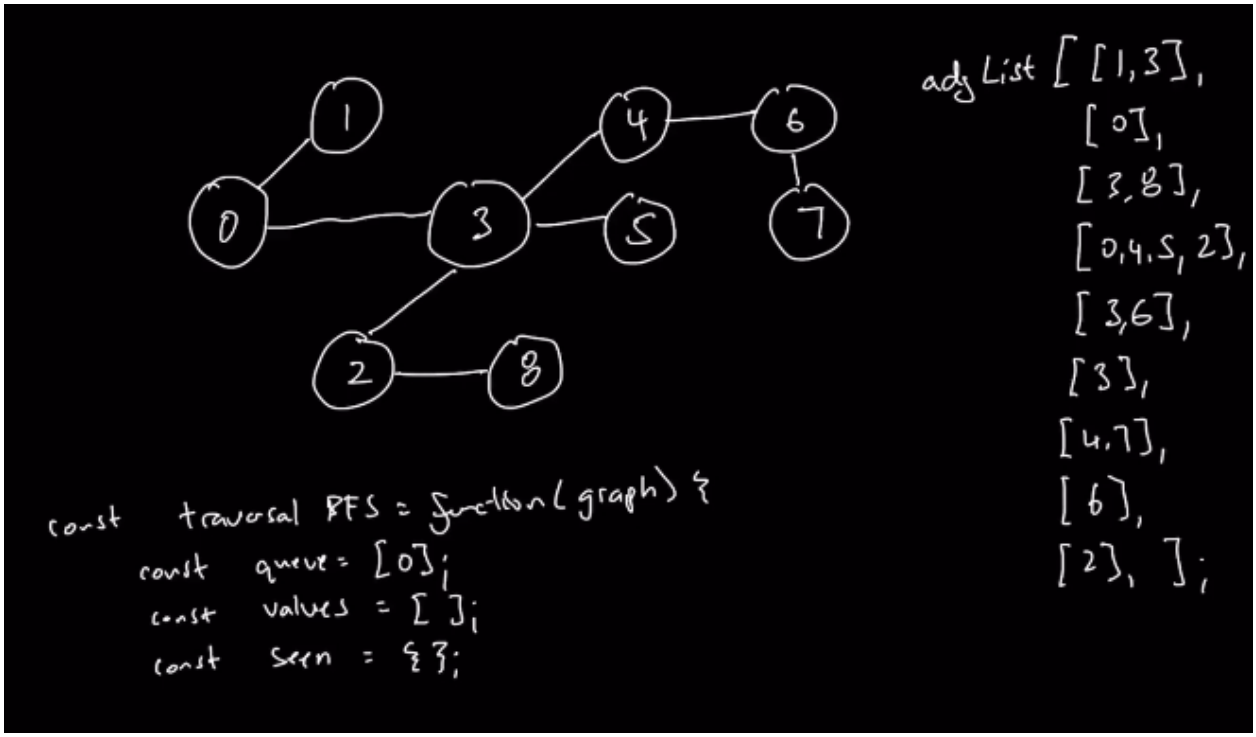


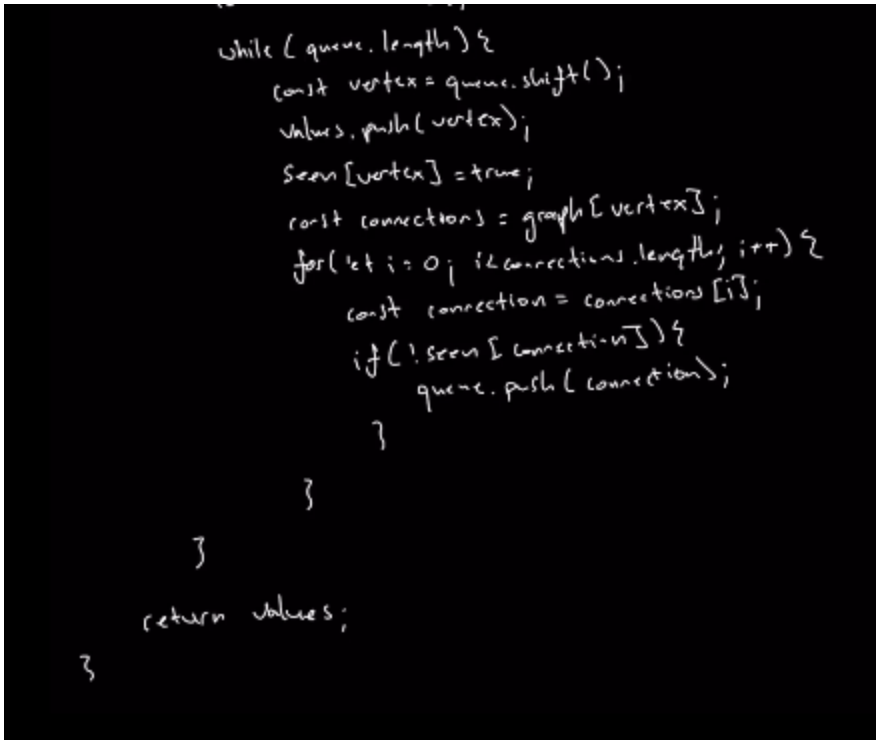


BFS always use a queue



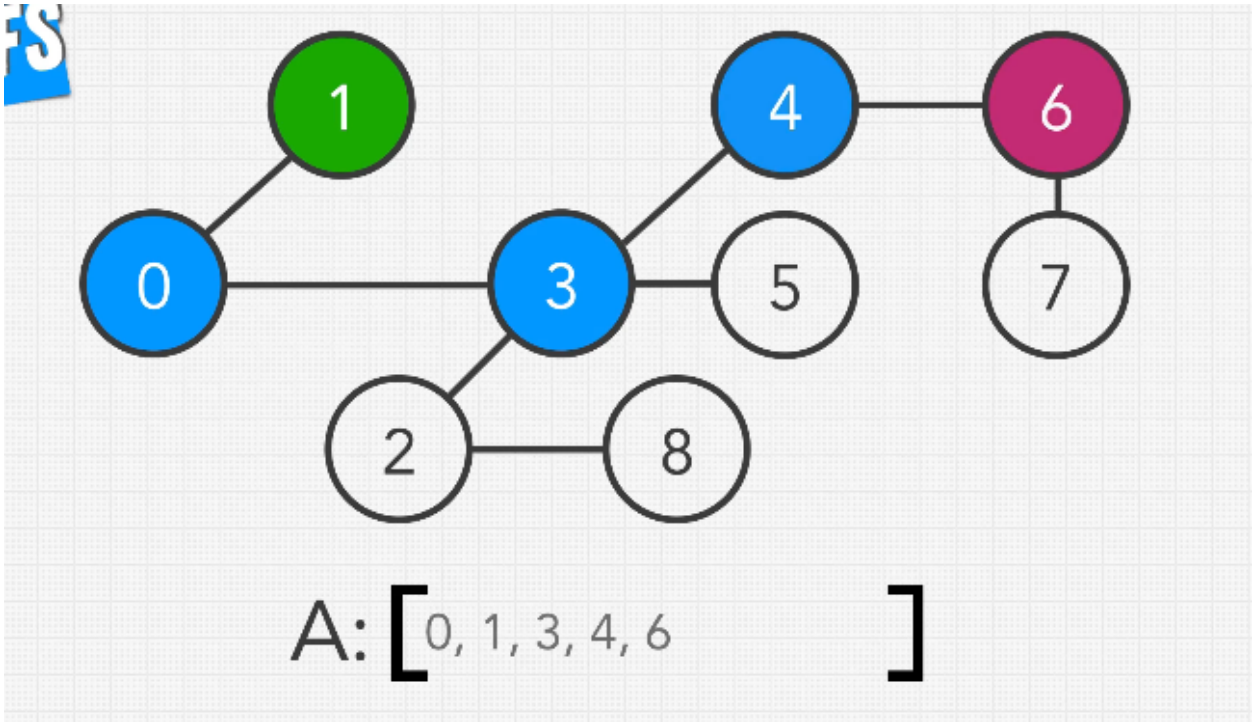
Adjacency list to BFS representation



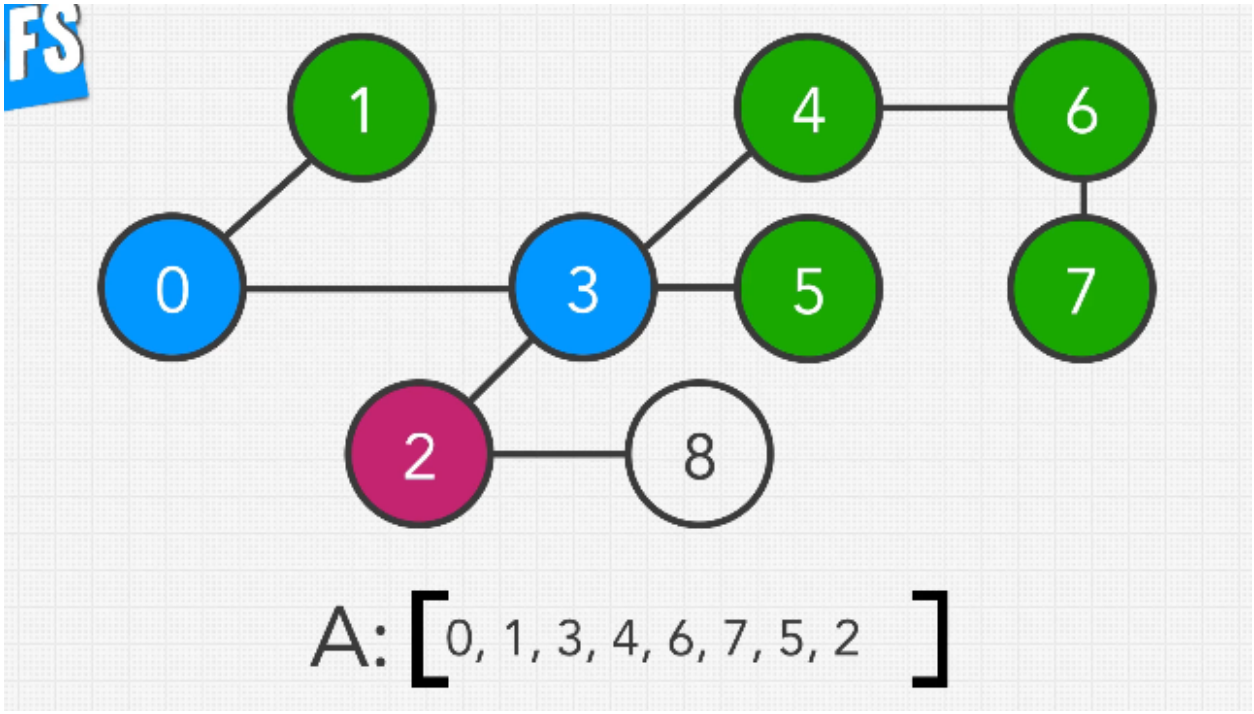


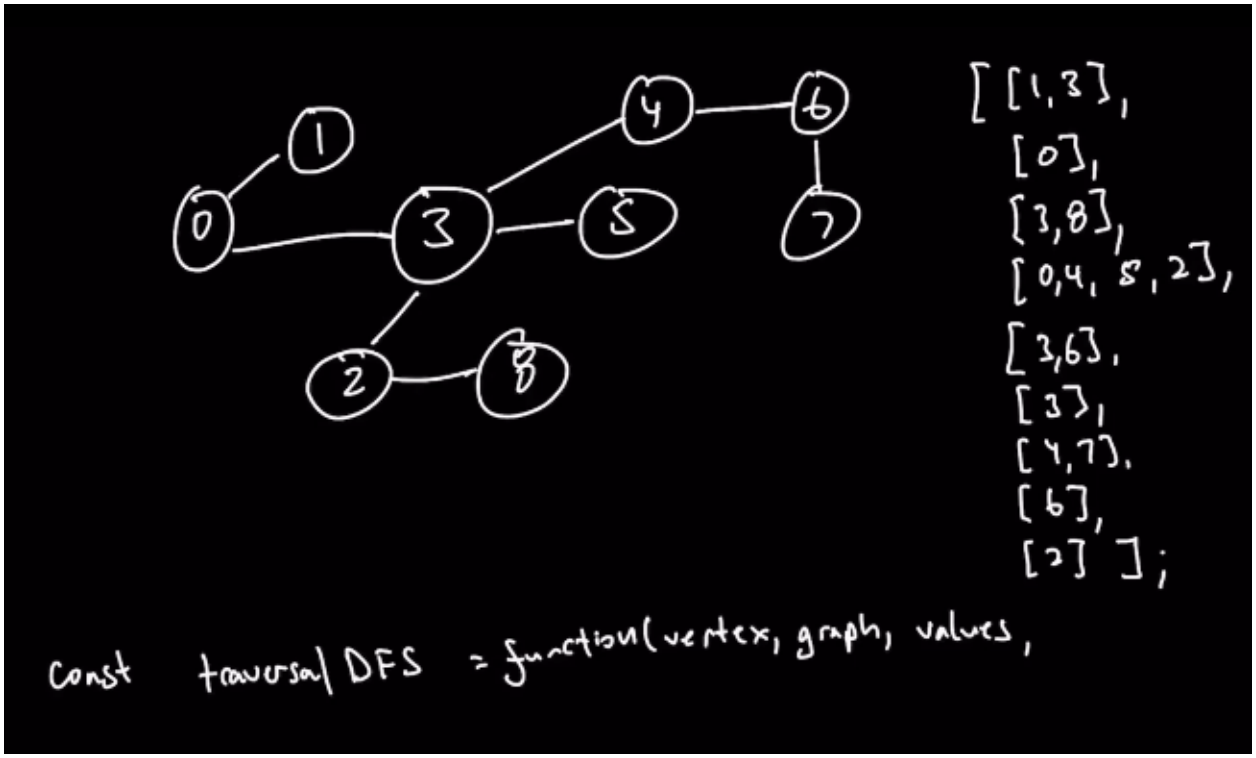
Graphs DFS

Depth first



It has to conserve a pattern

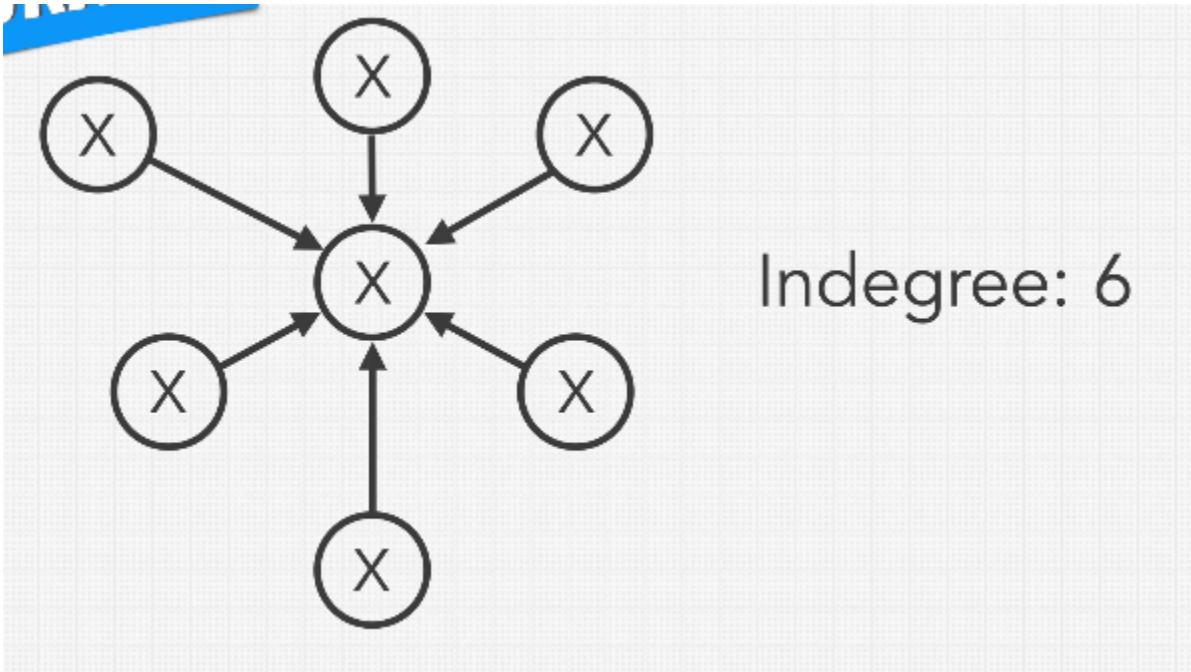





```
const traversalDFS = function(vertex, graph, values, seen){
  values.push(vertex);
  seen[vertex] = true;
  const connections = graph[vertex];
  for(let i = 0; i < connections.length; i++){
    const connection = connections[i];
    if(!seen[connection]){
      traversalDFS(connection, graph, values, seen);
    }
  }
}
```

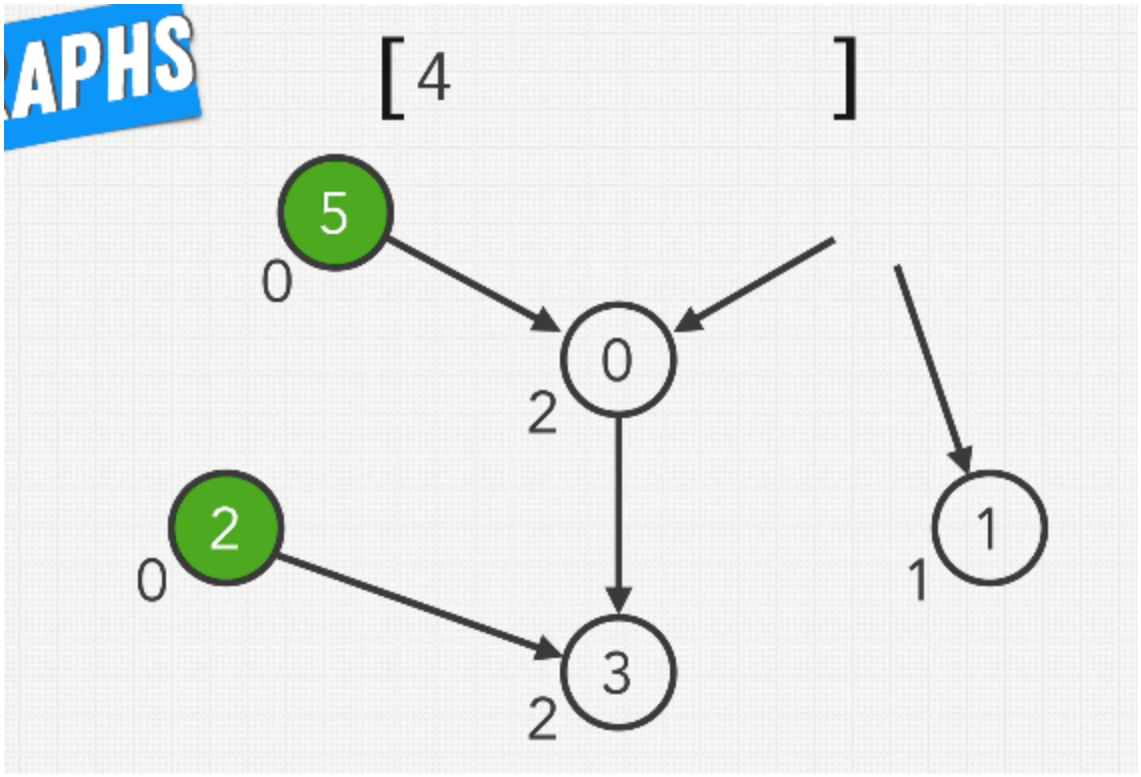
Graphs - Topological sort

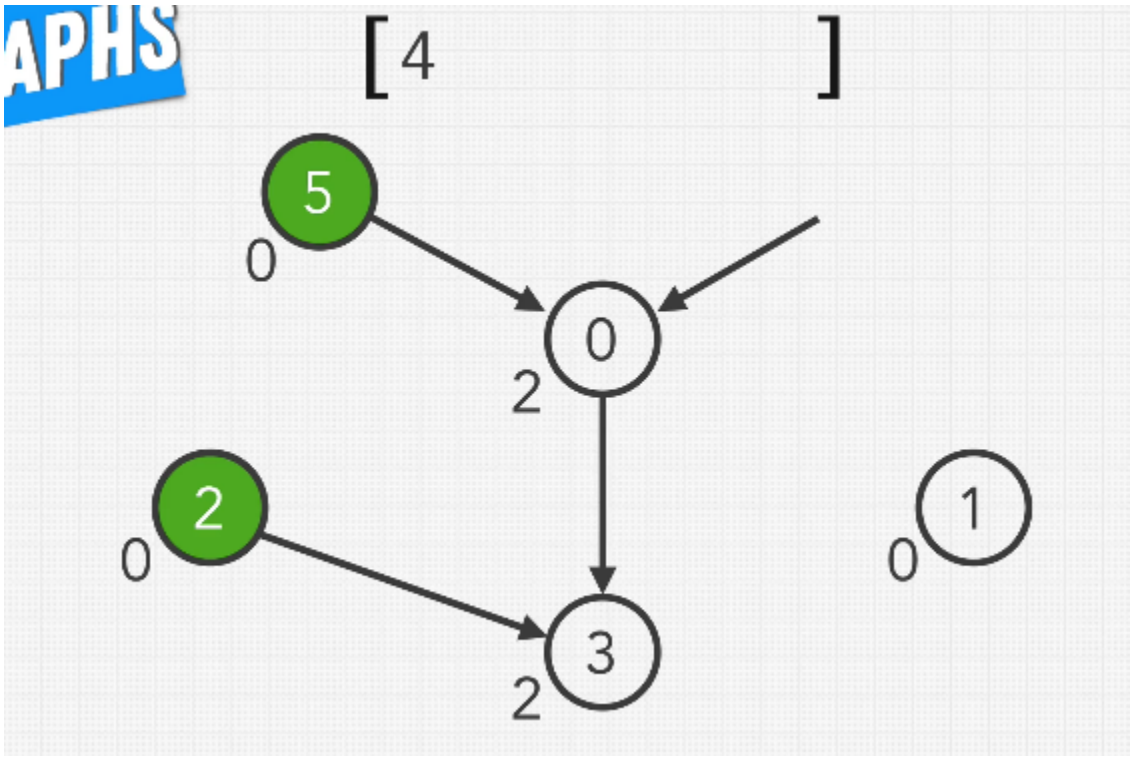
The connections determinants a indegree

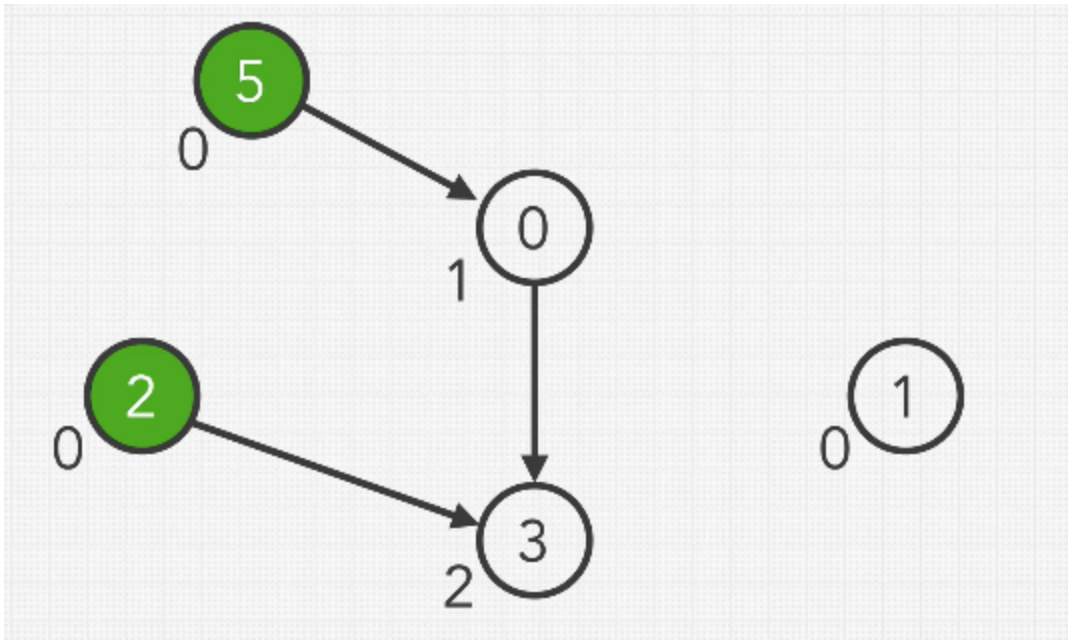


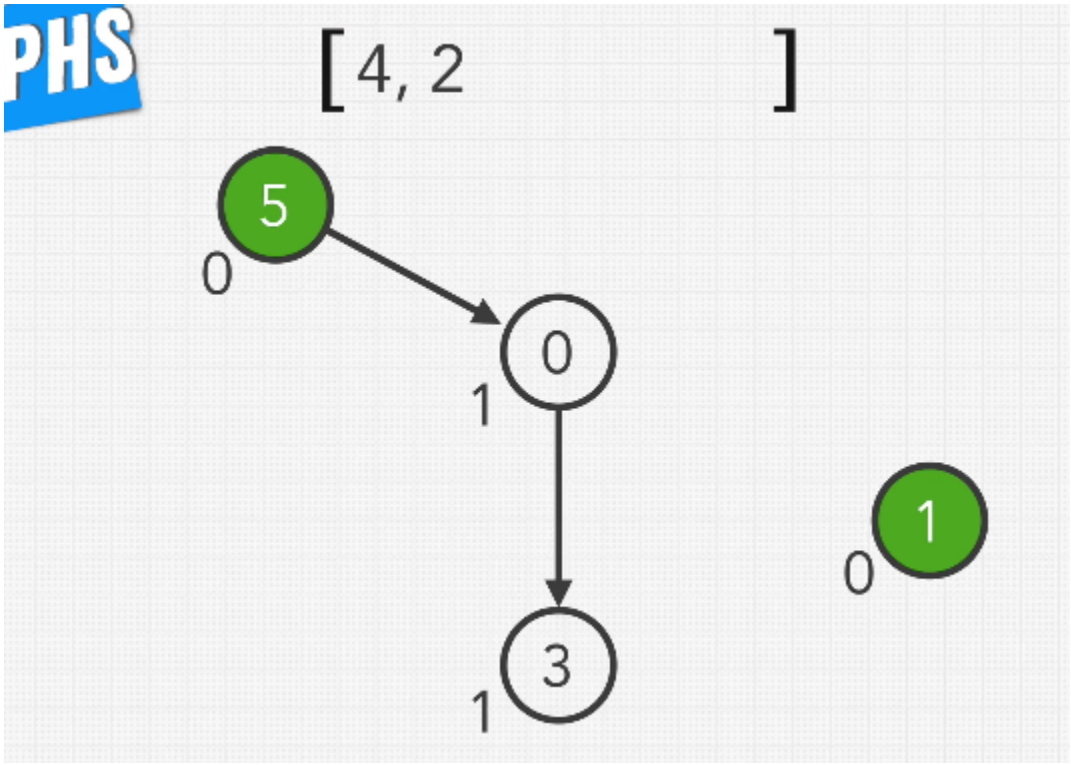
Topological sort is get into account this order

We can get the nodes with zero indegree, as long as i take that node i remove that


APHS
[4]
5
0
0
2
2
0
1
1
3
2

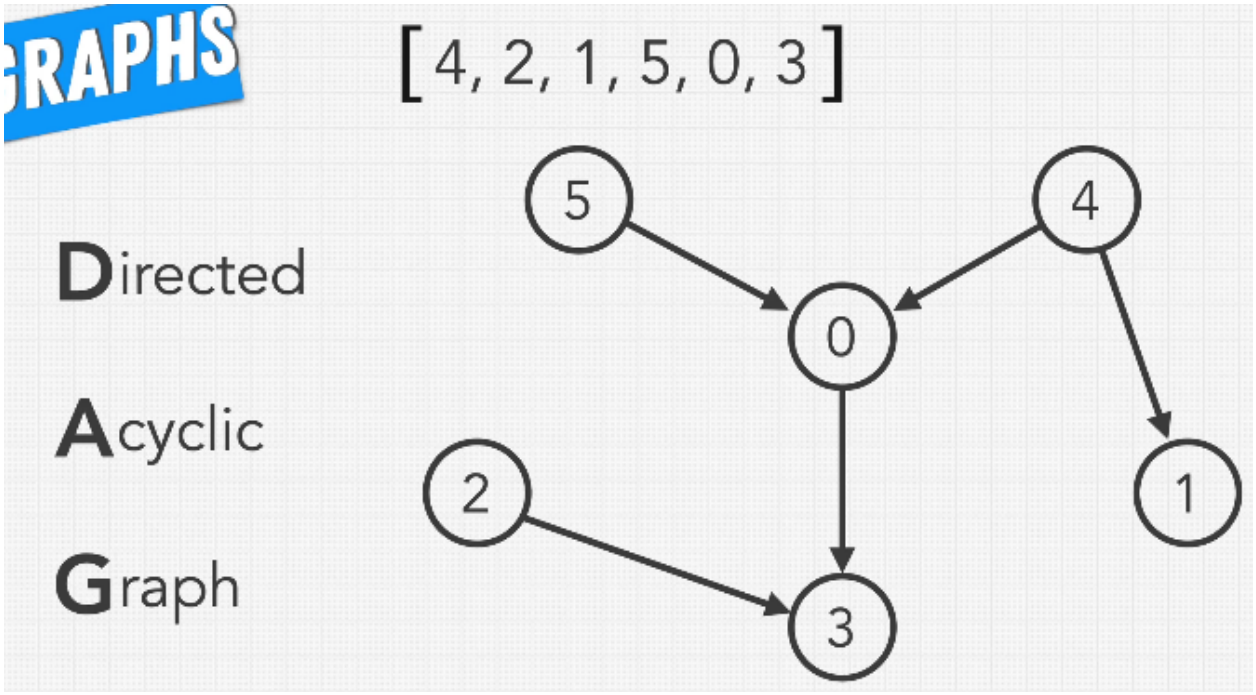

APHS
[4]
5
0
0
2
2
0
1
0
3
2

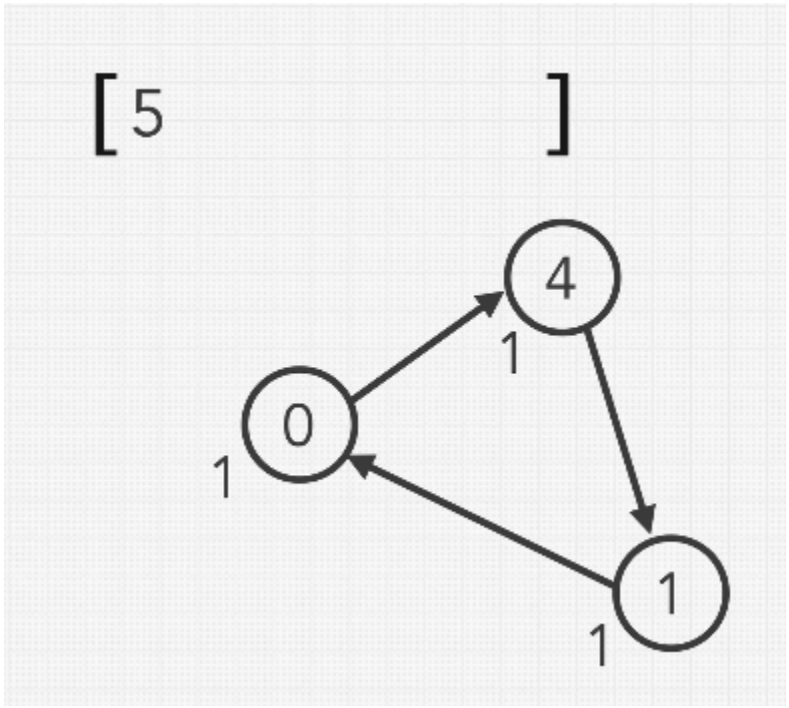

5
0
0
1
2
0
3
2
1
0


PHS
[4, 2]
5
0
0
1
1
0
3
1

[4, 2, 1, 5]

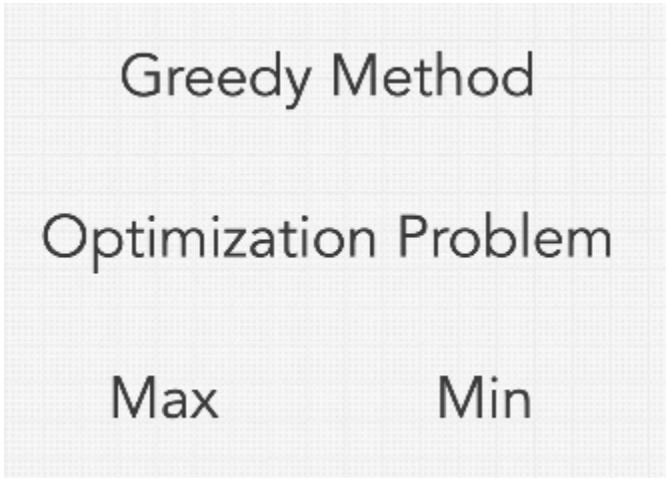
[4, 2, 1, 5, 0, 3]





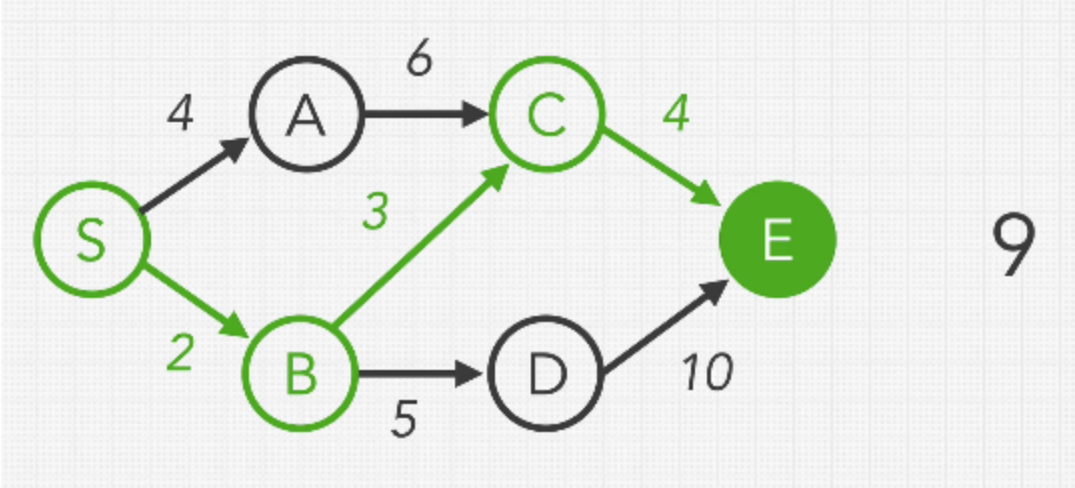
If we can’t take any of the remaining vertex, exist a cycle

Optimization - Greedy method

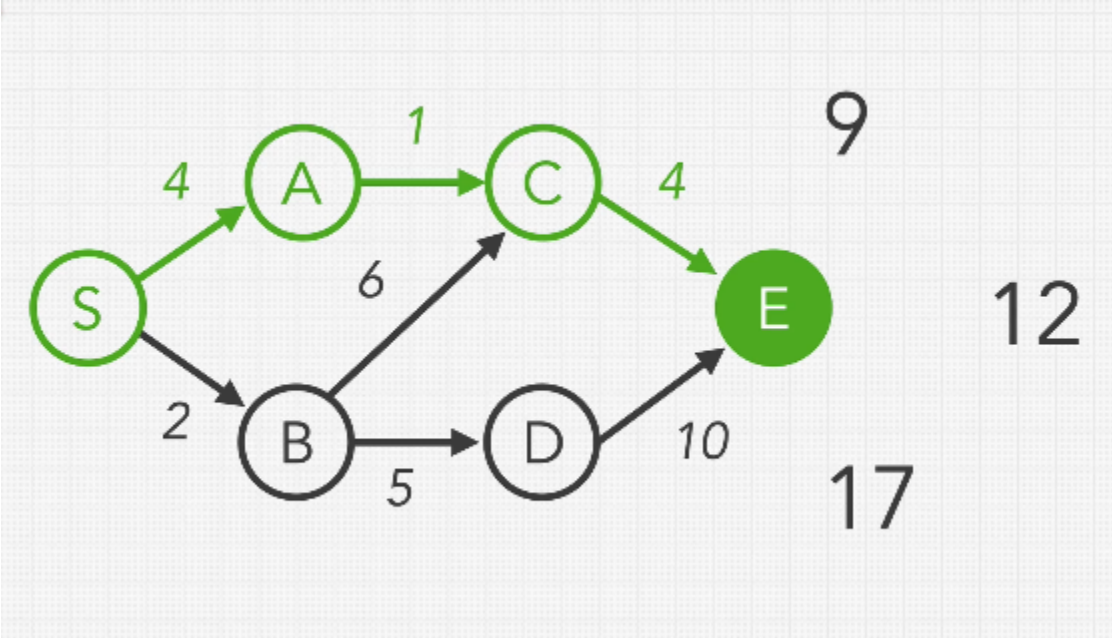


Greedy .- if i have to make a solution i pick a solution that made me the best option at that point

But have some problems

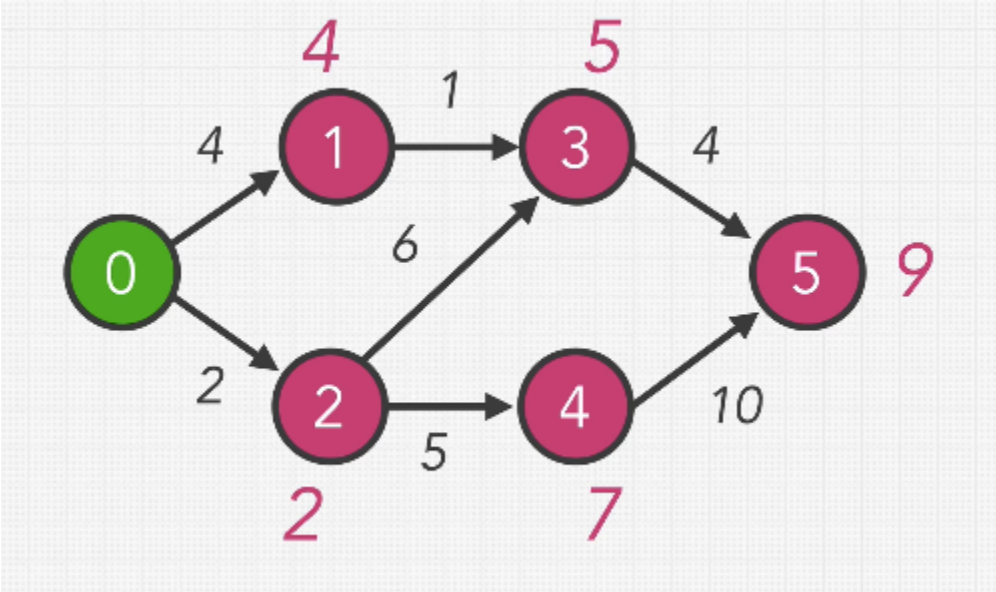


The problem:

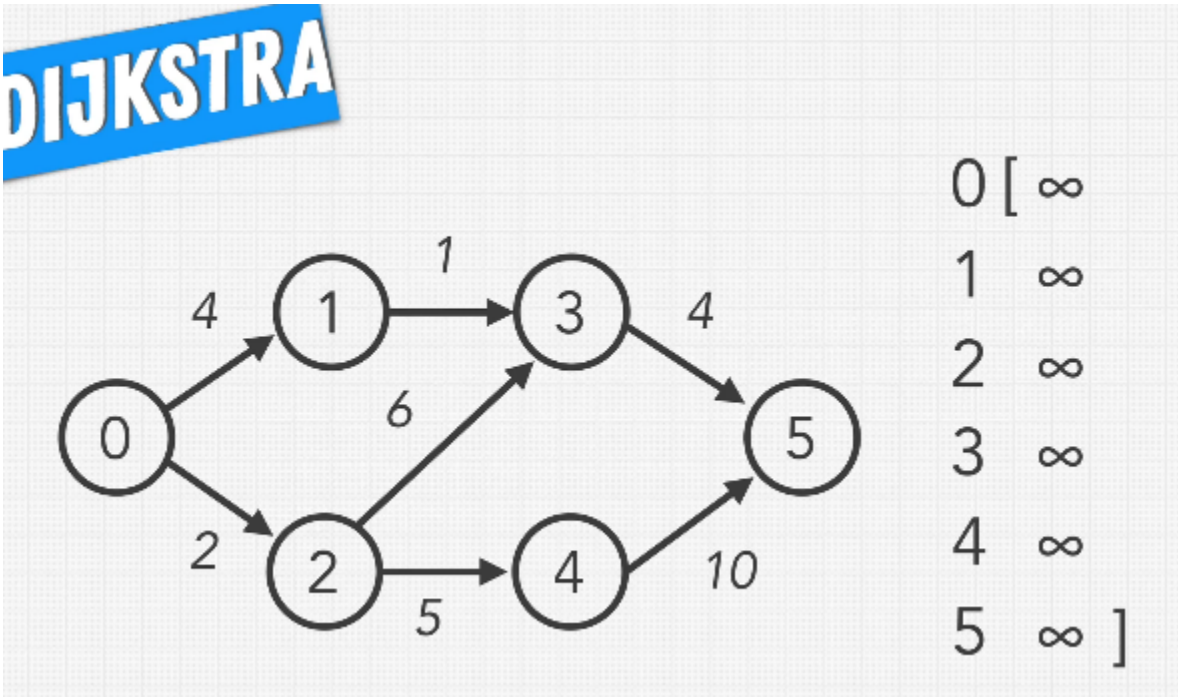


Sometimes picks the worst path

Optimization - Graph - Path - Dijkstra’s algorithm



We use some greedy



Measure the distance from a node to another node

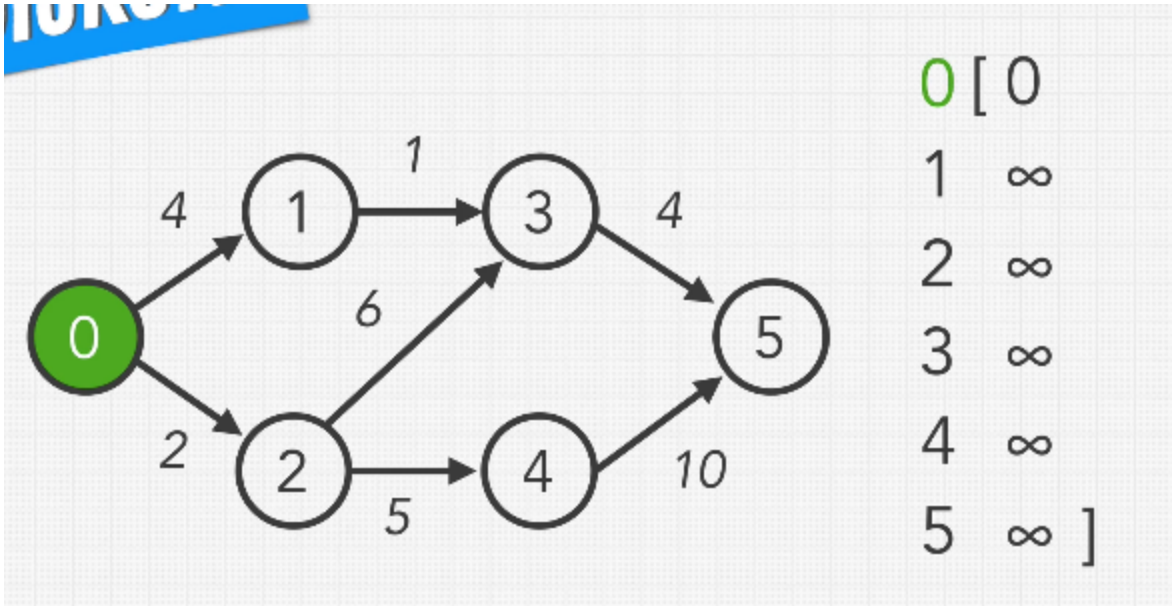
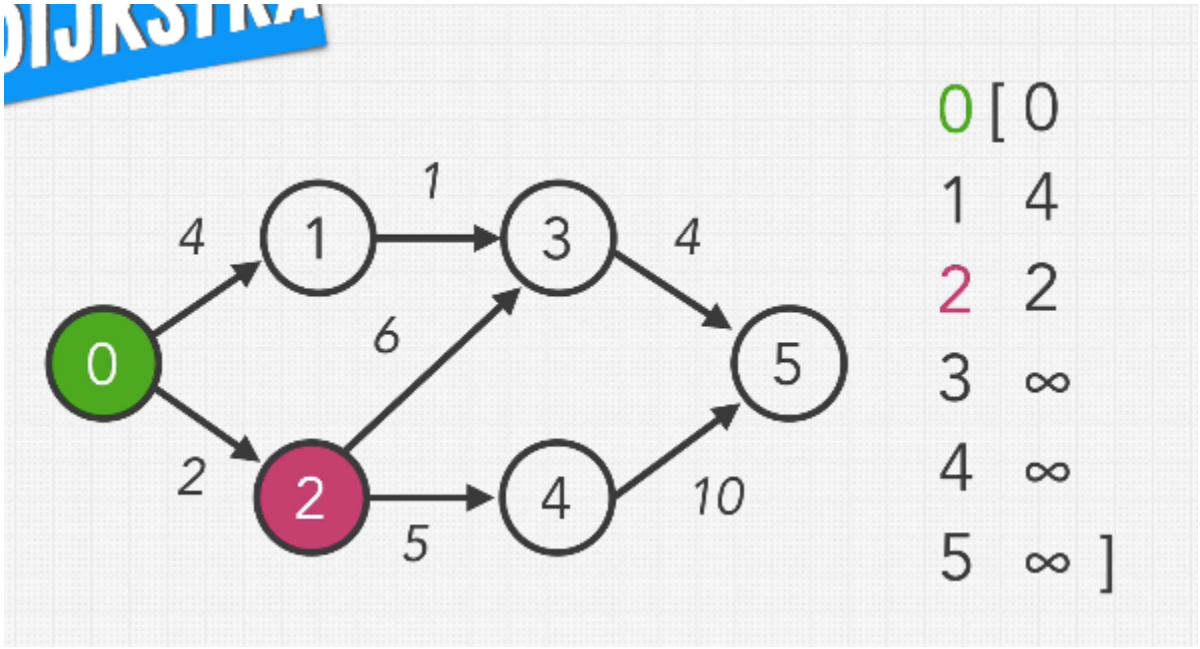


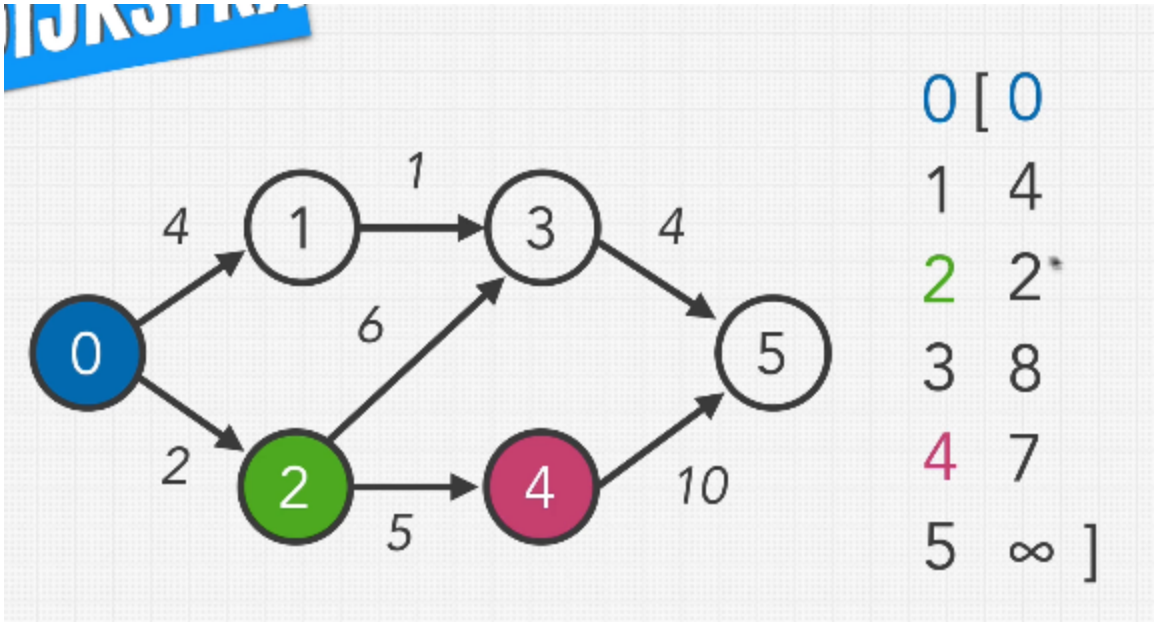
Figure out the distance from cero to other nodes



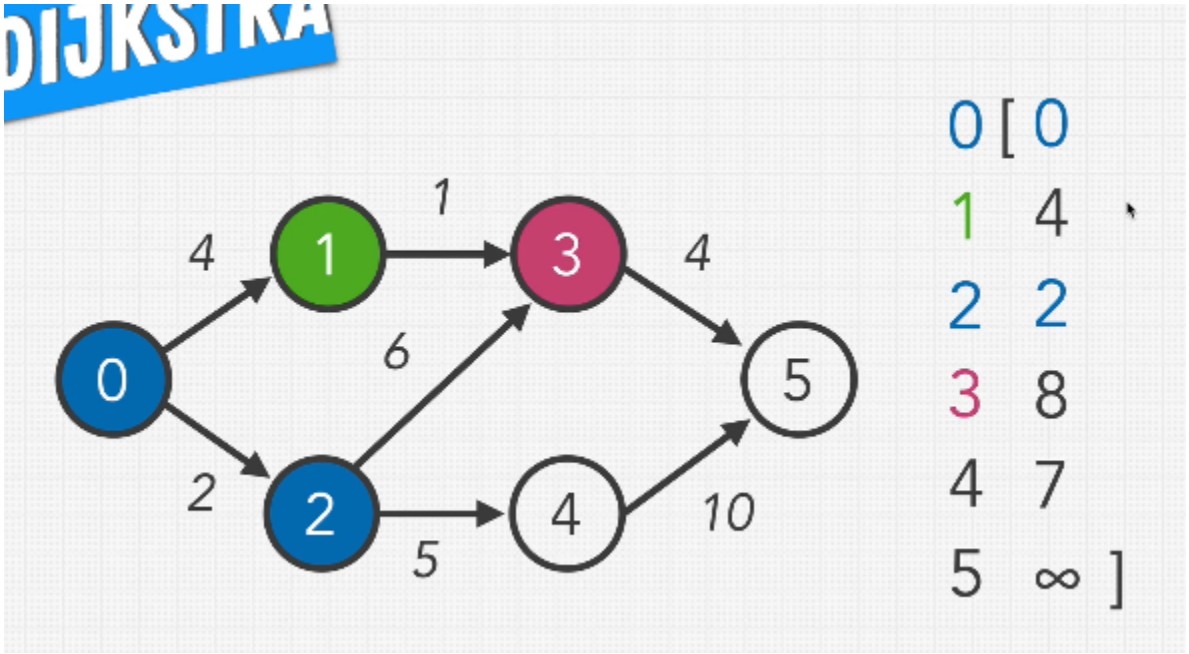
We never going back to zero

We use the 2 because of the smaller value (greedy)

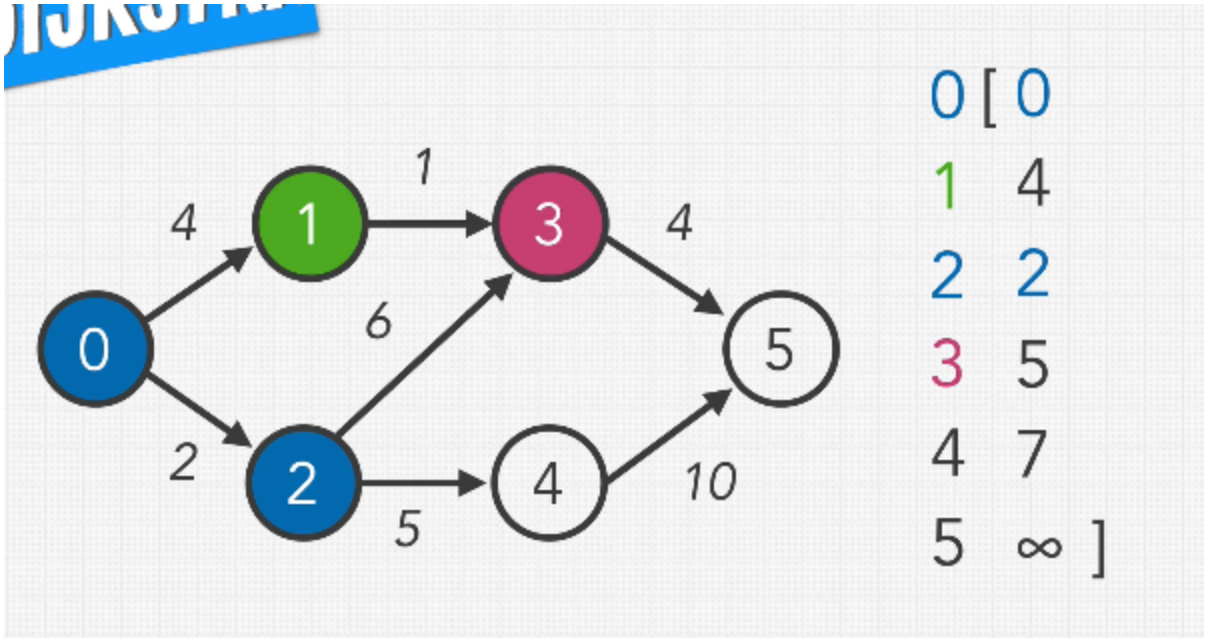
We add the current value

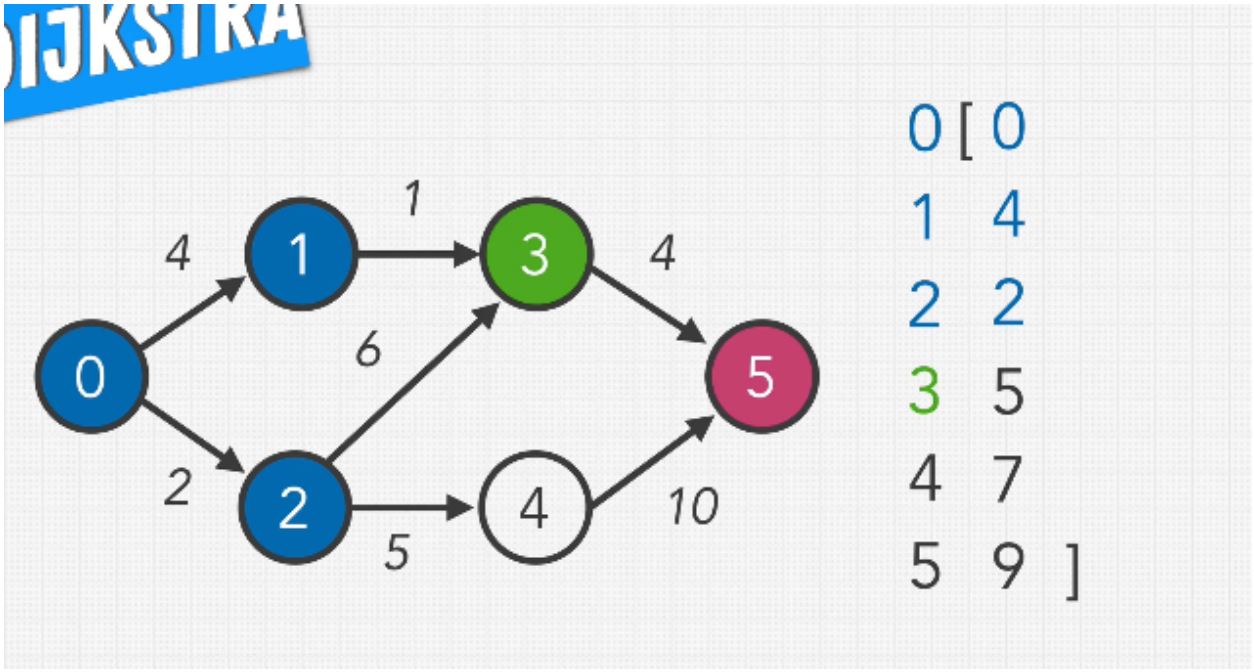


Then choose the smallest value not picked

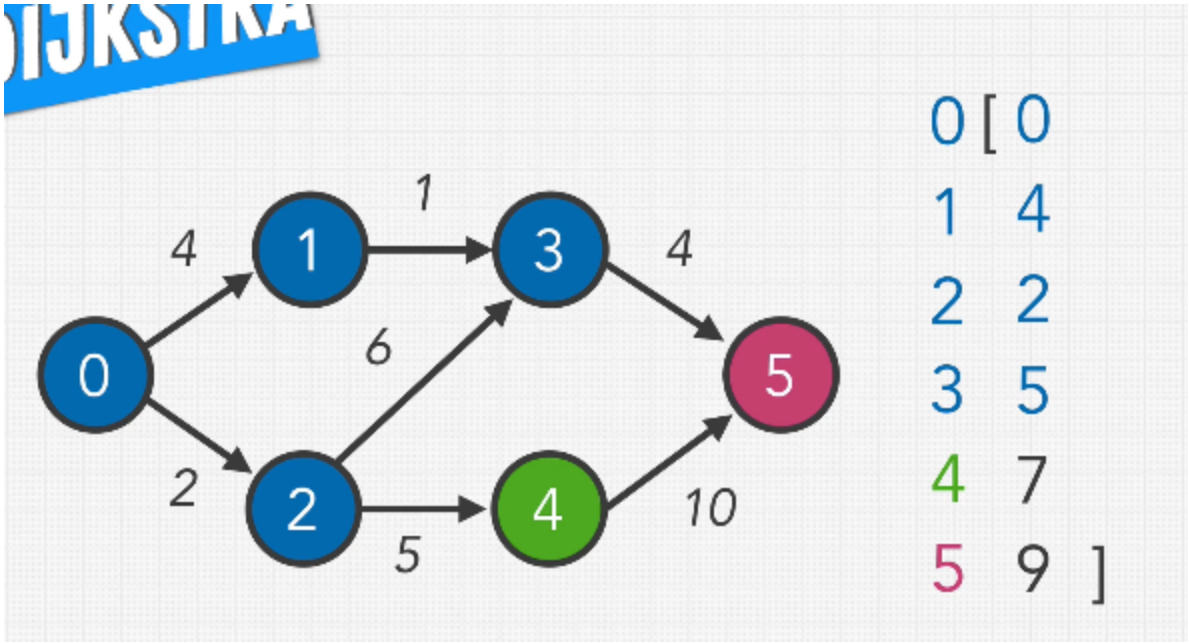


The actual value is less?
Yes

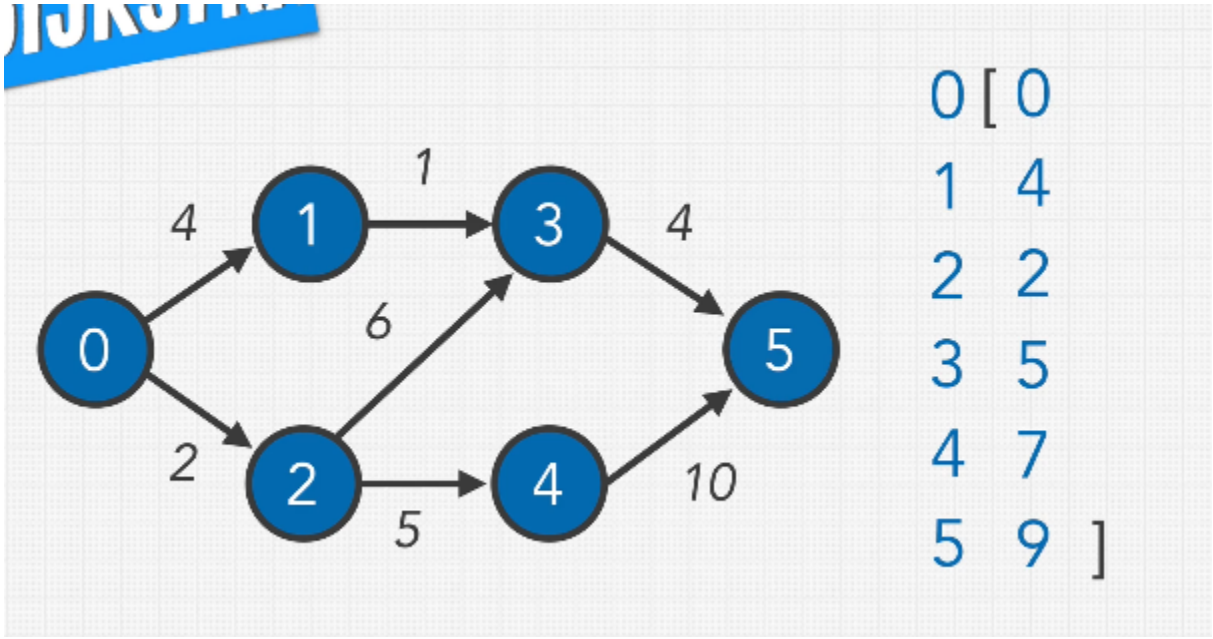




Then close three

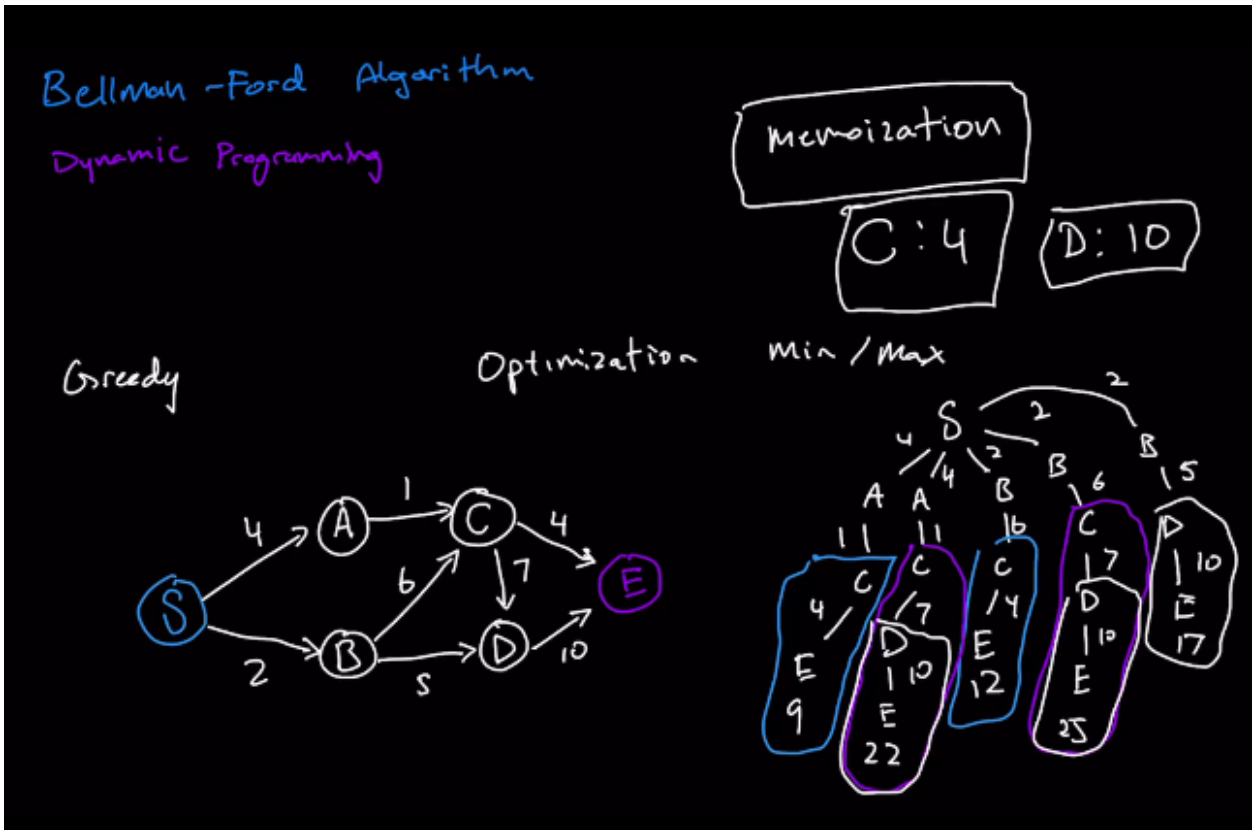


17 less than the current value at five? No
So is discard



Dijsktra can’t handle negative numbers

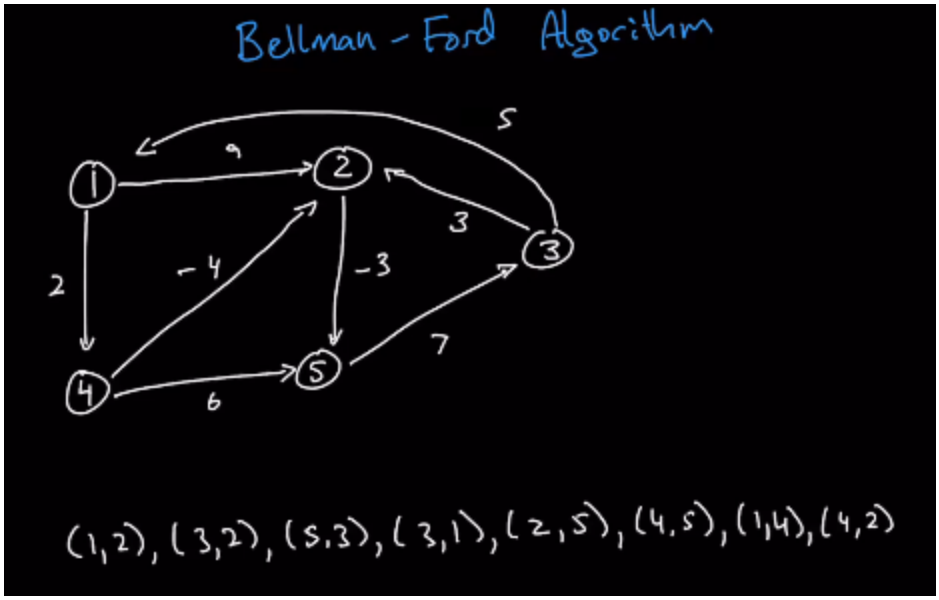
Optimization - Graph - Path- Bellman-Ford



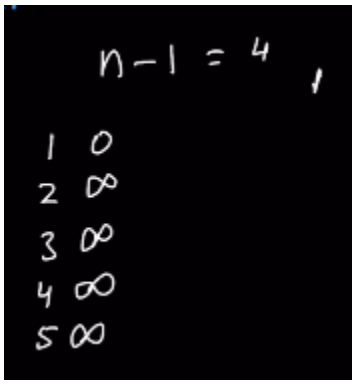
With this process we are repeating work

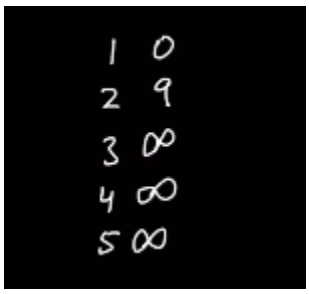
This is memoization

We reduce the work



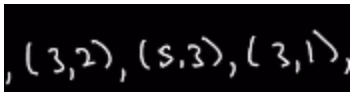
We keep track of lowest cost



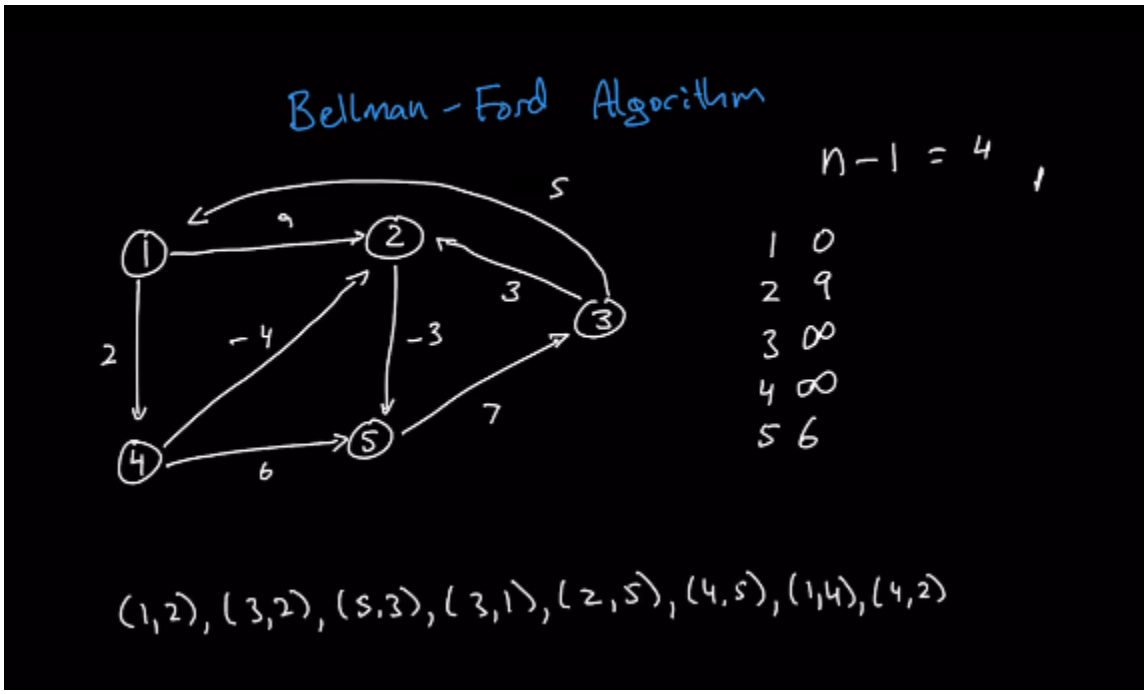


(3, 2), infinity + 9 is infinity so we ignore it

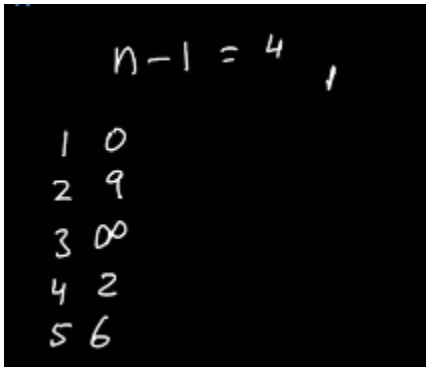
Also skip



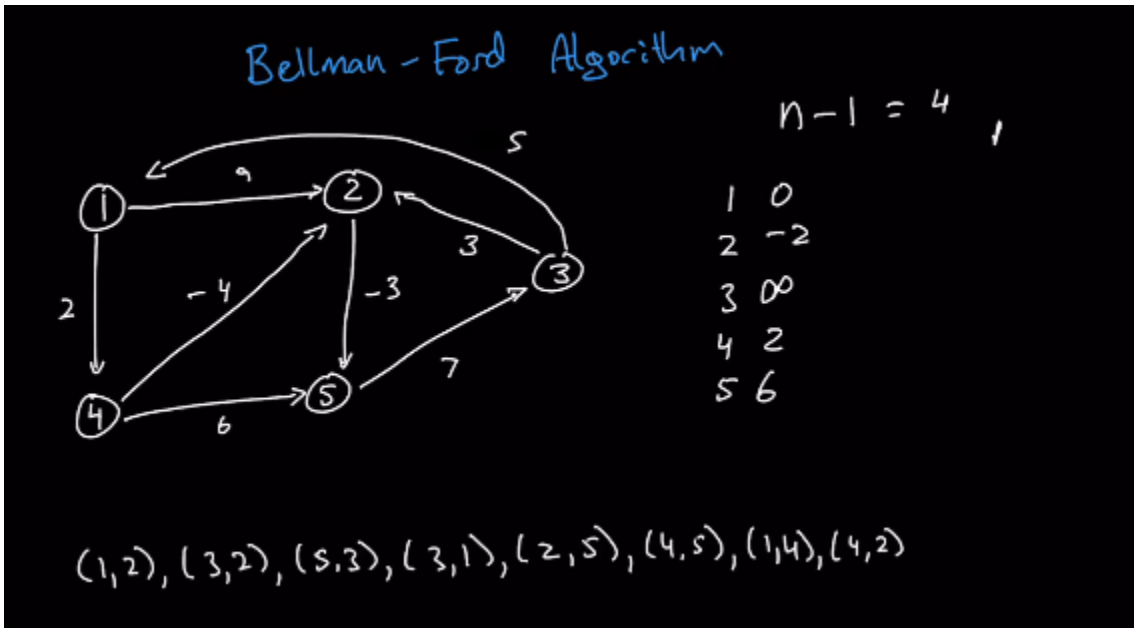
With (2, 5) we can make calculations



(1, 4)

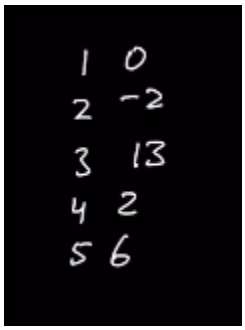


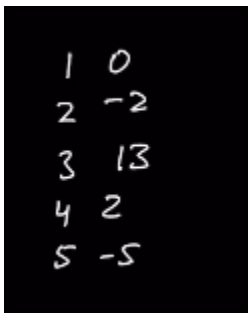
To (4,2)



We iterate again

(5,3)
Results in

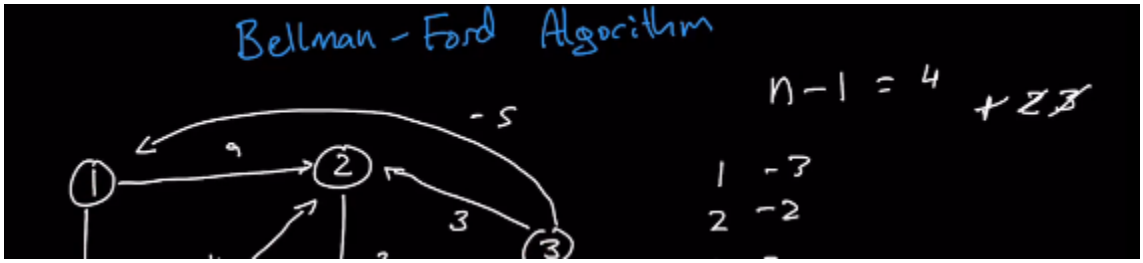




We should do n-1 iterations even if we already have the shortests values

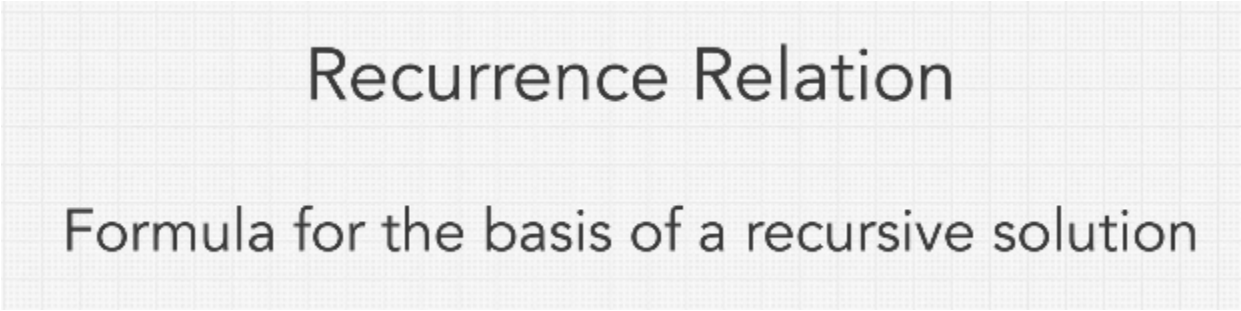
If we update a value we should iterate

This doesn't works in negative cycle



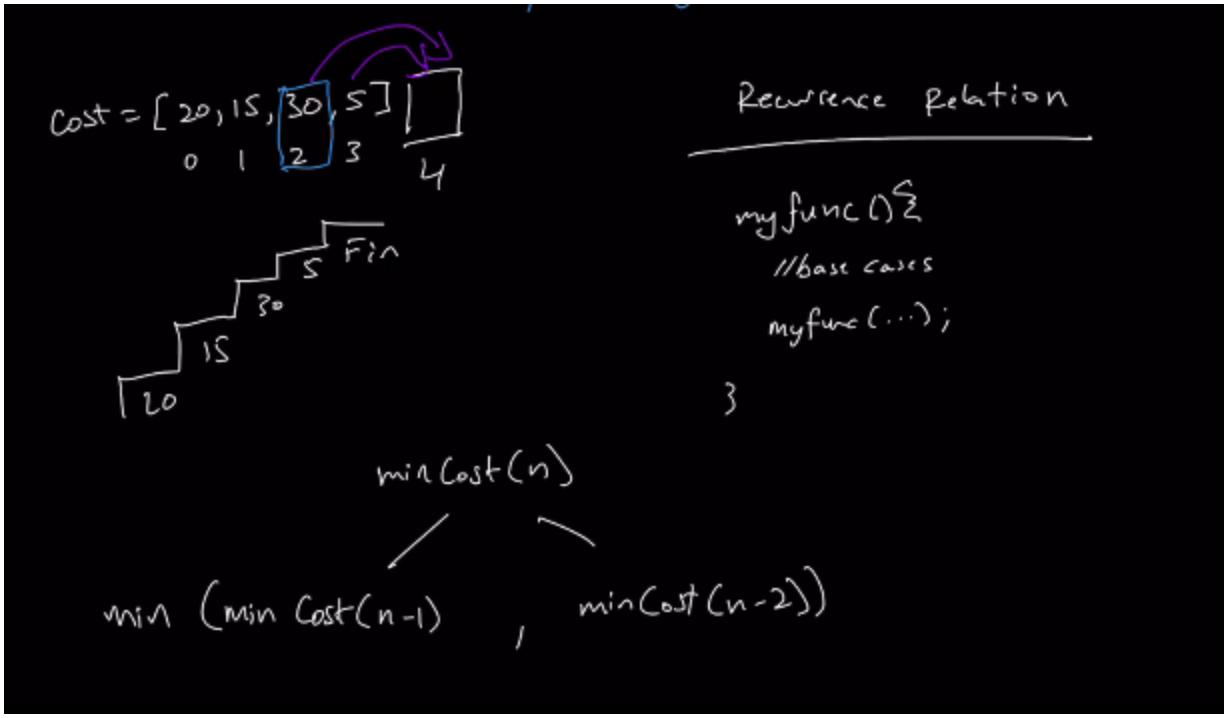
If doesn't exist negative weights, we should use dijkstra

Recurrence relation

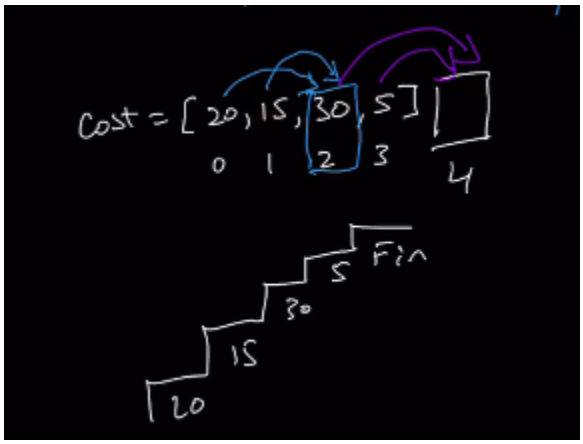


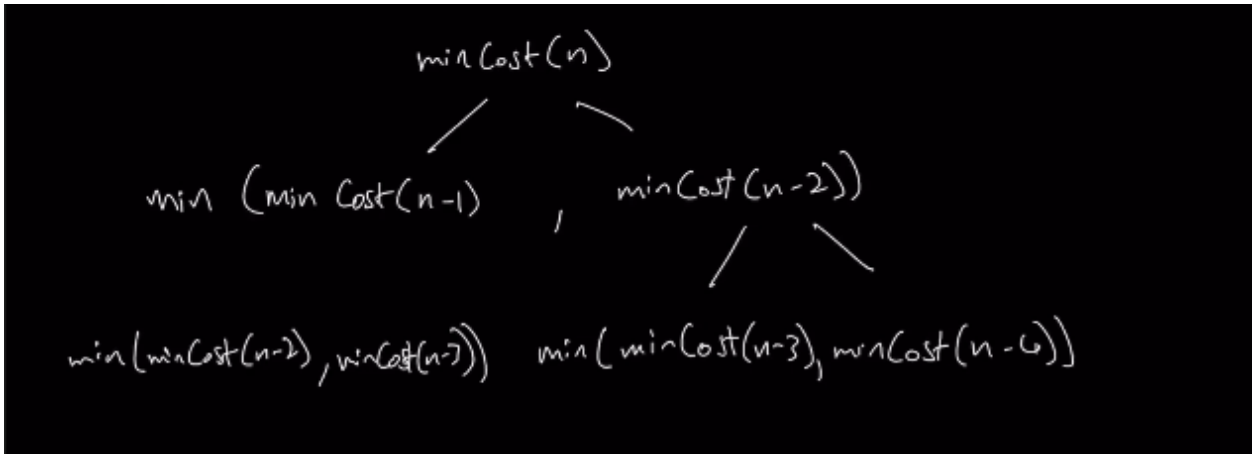
Can I redefine this problem in a repetitive way?

We can jump one from 5 or two from 30 so

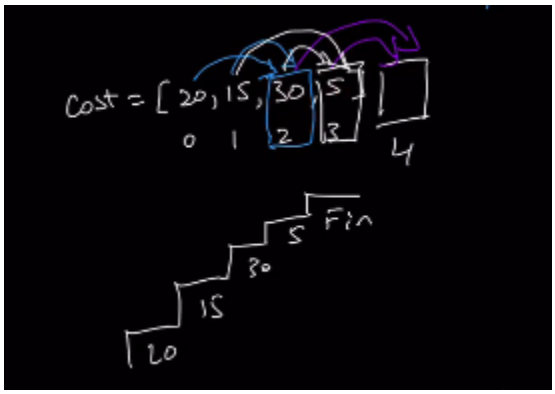


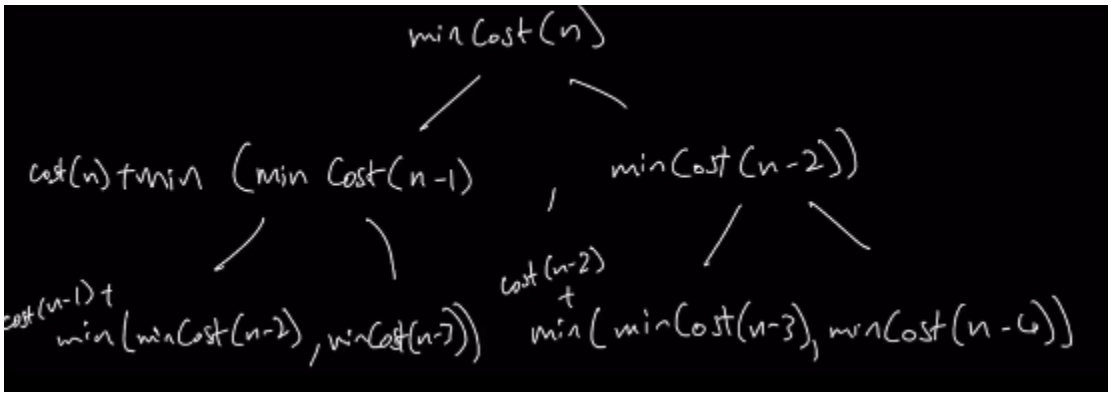
And what is the minimum cost for 30?





We can go back



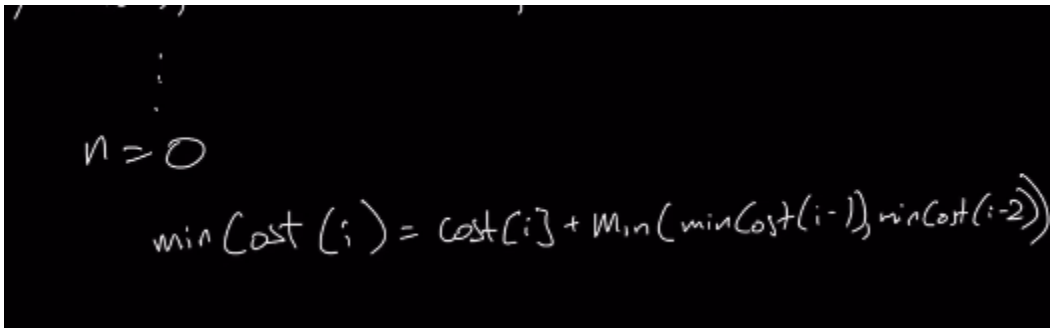


This is build solution in repetitive way

What we get back?

Then we can derive the formula

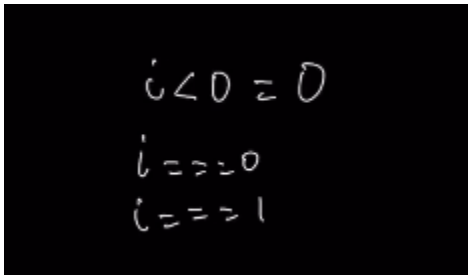
And get the base case



This is the recursion formula

We don’t care about implementation

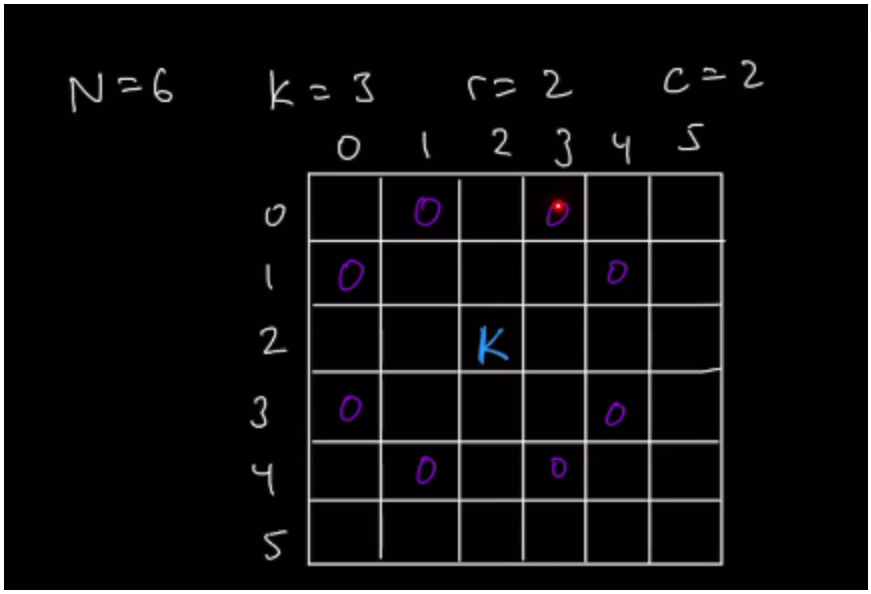
We can derivate direct solutions

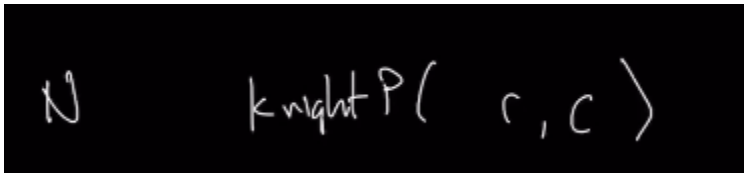


Example 2:

Reduce the problem to smaller but repetitive

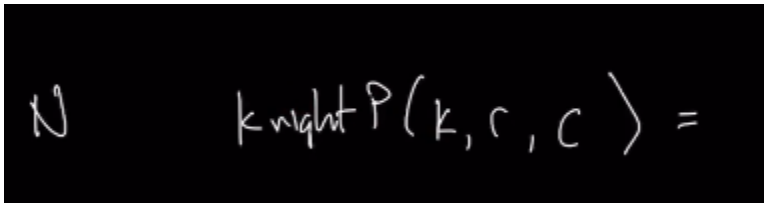
In the chessboard example the movement depend on the current (r,c) value



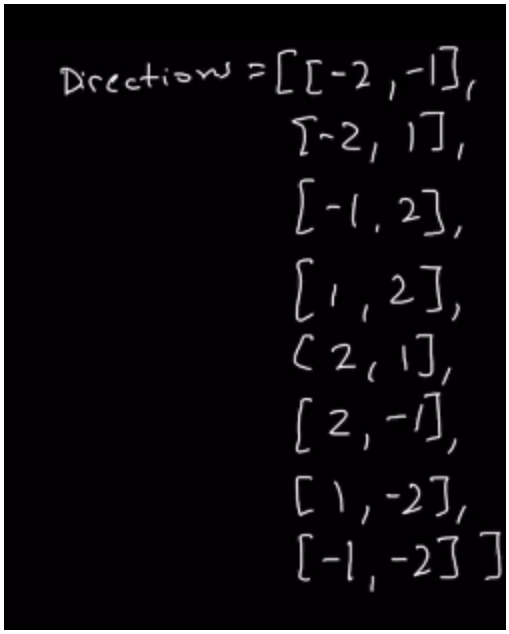


So r and c are definitely parameters

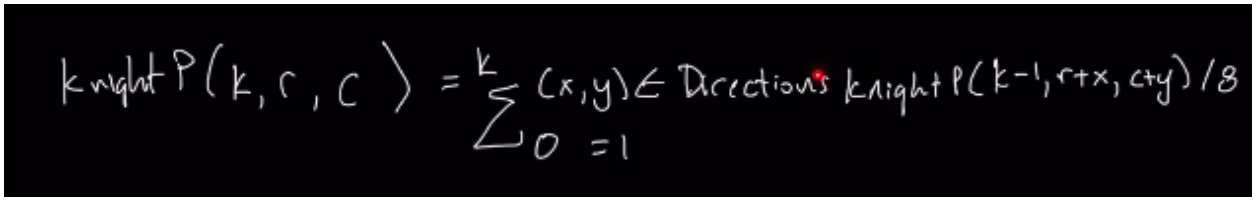
K also represent something in the steps



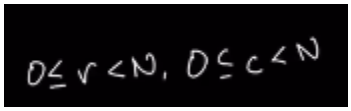
We can represent de directions



The number of recursion calls are

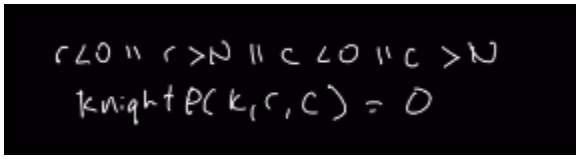


For the eight directions



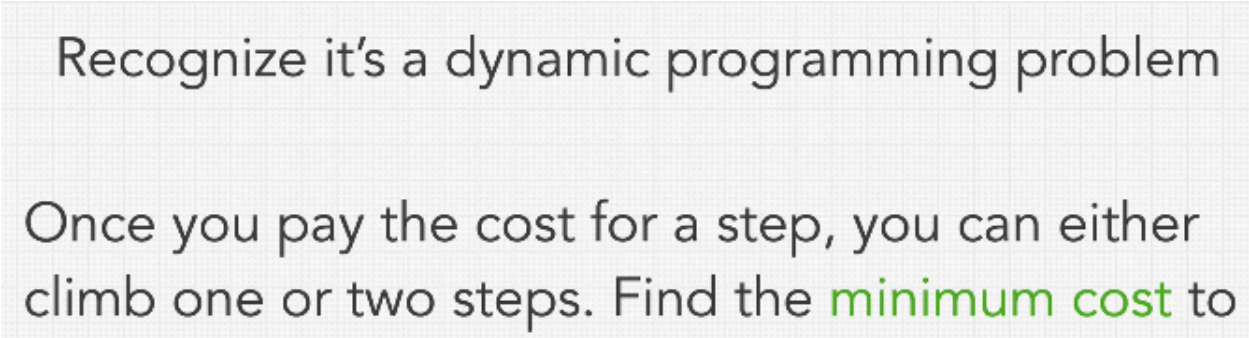
Only if we are inside the board

If we are out side



Dynamic programming

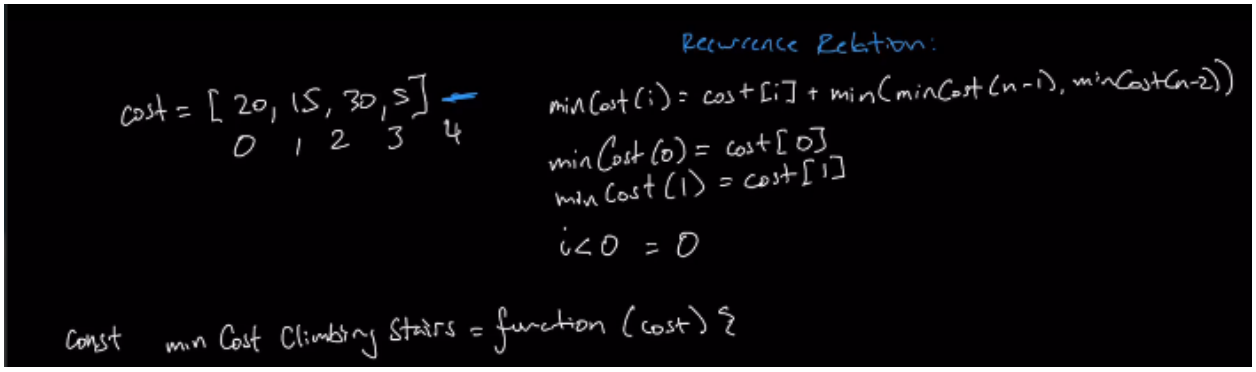
Recognize a dynamic programming (minimum and maximum) for optimization DP can be good option



DP is based on recursion

First we want to solve the problem and we can use dynamic programming.

1. Get recurrence relation

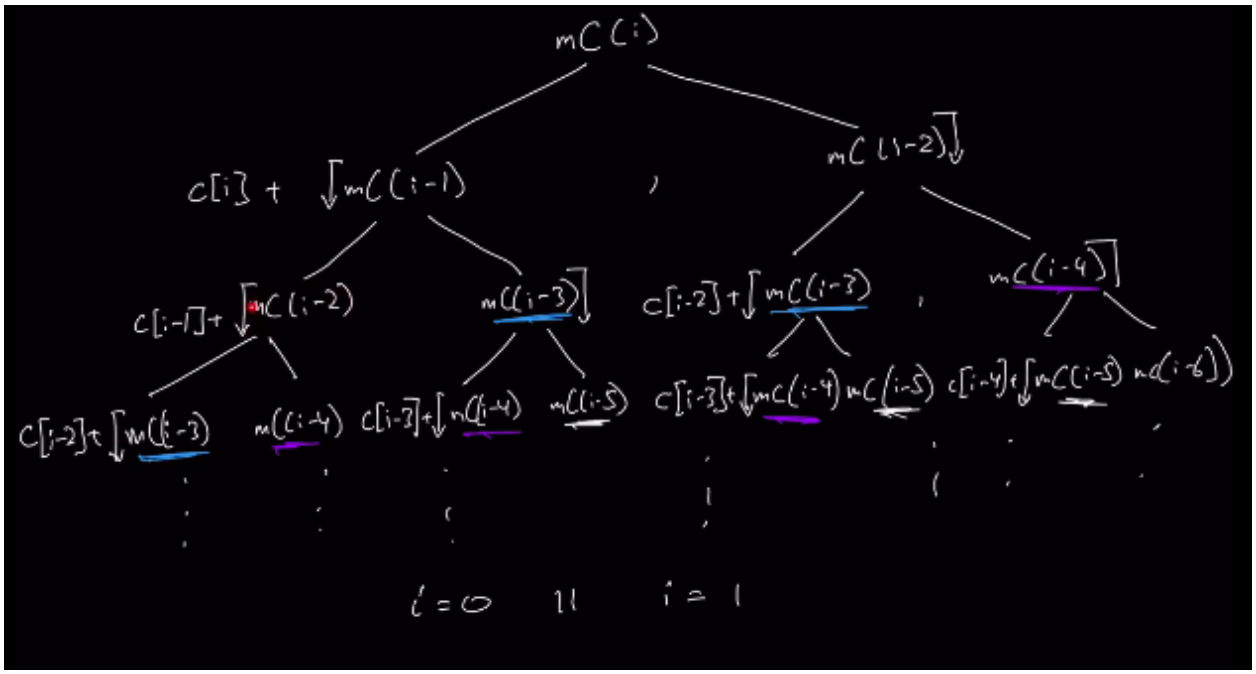


2. Implement recursive function

```
const minCostClimbingStairs = function (cost) {
    const n = cost.length;
    return Math.min(minCost(n-1, cost), minCost(n-2, cost));
};

const minCost = function(i, cost) {
    if (i<0) return 0;
    if (i===0 || i===1) return cost[i];
    return cost[i] + Math.min(minCost(i-1, cost), minCost(i-2, cost));
};
```

T: 2^n
S: 2^n

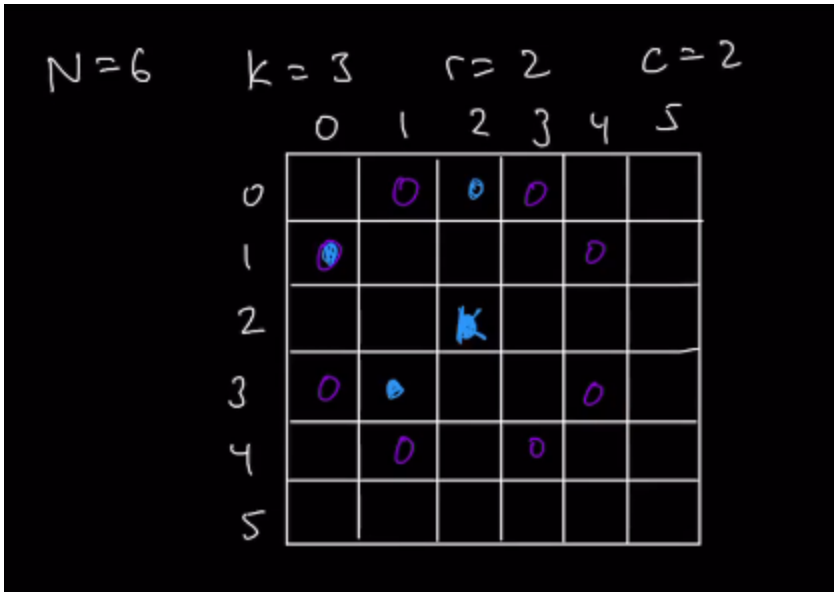
3. Optimice with memoization



Sometimes optimization is not only about min and max.

“This optimise the process of generating all possible movements”

For instance generate all possible solutions

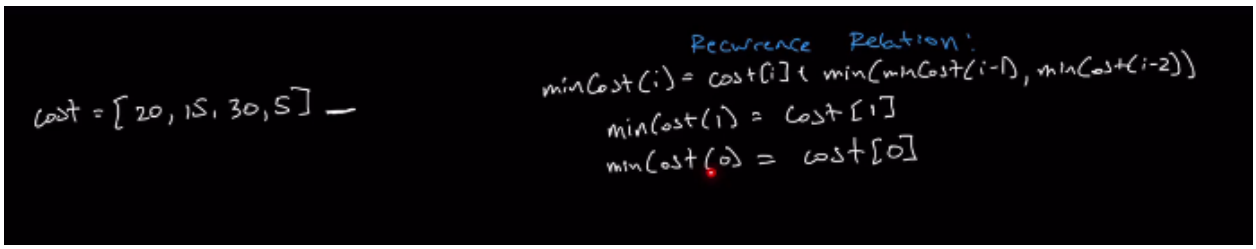


If exist an overlapping path, DP can be helpful

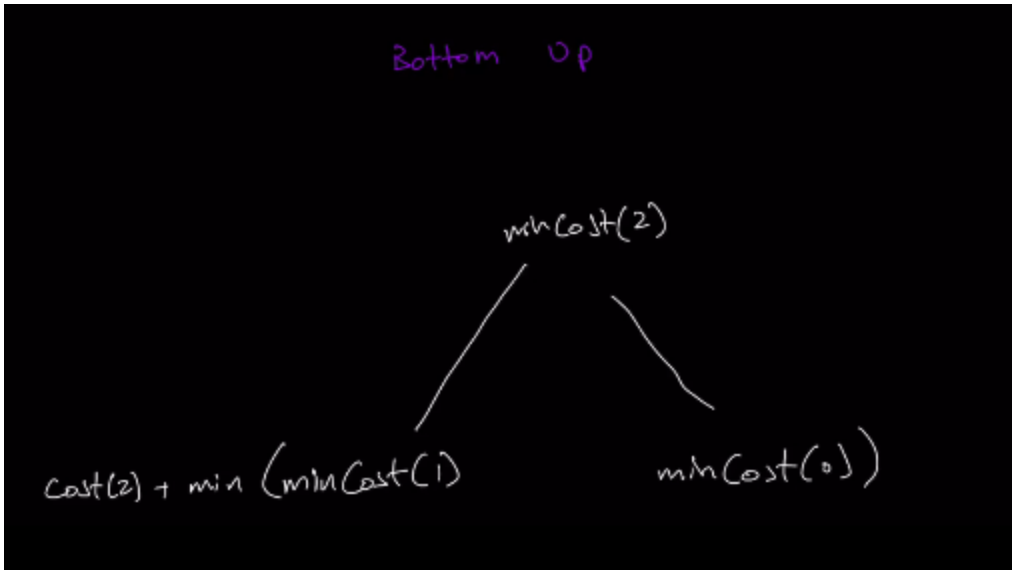
Tabulation - Bottom up

This is convert a recursive solution in an iterative solution

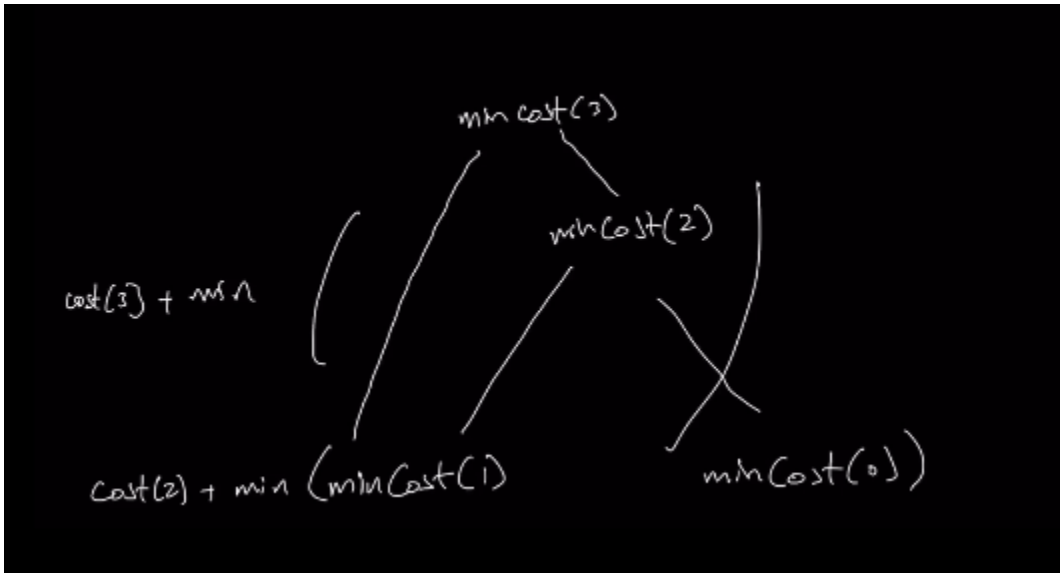
Eliminate the top down approach

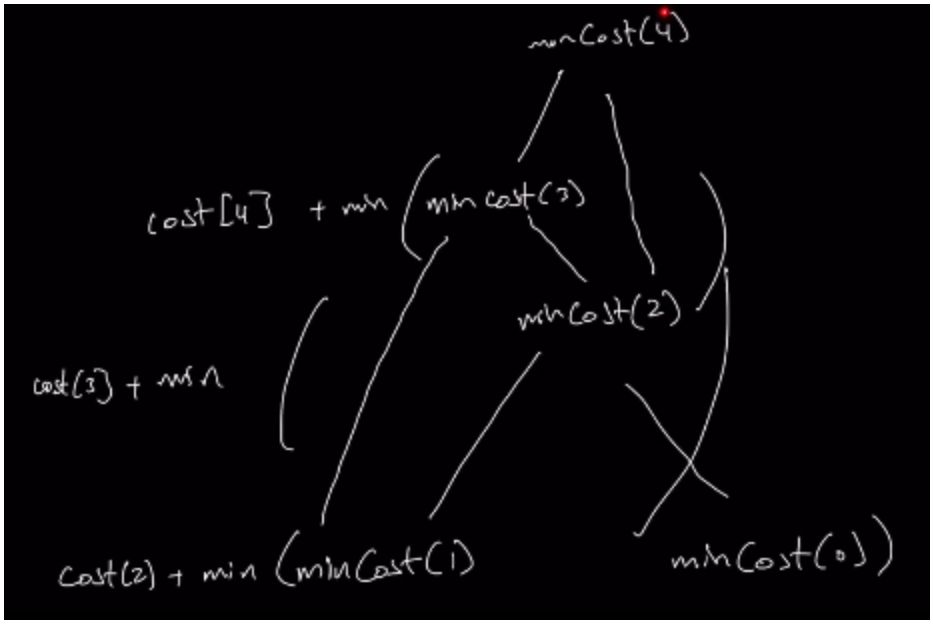


We start from the base cases

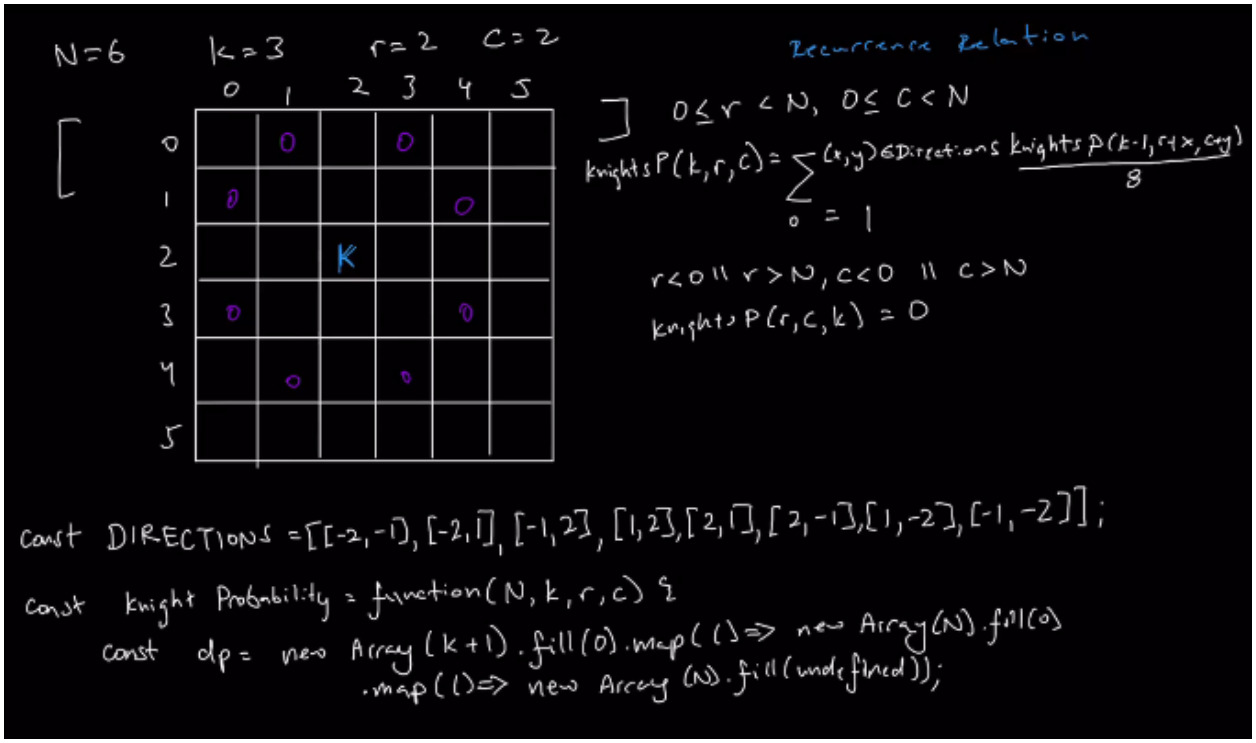


Then build the next solution





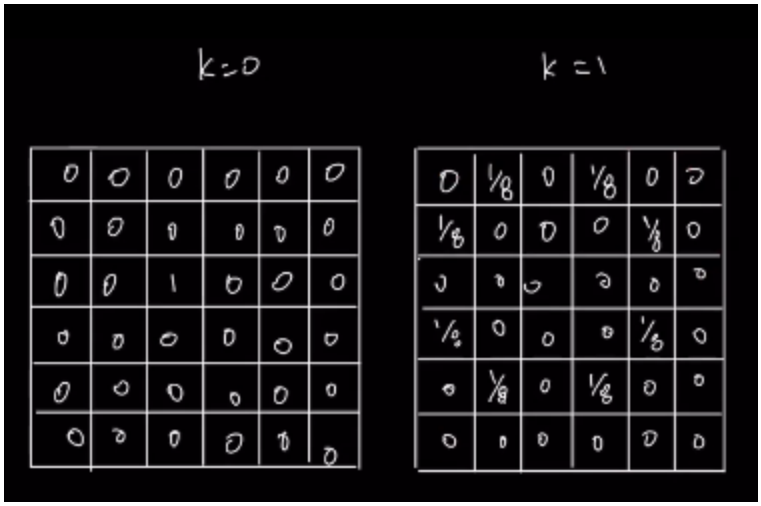
Example 2:

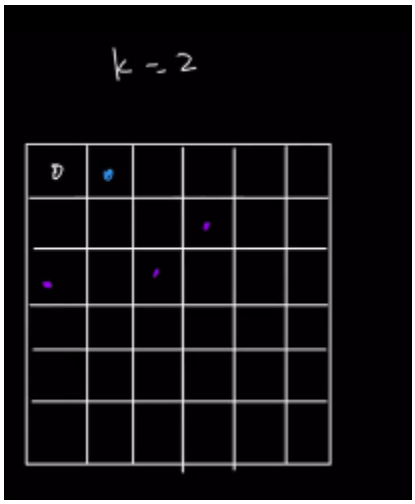


Here exist a depth of K

We generate all i need for this level of K, and fill up

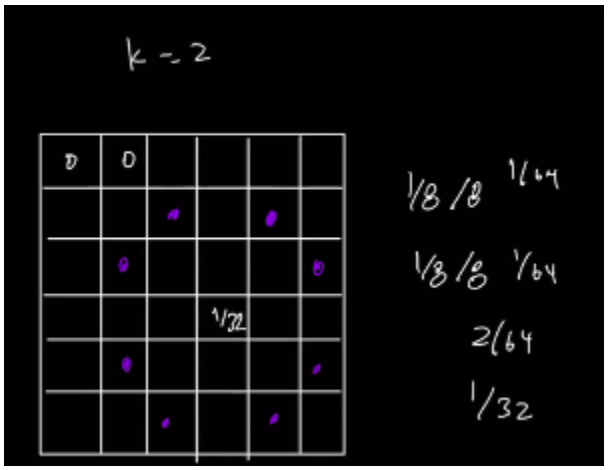
The problem want the probability so for 8 movements we fill with





We check for each cell what is the probability of reach that part

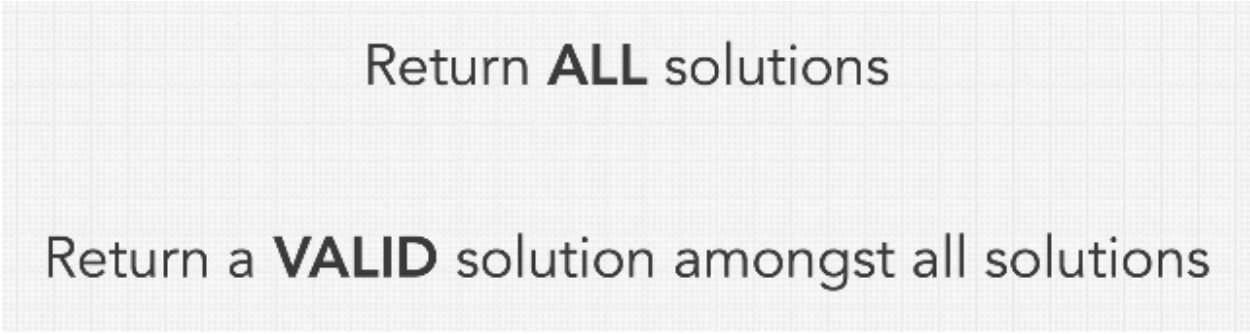
We divide k times the probability and sum



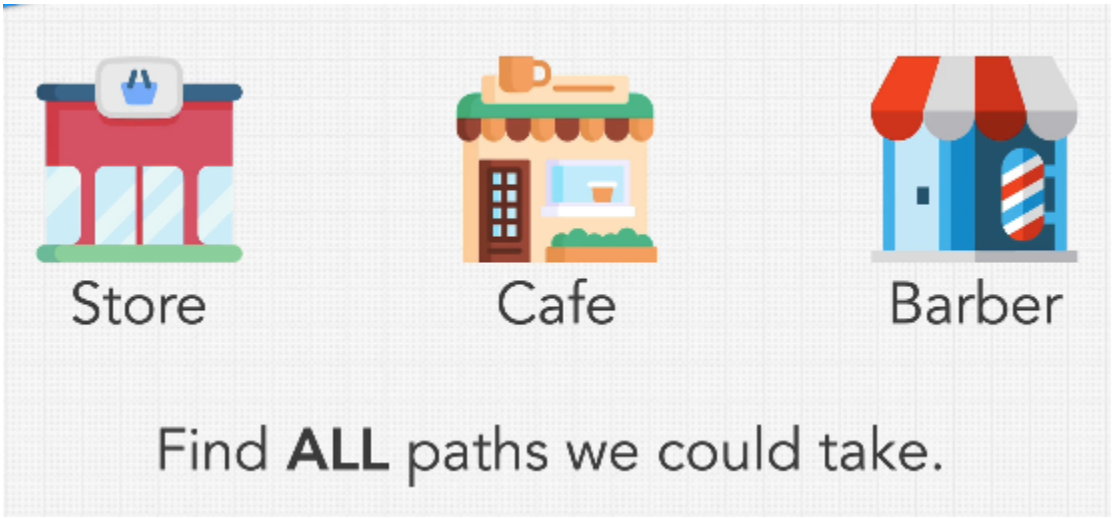
We try to drive the solution into table

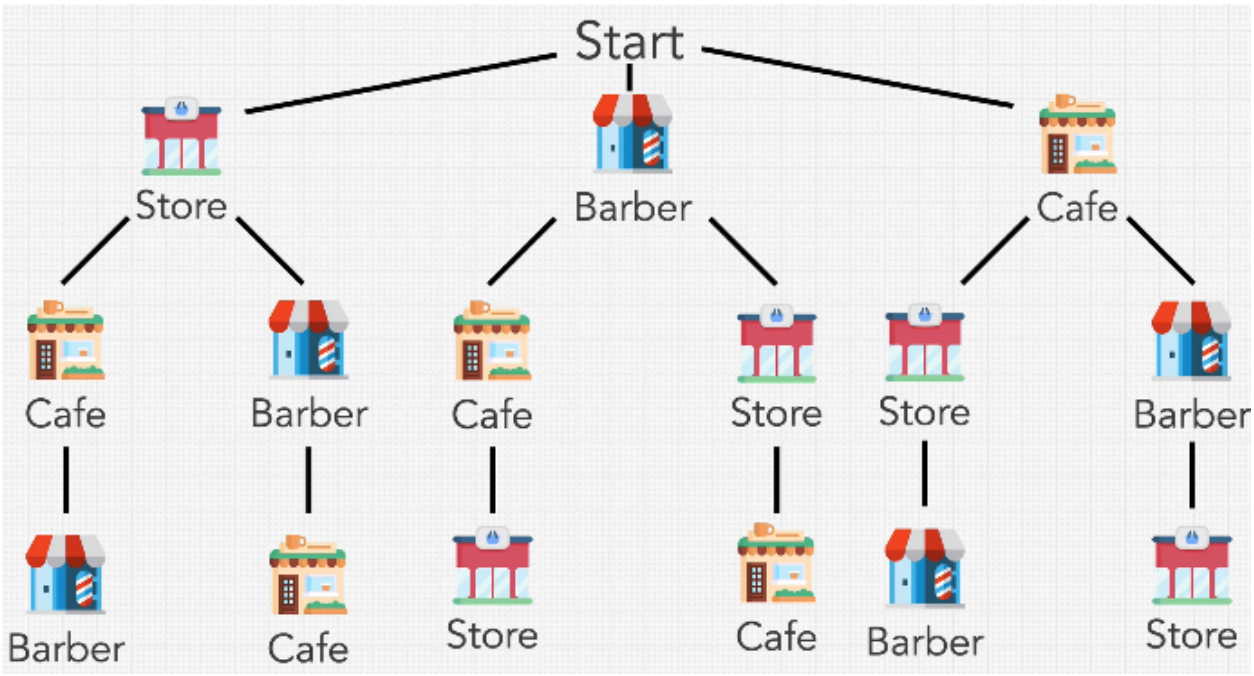
Backtracking

It’s like dynamic programming but we can recognize a problem of backtracking if:

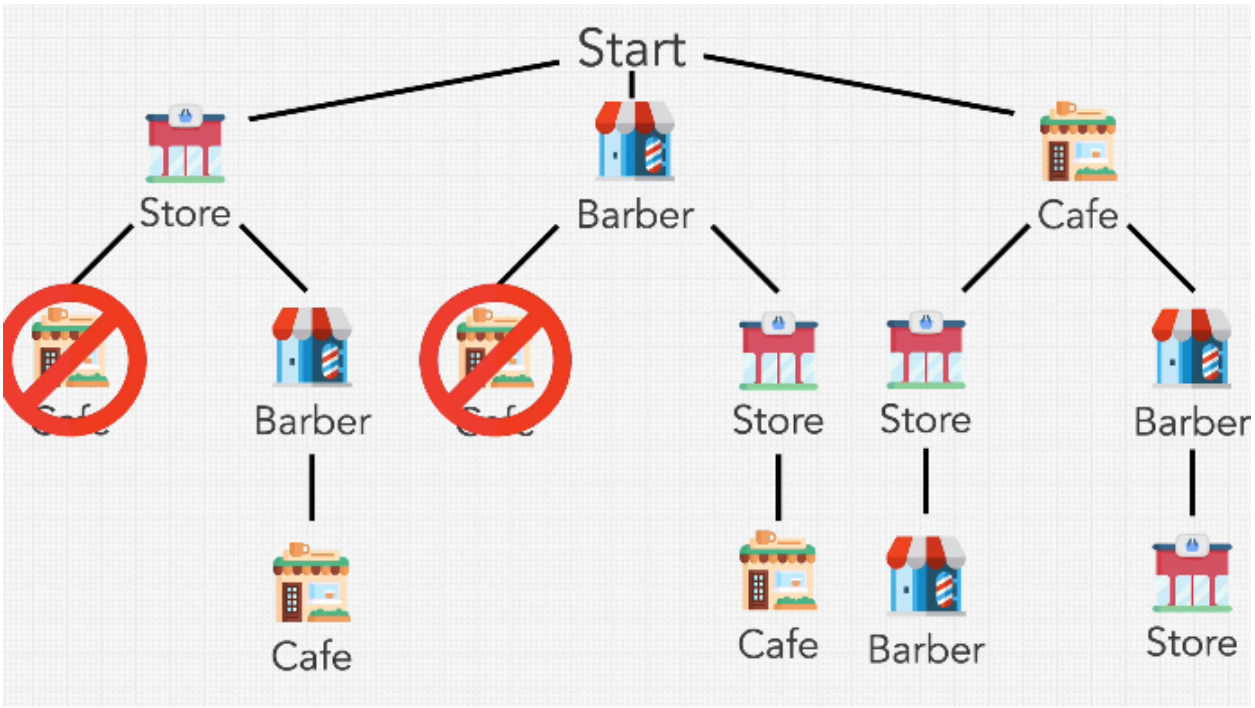


Example: we need to travel to three places





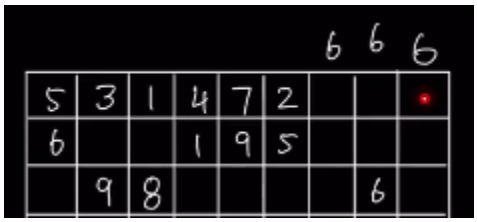
Example if constraint if cafe is the second step backtrack to where we have a valid solution



Backtracking is very difficult, and the abstract is the most important part

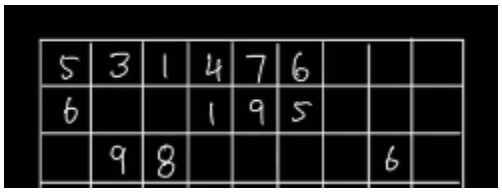
How to recognize a backtracking problem

All solutions, one valid solutions or one of all solutions



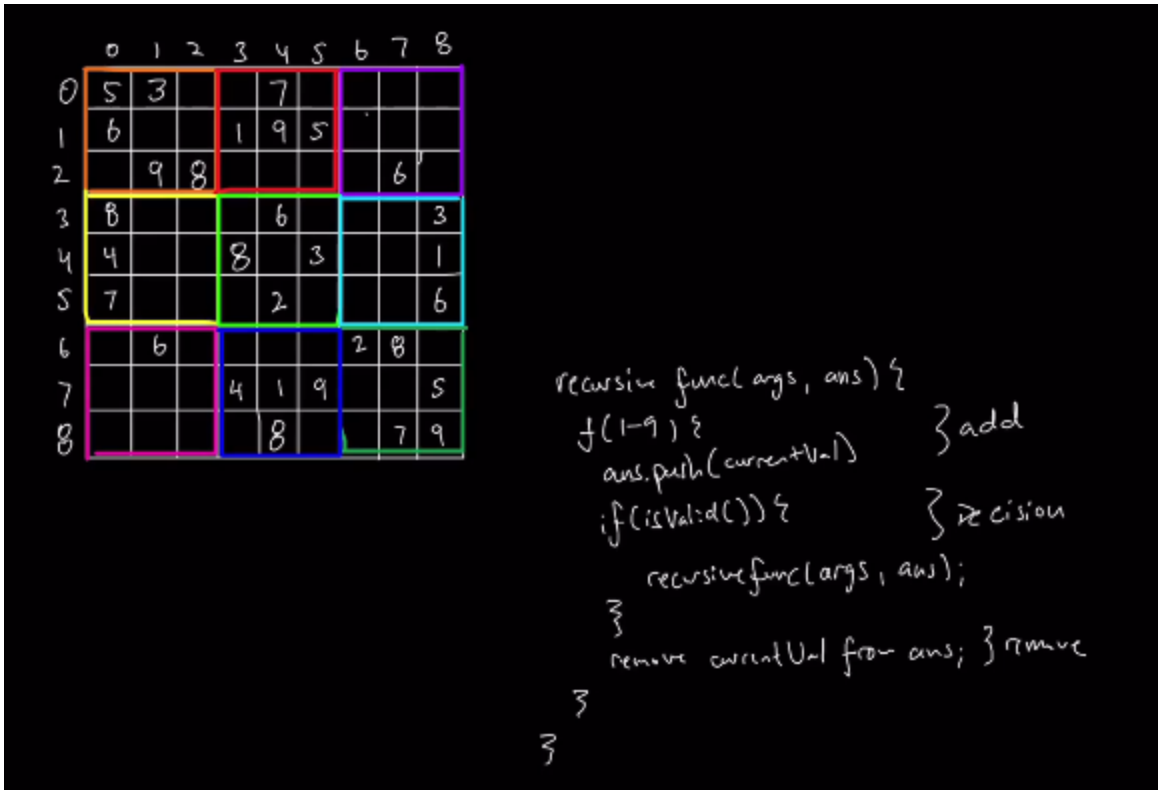
In this case exists an error

So we back track to another valid solution

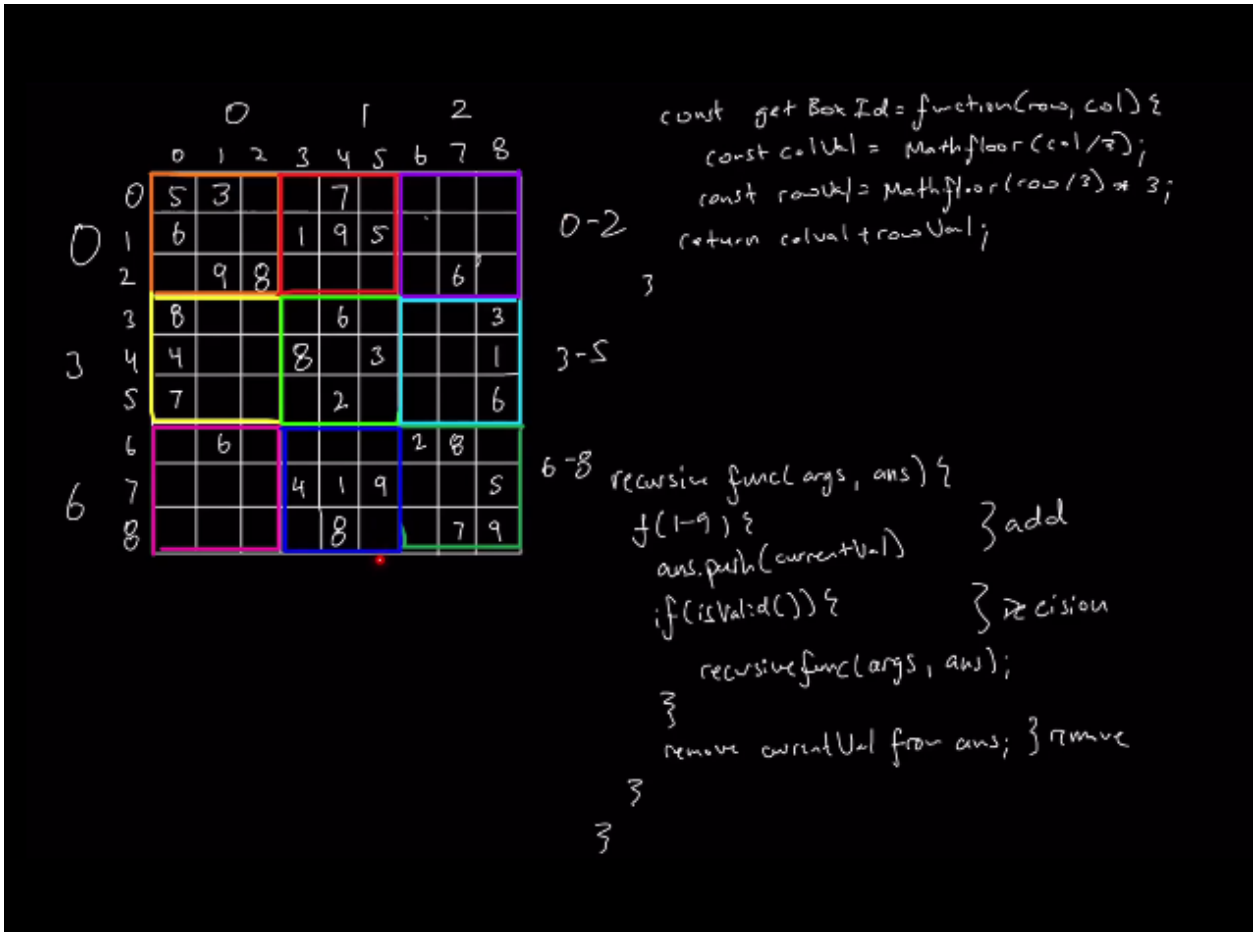


Sometimes the back track is more extensible not just one step

But backtracking doesn’’t cares



This is a general schema of backtracking



Keep track of all necessary